

Würzburger Institut für Verkehrswissenschaften
(Institute for Traffic Sciences)



Robert-Bosch-Straße 4, 97209 Veitshöchheim
Kontakt: silab@wiw.de

SILAB[®] **Referenz Daten-** **basis SCN^X**

Version 7.1

Stand: September 2022

INHALT

1	ÜBERBLICK.....	5
1.1	Was ist SCNX?	5
1.2	Zusammenhang mit den anderen SILAB-Modulen	8
1.3	Wichtige Begriffe	8
2	DEFINITION DER DATENBASIS FÜR SCNX	10
2.1	Notation	10
2.2	Überblick	10
2.3	Karten und Module	10
2.3.1	Instanzen von Modultypen	11
2.3.2	Ablauf von Modulinstanzen	12
2.3.3	Beispiel	13
2.4	Modultypen	14
2.4.1	Reine Area2-Module	14
2.4.2	Modultypen, in denen Courses und Area2s gemischt werden	16
2.5	Streckentyp Course	25
2.5.1	Streckendefinition	25
2.5.2	Lageplan	27
2.5.3	Höhenplan	30
2.5.4	Landschaftstypen	30
2.5.5	Querschnittsprofil (CrossSections)	31
2.5.6	Grafische Objekte (Objects)	36
2.5.7	Schilder als grafische Objekte	38
2.5.8	Weitere Hinweise zu grafischen Objekten	41
2.5.9	Verkehrsregeln (TrafficRules)	41
2.5.10	Zusätzliche Objektbänder	43
2.5.11	Zusätzliche, wiederholte Objekte	44
2.5.12	Points of Interest (POI)	45
2.5.13	Landschaftsgeneratoren	46
2.5.14	Topographie: Form und Höhe der Landschaft	56
2.5.15	OBS-IDs	57
2.5.16	Sichtbarkeitsradien	58
2.5.17	GPS-Koordinaten	59
2.6	Streckentyp Area2	59
2.7	Strecken-Events (Hedgehogs)	60
2.8	Aufsetzpunkte (SetupPoints)	61

3	HILFSPROGRAMME	63
3.1	SILABCFGXCheck	63
3.2	SILABPreview	63
4	VERWALTUNG VON SZENARIENPAKETEN MIT SILABSESSIONEDIT2	68
4.1	Überblick	68
4.2	Neue Szenarien-Datenbank anlegen	68
4.3	Bestehende Szenarien-Datenbank sichern/rücksichern	69
4.4	Neues Szenario der bestehenden Szenarien-Datenbank hinzufügen	69
4.5	Szenario einer bestehenden Szenarien-Datenbank bearbeiten	74
4.6	Szenario einer bestehenden Szenarien-Datenbank löschen	75
4.7	Eine geänderte Szenarien-Datenbank auf einem Simulator installieren	76
5	DPU-KONFIGURATION, ANPASSUNGEN UND ERWEITERUNGEN	77
5.1	Konfiguration der DPUs	77
5.1.1	DPUSCNX	77
5.1.2	Initialisierung der DPUSCNX	77
5.1.3	Bedingte Verknüpfungen	79
5.1.4	Ausgänge der DPUSCNX	79
5.1.5	Visualisierung	79
5.2	Konfiguration von SCNX („SCNX Params“)	79
5.2.1	Definition von Straßenbelägen für Courses	80
5.2.2	Definition von Overlay-Texturen	81
5.2.3	Definition von Bebauungstypen	84
5.2.4	Konfiguration der Landschaftsgenerierung	88
5.2.5	Konfiguration der Schildergenerierung	91
5.2.6	Andere Parameter	94
5.3	Konfiguration von SCNXSGE („SCNXSGE Params“)	95
5.3.1	SCNXSGE CourseLaneCell	96
5.3.2	SCNXSGE CourseParams	97
5.3.3	SCNXSGE Ground	97
5.3.4	SCNXSGE Panorama	97
5.3.5	SCNXSGE House	98
5.3.6	SCNXSGE SignGeneration	98
5.3.7	SCNXSGE Debug	98
6	REFERENZ	99
6.1	Referenz der Standard-Straßenbeläge	99

6.2	Referenz der Standard-Overlaytexturen	100
6.3	Referenz der Standard-Straßenbebauungen	102
6.4	Referenz der Panorama-Texturen	105
6.5	Referenz der Verkehrsschilder/-regeln	107

1 ÜBERBLICK

1.1 Was ist SCNX?

SCNX ist die Datenbasis für die SILAB-Fahrsimulation. Die Datenbasis enthält sowohl Informationen über den Verlauf, die Gestaltung und den Aufbau des befahrbaren Streckennetzes als auch über Aussehen und Höhenprofil der Umgebung.

Im Unterschied zu den meisten anderen Fahrsimulatoren verwendet SILAB keine feste Datenbasis, in der getrennte Beschreibungen für die grafische Darstellung und die logische Struktur von Straßennetzwerk und Umgebung vorliegen.

Stattdessen wird die Datenbasis in kleine **Module** aufgeteilt, die einzelne Situationen (oder „Szenarien“) beschreiben. Diese können flexibel, in Abhängigkeit vom Fahrerverhalten oder vom Wunsch des Instructors, während des Versuchsablaufs zusammengestellt („verknüpft“) werden. Dabei ist es zulässig, Module auch so zu verknüpfen, dass keine global geometrisch konsistente Struktur des Straßennetzwerks entsteht. Beispielsweise kann der Fahrer über eine Zwischenstrecke zu einer Kreuzung zurückgeführt werden, ohne dass er wirklich einen Kreis gefahren ist (siehe Abbildung 6). Nur innerhalb des Sichtbereichs des Fahrers muss stets ein geometrisch konsistentes Straßennetzwerk vorhanden sein. Hierdurch kann eine höhere Flexibilität bei der Versuchsgestaltung und eine schnellere Abfolge relevanter Situationen erreicht werden.

SILAB unterstützt zwei grundsätzlich verschiedene Arten von Streckentypen, die als **Course** und **Area2** bezeichnet werden.

Mit Hilfe von *Courses* können leicht längere Straßenabschnitte mit gleich bleibendem Straßenquerschnitt realisiert werden, die eine abwechslungsreiche Umgebung (Berge, Wälder, Felder und vereinzelte Häuser) besitzen. Dabei wird die Umgebung anhand weniger anzugebender Parameter automatisch von SCNX generiert. Sie werden als Textdatei (Script) mit einem geeigneten Texteditor¹ entworfen. Um den Entwurf zu erleichtern, können in den Editor Funktionen wie eine Syntaxprüfung (siehe Kap. 3.1) und Voransicht (siehe Kap. 3.2) eingebunden werden.

¹ Hier ist ein Plain-Text-Editor wie Geany, TextPad oder der Crimson Editor gemeint.



Abbildung 1 Beispiel eines Streckenstücks vom Typ Course

Area2-Streckenstücke kommen zum Einsatz, wenn sich der Straßenquerschnitt ändert (z.B. bei einer Fahrbahnverengung). Sie können aber auch zur Darstellung kompletter Kreuzungen, komplexer Verkehrsknotenpunkte, Autobahnauffahrten und ganzer innerstädtischer Häuserblöcke verwendet werden. Diese können beliebig mit grafischen Objekten ausgestaltet werden und mit Verkehrsdefinitionen angereichert werden. Wegen der hohen Komplexität von Area2-Modulen werden diese mit dem grafischen Editor SILABAEEdit entworfen.



Abbildung 2 Beispiel eines Area2-Streckenstücks

In beiden Fällen wird dabei ein *Single-Source-Konzept* beim Szenario-Entwurf verfolgt: Aus derselben Szenariobeschreibung (Script bzw. in SILABAEEdit erstelltes Modul) wird sowohl die logische Definition des Szenarios, welche Grundlage für die Verkehrssimulation und die Simulation der Fahrdynamik ist, und die grafische Darstellung für die Visualisierung erzeugt. Dadurch sind Inkonsistenzen von vornherein ausgeschlossen.

Die Datenbasis SCNX der Fahrsimulation SILAB bietet unter anderem die folgenden Funktionen:

- **Straßenquerschnitte:** Die Anzahl der Spuren einer Straße, deren Breite und relative Höhe sowie Fahrbahnhöhen und Bebauungen am Rand bzw. auf den Fahrspuren können festgelegt werden.
- **Lage- und Höhenplan von Straßen:** Für Straßen kann der Streckenverlauf in der Ebene sowie der Höhenverlauf entlang der Straße festgelegt werden.
- **Landschaftshöhen:** Die Landschaft, die das Straßennetzwerk umgibt, hat ein Höhenprofil; es können Berge und Hügel erzeugt werden. Ab der Version SILAB 2.0 können auch Bergstrecken und Brücken dargestellt werden. Die Genauigkeit des Höhenmodells wurde deutlich erhöht, so dass nun auch kleine Unebenheiten dargestellt werden können. Die Landschaft kann vollständig befahren werden.

- **Automatische Landschaftsgenerierung:** Es ist möglich, jeder Seite einer Straße einen Landschaftstypen zuzuweisen und dessen Aussehen über einige wenige Parameter zu beeinflussen. Während der Simulation werden automatisch die entsprechenden Landschaftsobjekte, z.B. Bäume oder Häuser, platziert. Des Weiteren kann sich das Aussehen einer Landschaft jedes Mal verändern, wenn ein Straßenabschnitt für den Fahrer sichtbar wird.
- **Stark verbesserte Verkehrsknoten:** Statt der Area-Elemente werden ab Version 2.0 Area2-Elemente verwendet, die komfortabel mit dem grafischen Editor SILABAEEdit (vgl. Dokument SILABAEEdit) erstellt werden können.
- **Innerorts-Szenarien:** Mit Area2-Elementen kann eine Stadt-Umgebung mit Häuserzeilen, komplexen Kreuzungen mit Sperrflächen, und einer Vielzahl von grafischen Objekten aufgebaut werden.
- **Bedingte Verknüpfungen:** Die Abfolge von Strecken im Straßennetzwerk kann von Bedingungen an Variablen anderer DPU's der Simulation abhängig gemacht werden.

1.2 Zusammenhang mit den anderen SILAB-Modulen

Die Informationen der Datenbasis SCNX werden von der Visualisierung SCNXSGE genutzt, um automatisch aus der Beschreibung des Streckennetzes und der Umgebung eine grafische Repräsentation zu erzeugen, die dem Fahrer präsentiert wird. Die Verkehrssimulation TRFX und die Fußgänger-Simulation MOV greifen ebenfalls auf die Datenbasis SCNX zu, um autonomen Verkehr und Fußgänger an der richtigen Stelle in die Simulation einzufügen. Dazu muss die Definition der Datenbasis allerdings um Anweisungen für die Verkehrssimulation (siehe „Referenz Verkehrssimulation TRFX“) und für die Fußgängersimulation (siehe „Referenz Fußgängersimulation MOV“) erweitert werden.

In den SILAB Entwicklerpaketen steht die Schnittstelle iSCNX zur Verfügung, die aus eigenen, in C++, Java oder Ruby implementierten DPU's heraus detaillierten Zugriff auf das Streckennetz erlaubt. Die Objekte der Datenbasis werden auch in der Objekt-Datenbank ODB abgebildet, die vom SILAB Entwicklerpaket aus einen einfachen Zugriff auf Objekte in einem bestimmten Umfeld des Fahrzeugs ermöglicht und dadurch die Simulation von Assistenzsystemen erleichtert.

1.3 Wichtige Begriffe

In diesem Handbuch werden folgende wichtige Begriffe verwendet:

- **Streckenstück** oder auch **Strecke:** Die kleinste Einheit eines Streckennetzes. Folgende Strukturen sind Streckenstücke:
 - ein durchgängiger Abschnitt einer Straße (Landstraße, Autobahn usw.)
 - eine einzelne Kreuzung
 - eine Autobahnauf- bzw. Abfahrt
 - eine Spurverbreiterung/ -verengung
 - ein Stück Straße, in dem Spuren dazu kommen oder wegfallen
 - ein Teil einer Ortschaft mit mehreren (u.U. sehr komplexen) Kreuzungen und Verbindungsstraßen.
- **Streckennetz:** Streckenstücke werden zu einem Netz von Straßen verknüpft.
- **Verkehrsdefinition:** Vorschriften, wie sich die Fahrzeuge der Verkehrssimulation auf einem vorgegebenen Streckennetz verhalten sollen.

- **Fußgängerdefinition:** Vorschriften, wie sich die Fußgänger auf einem vorgegebenen Streckennetz verhalten sollen.
- **Modul** oder auch **Szenario:** Ein Modul bildet eine abgeschlossene Einheit aus einem Streckennetz und einer Verkehrsdefinition.
- **Modul-Abfolge:** Die Verknüpfung mehrerer Module in einer bestimmten Reihenfolge, wie sie während der Simulation dem Fahrer dargeboten werden.
- **Aufsetzpunkt:** Eine Stelle im Streckennetz eines Moduls, an der die Simulation gestartet werden kann.
- **Karte:** Eine Modul-Abfolge und eine Menge von Aufsetzpunkten.

2 DEFINITION DER DATENBASIS FÜR SCNX

2.1 Notation

Die Syntax der Datenbasisdefinition wird mit Grammatiken beschrieben. Dabei werden folgende Konventionen verwendet:

<i><Kursive Namen></i>	sind Bestandteile und Unterabschnitte der Datenbasis-Definition und werden in weiteren Grammatiken näher erläutert.
Fettgedrucktes { ... };	ist Text, der wortwörtlich in die Konfigurationsdatei übernommen werden muss. Bitte achten Sie auch auf die Groß- und Kleinschreibung.
(1...*) (und Ähnliches)	sind Wiederholungsangaben. Diese beschreiben, dass ein Bestandteil der Konfiguration je nach Bedarf unterschiedlich oft auftreten kann. Das Beispiel (1...*) bedeutet, dass der Bestandteil mindestens einmal auftreten muss, aber keine Obergrenze für die Anzahl existiert.
Vorangestellte (,) (u.Ä.)	gibt an, dass Bestandteile beliebig wiederholt werden können und in der Liste mit Kommata getrennt werden.

2.2 Überblick

In einer Konfigurationsdatei von SILAB (siehe auch „SILAB Konfiguration und Bedienung“) wird die Datenbasis durch den folgenden Abschnitt definiert:

Grammatik 1 Integration einer Datenbasis-Definition

```

<Datenbasis-Abschnitt> →
    <SCNXSGE Konfiguration(→Kap.5.3 Seite 95)>
    SILAB SCN
    {
        <SCNX Konfiguration(→Kap. 5.2 Seite 79)>
        <Datenbasis-Definition(→Grammatik 2 Seite 11)>
    };

```

2.3 Karten und Module

Die Definition einer Datenbasis besteht aus einer oder mehreren Karten (im Skript: **Map**). Eine Karte hat einen Namen und repräsentiert eine bestimmte Abfolge von Modulen. Der Parameter **Map** der DPUSCNX (vgl. Dokument DPUSCNX) legt fest, welche Karte für den Versuch verwendet wird.

Jede Karte besteht aus Modulen. Ein Modul kapselt inhaltlich abgeschlossene Elemente eines Streckennetzes. Beispielsweise können einzelne Situationen durch Module repräsentiert werden. Die Module einer Karte werden miteinander verknüpft. Dadurch wird der Ablauf einer Simulatorfahrt festgelegt.

Es kann vorkommen, dass zwei Karten einer Datenbasis-Definition dieselben Module verwenden. Eine Karte könnte beispielsweise zum Training bestimmter Szenarien benutzt werden, während der eigentliche Versuch in einer anderen Karte enthalten ist. Deswegen sind die Module einer Karte Instanzen von sog. Modultypen. Sie werden im Skript außerhalb der Karten, die sie verwenden, definiert. Ein Modultyp kann in einer Karte mehrfach instanziiert werden, was einer Wiederholung einer Situation an verschiedenen Stellen des Ablaufs entspricht. Der Mechanismus ähnelt dem höherer Programmiersprachen: Die Modultypen entsprechen Datentypen, von denen (u. U. mehrfach) Instanzen erzeugt werden.

Eine Datenbasis-Definition hat also folgenden grundlegenden Aufbau:

Grammatik 2 Grundlegender Aufbau einer Datenbasis-Definition

```

<Datenbasis-Definition>→
  (1...*)<Querschnittsprofil (→Abschnitt 2.5.5)>
  (1...*)<Modultyp>
  (1...*)<Karte>
<Modultyp>→
  define Module <Name Modultyp>
  {
    <Definition Modultyp (→Grammatik 5 Seite 16)>
  }; |
  <Einbindung Area2 als Modul (→Abschnitt 2.4.1)>
<Karte>→
  Map <Name Karte>
  {
    (1...*)<Modulinstantz (→Grammatik 3 Seite 11)>
    <Modulablauf (→Grammatik 4 Seite 12)>
    <Aufsetzpunkte (→Grammatik 24 Seite 62)>
  };

```

Auf die Aufsetzpunkte wird im Abschnitt 2.8 eingegangen.

2.3.1 Instanzen von Modultypen

Eine Instanz eines Modultyps erhält einen Namen, mit dem sie bei der Festlegung des Modulablaufs adressiert wird.

Grammatik 3 Modul-Instanziierung und Modulablauf

```

<Modulinstantz>→
  <Name Modultyp> <Name Modulinstantz> ;

```

Von einem Modultyp können mehrere Instanzen erzeugt werden.

2.3.2 Ablauf von Modulinstanzen

Wenn mehrere Instanzen von Modultypen in einer Karte erzeugt werden, dann muss ihre Abfolge spezifiziert werden. Hierzu werden bereits in einem Modultyp Strecken, die aus dem Modul herausführen, als Anschlussstellen für andere Module festgelegt. Eine solche Anschlussstelle heißt „Port“ eines Moduls. Ein Port mit Nummer i einer Modul-Instanz **mod** wird mit **mod.Port*i*** bezeichnet. Die Definition solcher Ports wird in Abschnitt 2.4.2.1 erläutert.

Grammatik 4 Modulablauf

```

<Modulablauf> →
    Connections =
    {
        (,) <Modulverknüpfung>
    };
<Modulverknüpfung> →
    <einfache Modulverknüpfung>
    <bedingte Modulverknüpfung>
<einfache Modulverknüpfung> →
    <Name Modulinstanz>.<Port> <-> <Name Modulinstanz>.<Port>
<bedingte Modulverknüpfung> →
    <Name Modulinstanz>.<Port> <-> CondVar<Nummer>
    (
        (,) <Bedingung>
    )
<Port> →
    Port<Nummer>
<Bedingung> →
    <Operator> <Zahl> : <Name Modulinstanz>.<Port>
<Operator> →
    default | = | < | > | <= | >= | !=

```

Wie obige Grammatik zeigt, gibt es zwei Möglichkeiten, Modulinstanzen zu einem Ablauf zu verknüpfen:

1. Einfache Modulverknüpfungen sind von der Art

<Name Modulinstanz>.<Port> <-> <Name Modulinstanz>.<Port>

beispielsweise

mod1.Port3 <-> mod4.Port8

Eine solche Verknüpfung bedeutet, dass der Fahrer vom Port3 von mod1 immer zum Port8 von mod4 gelangt.

2. Bedingte Modulverknüpfungen, z.B.:

```
mod1.Port3 <-> CondVar4
(
  >= 4 : mod2.Port1,
  >= 2 : mod3.Port5,
  default : mod4.Port2
)
```

In diesem Beispiel wird während der Simulation anhand des Wertes der Variablen `CondVar4` entschieden, was an `mod1.Port3` angeschlossen wird. Ist z.B. `2 <= CondVar4 < 4`, dann wird `mod3.Port5` mit `mod1.Port3` verknüpft.

Bei der Verknüpfung von Modulinstanzen zu einem Ablauf sind einige Punkte zu beachten. Da der Mechanismus – insbesondere bei bedingten Verknüpfungen – völlig analog zu den Verknüpfungen von Streckeninstanzen innerhalb eines Moduls funktioniert, wird darauf in Abschnitt 2.4.2.2 näher eingegangen.

Bemerkung: Innerhalb einer Karte darf es beliebig viele **Connections = { ... }**; Abschnitt geben.

2.3.3 Beispiel einer aus zwei Modulen zusammengesetzten Karte

Die Definition einer Karte soll anhand eines einfachen Beispiels verdeutlicht werden. Die Datenbasis besteht aus zwei Szenarien, jedes davon wird in einem eigenen Modultyp definiert. Im ersten (Modultyp `Situation_1`) muss der Fahrer eine Kreuzung durchfahren, im zweiten (Modultyp `Situation_2`) hat er eine Car-Follow-Aufgabe zu bewältigen. In einer Karte `Versuch` wird deswegen von jedem dieser Modultypen eine Instanz angelegt (`Sit1` bzw. `Sit2`). Die beiden Instanzen werden über Ports, die in der Definition der Modultypen festgelegt werden (vgl. Kapitel 2.4.2.1) verbunden. Die Teile der Definition, die an späterer Stelle in diesem Kapitel erläutert werden, sind durch „...“ gekennzeichnet.

```
define Module Situation_1
{
  ...
};

define Module Situation_2
{
  ...
};

Map Versuch
{
  Situation_1 Sit1;
  Situation_2 Sit2;
  Connections =
  {
    Sit1.Port2 <-> Sit2.Port1
  };
  ...
};
```

Anhand einer weiteren Karte `Versuch2`, die sich im selben Skript wie obige Karte `Versuch` befindet, soll demonstriert werden, wie Modultypen mehrfach instanziiert werden:

```
Map Versuch2
{
  Situation_1 Sit1;
  Situation_2 Sit2;
  Situation_1 Sit3;
  Situation_2 Sit4;
  Connections =
  {
    Sit1.Port2 <-> Sit2.Port1,
    Sit2.Port2 <-> Sit3.Port1,
    Sit3.Port2 <-> Sit4.Port1
  };
  ...
};
```

Beim Durchfahren der Karte `Versuch2` muss der Fahrer zunächst die Kreuzung (`Sit1`) meistern, dann die Car-Follow-Aufgabe (`Sit2`), dann ein weiteres Mal die Kreuzung (`Sit3`) und schließlich eine zweite Instanz der Car-Follow-Aufgabe.

2.4 Modultypen

2.4.1 Reine Area2-Module

Mit dem Editor SILABAEEdit erzeugte Area2-Szenarien können als kompletter Modultyp exportiert werden². Dies empfiehlt sich immer dann, wenn die Area2 ein abgeschlossenes (z.B. Innenstadt-) Szenario darstellt. Dadurch werden Verkehrs- und Fußgängerdefinition automatisch für dieses Modul integriert. Es müssen die rot umrandeten Einstellungen in SILABAEEdit vorgenommen werden:

² Alternativ kann ein Area2-Szenario auch als Streckenabschnitt exportiert werden, siehe Kap. 2.5.10.

Abbildung 3 Export eines Area2 als Modul

Das exportierte Modul kann nun wie ein von Hand im Skript definiertes eingebunden werden, in dem zunächst die angegebene CFG-Datei inkludiert wird:

```
SILAB SCN
{
  ...
  include SILABDemo\Kreuzung_4_4_2_2.cfg
```

Das Modul kann dann unter seinem angegebenen Modultyp-Namen instanziiert und in die Karte eingebettet werden. Da die in SILABAEEdit nach dem Export angezeigten Ports des Area2 auf identisch benannte Modulports übertragen werden, können diese zur Verknüpfung verwendet werden.

```
Map Map1 {
  Kreuzung42 k1;
  ...
  Connections = {
    k1.Port1 <-> ...
  };
};
```

2.4.2 Modultypen, in denen Courses und Area2s gemischt werden

Durch Modultypen werden inhaltlich abgeschlossene Teile eines Straßennetzwerks repräsentiert. Grundlage hierfür ist ein im Modultyp definiertes Streckennetz, das sich aus einzelnen Streckenstücken zusammensetzt. Die Streckenstücke werden auch hier zunächst als Typen definiert, von denen Instanzen erzeugt werden. Durch die Verknüpfung dieser Instanzen wird das Streckennetz festgelegt. Wie bei den Modultypen und –instanzen im vorhergehenden Abschnitt bringt dies den Vorteil, dass einzelne Streckenstücke in einem Modultyp mehrfach verwendet werden können. Um zu gewährleisten, dass die Definition eines Modultyps in sich abgeschlossen und damit leicht wieder verwendbar ist, müssen die Typen für die Streckenstücke *innerhalb* des Modultyps spezifiziert werden. Ein bestimmter Streckentyp kann also nicht in mehreren Modultypen verwendet werden.

Grammatik 5 Definition eines Modultyps

```
<Definition Modultyp>→
  define Module <Name Modultyp>
  {
    ModuleID = <Nummer> ;
    (1...*)<Streckentyp>(→Grammatik 6 Seite 17)>
    (1...*)<Streckeninstanz>(→Grammatik 6 Seite 17)>
    (0..*)<Streckennetz>(→Grammatik 6 Seite 17)>
    (0..*)<Events Streckeninstanzen>(→Grammatik 23 Seite 61)>
    (0,1)<Verkehrsdefinition>(→Referenz Verkehrssimulation TRFX)>
  };
```

Der Teil *<Events Streckeninstanzen>* in obiger Grammatik wird in Abschnitt 2.7 behandelt. Die Beschreibung der *<Verkehrsdefinition>* findet sich im Dokument „Referenz Verkehrssimulation TRFX“.

Bei der Definition muss für den Modultyp eine eindeutige Kennung (**ModuleID**) vergeben werden. Zusammen mit einem Zähler für die Instanzen eines Moduls können damit Abschnitte eines Versuchs identifiziert werden³. Diese Werte stehen als Ausgänge *ModuleTypeID* und *ModuleInstanceID* der DPUSCNX zur Verfügung. Die Aufzeichnung dieser Werte als Spalten der DPUDataFile ermöglicht es beispielsweise, eine umfangreiche Datei mit Messwerten in die einzelnen Module zu zerlegen. Die statistische Auswertung der Teildateien kann dann Szenario-bezogen erfolgen.

Es gibt zwei unterschiedlichen Arten von Streckentypen:

- **Course:** Straße mit einheitlichem Querschnittsprofil ohne Abzweigungen.
- **Area2:** Kreuzungen, Kreisverkehre und Übergänge zwischen Straßen mit unterschiedlichem Querschnittsprofil (z.B. Auffahrten auf Autobahnen) sowie innerstädtische Gebiete.

Die folgende Grammatik zeigt, wie Streckentypen definiert bzw. instanziiert werden und wie aus den Instanzen Streckennetze aufgebaut werden.

³ Für die ModuleID sind positive Zahlen im Wertebereich bis maximal $2^{31}-1$ (2147483647) zulässig.

Grammatik 6 Streckentypen , -instanzen und -netze

```

<Streckentyp> →
    <Streckentyp Course> |
    <Streckentyp Area2>
<Streckentyp Course> →
    define Course <Name Streckentyp>
    {
        <Definition Course (→ Grammatik 8 Seite 26)>
    };
<Streckentyp Area2> →
    include <AreaVerzeichnis>/<AreaName>_Area2.cfg (→ Kap. 2.4.2.3)
<Streckeninstanz> →
    <Name Streckentyp> <Name Streckeninstanz>;
<Streckennetz> →
    Connections =
    {
        (,)[<Streckenverknüpfung>|<Definition Modulport>]
    };
<Streckenverknüpfung> →
    <einfache Streckenverknüpfung(→ Grammatik 7 Seite 18)> |
    <bedingte Streckenverknüpfung(→ Grammatik 7 Seite 18)>
<Definition Modulport> →
    <Port Modulinstanz> <-> <Name Streckeninstanz>.<Port>
<Port Modulinstanz> →
    Port<Nummer>

```

Auch bei Strecken kann, ähnlich wie bei der Instanziierung von Modultypen, ausgenutzt werden, dass von einem Streckentyp mehrere Instanzen erzeugt werden können.

Bemerkung: Es darf innerhalb eines Modultyps beliebig viele **Connections = { ... };** Abschnitte geben.

2.4.2.1 Definition Modulport

In Abschnitt 2.3 wurde gezeigt, wie Modulinstanzen über ihre Ports zu einem Ablauf verknüpft werden können. Bei der Verknüpfung zweier Modulinstanzen müssen auch ihre jeweiligen Streckennetze verknüpft werden. Hierzu müssen Ports als Anschlussstellen von Modultypen definiert werden. Modulports heißen **Port1**, **Port2** usw. Jedem dieser Ports wird ein freier Port einer Streckeninstanz zugewiesen. Wie in Grammatik 6 (S. 17) zu sehen ist, erfolgt dies im **Connections = { ... };** – Abschnitt.

Im Modul `Situation_1` des Beispiels aus Abschnitt 2.3.3 werden die Modulanschlüsse in der ersten und letzten Zeile der Verknüpfungen definiert:

```

Connections =
{

```

```

Port1 <-> S1.Begin,
S1.End <-> X1.Port4,
X1.Port1 <-> S2.End,
X1.Port3 <-> S4.End,
X1.Port2 <-> S3.Begin,
S3.End <-> Port2
};

```

2.4.2.2 Streckenverknüpfungen

Jede Strecke⁴ hat Ports, über die sie mit anderen Strecken verbunden werden kann. Course-Instanzen haben zwei Ports (**Begin** und **End**), die dem Anfang bzw. dem Ende des Lageplans zugeordnet sind. Area2-Instanzen haben unterschiedlich viele Ports, die mit **Port1**, **Port2** usw. bezeichnet werden und in SILABAEEdit festgelegt werden. Die Streckentypen dürfen beliebig untereinander verbunden werden.

Grammatik 7 Streckenverknüpfungen

```

<einfache Streckenverknüpfung> →
    <Name Streckeninstanz>.<Port> <-> <Name Streckeninstanz>.<Port>
<bedingte Streckenverknüpfung> →
    <Name Streckeninstanz>.<Port> <-> CondVar<Nummer>
    (
        <Bedingung>
    )
<Bedingung> →
    <Operator> <Zahl> : <Name Streckeninstanz>.<Port>
<Operator> →
    default | = | < | > | <= | >= | !=

```

Die Verknüpfung von Strecken zu einem Streckennetz erfolgt unabhängig von deren Lage- und Höhenplan. Betrachtet man also Lage- und Höhenplan von Strecken als deren geometrische Repräsentation, dann wird das Streckennetz auf einer topologischen Ebene beschrieben. Dadurch lassen sich Streckennetze aufbauen, die nur lokal geometrisch konsistent sind. Dies bedeutet, dass in jedem Simulationsschritt nur von dem Teil des Streckennetzes eine maßstabsgetreue Karte angefertigt werden kann, der sich im Sichtbarkeitsbereich des Fahrers befindet. Die folgenden Beispiele sollen die Möglichkeiten, die sich daraus ergeben, verdeutlichen.

Einfache Streckenverknüpfungen

Das Streckennetz im Modultyp `Situation_1` des Beispiels aus Abschnitt 2.3.3 wird so definiert:

⁴ Mit „Strecke“ ist im Folgenden eine Streckeninstanz gemeint.

```

Connections =
{
  ...
  S1.End <-> X1.Port1,
  X1.Port2 <-> S2.End,
  X1.Port3 <-> S4.End,
  X1.Port4 <-> S3.Begin,
  ...
};

```

An alle vier Ports der X-Kreuzung werden Straßen $s_1, \dots, s_4$ angeschlossen. Der Fahrer nähert sich der X-Kreuzung auf der Straße s_1 . Die X-Kreuzung wird automatisch so gedreht und verschoben, dass sie mit dem Port1 „glatt“ an das Ende von s_1 anschließt. Ebenso werden die Straßen $s_2, \dots, s_4$ automatisch „glatt“ an die X-Kreuzung angeschlossen.

Das Streckennetz ist in Abbildung 4 dargestellt.

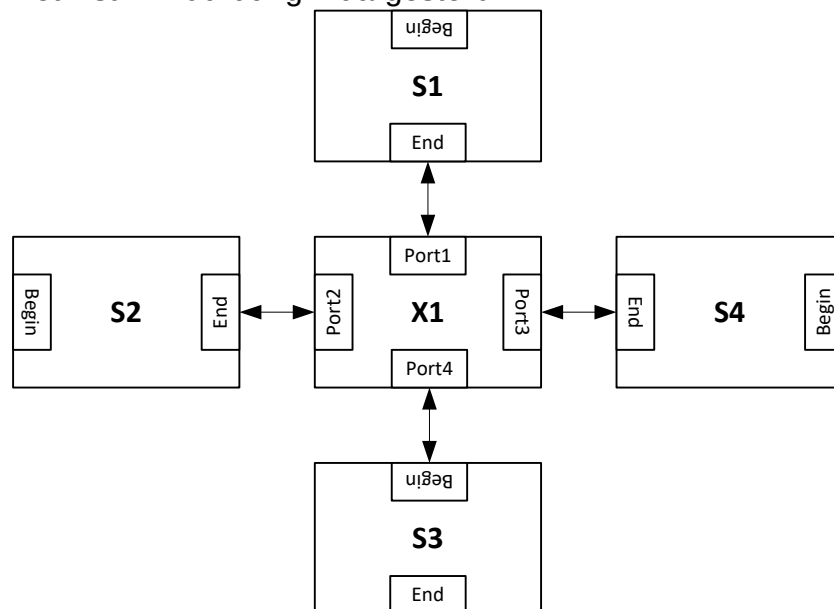


Abbildung 4: Einfaches Streckennetz

Topologische Schleifen

Strecken können zu Schleifen verknüpft werden, obwohl sie geometrisch, d.h. aufgrund ihres Lage- und Höhenplans, keine Schleifen bilden. Solche Schleifen werden als „topologische Schleifen“ bezeichnet. Die einfachste Form besteht aus zwei Straßen (also Instanzen von Course-Typen), im folgenden Beispiel s_1 und s_2 genannt, deren Lageplan jeweils eine Gerade ist. Die Geraden sind folgendermaßen verknüpft:

```

Connections =
{
  s1.End <-> s2.Begin,
  s2.End <-> s1.Begin
};

```

Das Streckennetz ist in Abbildung 5 dargestellt.

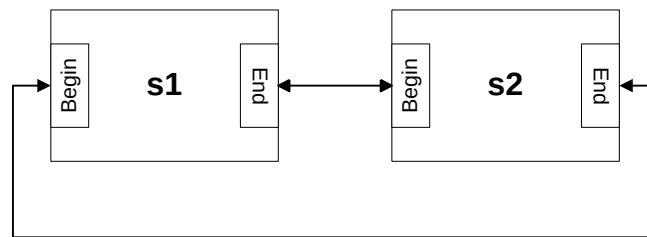


Abbildung 5: Topologische Schleife

Um während der Simulation die geometrische Inkonsistenz aufzulösen, wird die eingeschränkte Sichtweite des Fahrers ausgenutzt. Befindet er sich (von *s1* kommend) auf *s2*, dann ist an einem bestimmten Punkt *s1* nicht mehr im Rückspiegel erkennbar. Die Strecke *s1* kann von ab dieser Position vom Fahrer unbemerkt an das Ende von *s2* angeschlossen werden. Dies geschieht automatisch, sobald der Port *s2.End* für den Fahrer sichtbar wird.

In SILAB wird dies wie folgt umgesetzt: Die Gerade *s2* muss über den Sichtbereich des Fahrers hinausreichen. Nur dann ist gewährleistet, dass *s1* nicht mehr im Sichtbereich des Fahrers liegt. Um dies leicht anhand des Lageplans von Strecken (in denen auch Kurven vorkommen können) erkennen zu können, wird der Sichtbarkeitsbereich folgendermaßen definiert:

- Bei **Courses** bedeutet eine Ausdehnung *S* des Sichtbarkeitsbereichs, dass der Fahrer von seiner aktuellen Position aus *S* Meter nach vorne und nach hinten *entlang der Strecke* sehen kann. Die Länge von *S* beträgt standardmäßig 2 km. Sie kann aber für einzelne Courses über die Parameter *Begin.rVis* bzw. *End.rVis* angepasst werden (siehe Kap. 2.5.16).
- Bei **Area2s** wird der Sichtbarkeitsbereich in SILABAEEdit als Kreis um einen Port mit einem bestimmten Radius angegeben. Wenn sich der Fahrer im Inneren dieses Kreises befindet, gilt der entsprechende Port als sichtbar.

Topologische Verkehrsknotenpunkte

Der zweite, global geometrisch inkonsistente Typ von Streckennetzwerken, der mittels topologischer Verknüpfungen aufgebaut werden kann, heißt „topologischer Verkehrsknotenpunkt“. Solche Konstruktionen kommen dann zum Einsatz, wenn der Fahrer an Kreuzungen zwar freie Richtungswahl hat, der weitere Versuchsverlauf jedoch nicht von einer falschen Richtungswahl gefährdet sein soll. Ein Beispiel für ein solches Streckennetz ist in Abbildung 6 zu sehen.

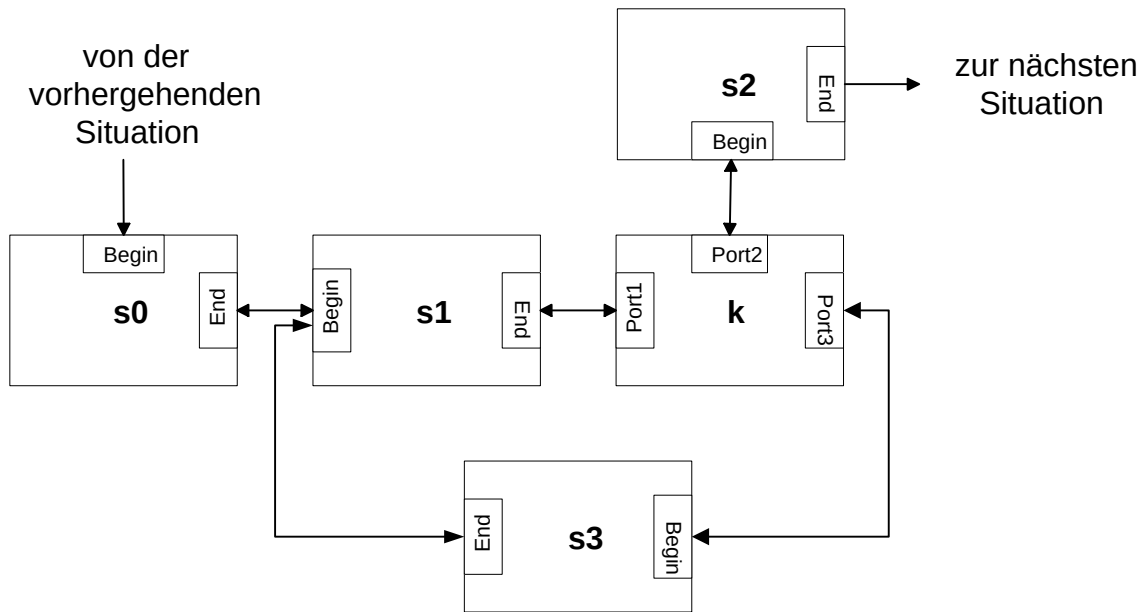


Abbildung 6: Topologischer Verkehrsknotenpunkt

Der Fahrer kommt von den Course-Instanzen s_0 und s_1 und hat die Aufgabe, an der Area2-Instanz k , die eine T-Kreuzung modelliert, links abzubiegen. Wenn er links abbiegt, gelangt er über die Course-Instanz s_2 zur nächsten Situation des Versuchs. Führt er jedoch fälschlicherweise gerade durch die Kreuzung, wird er über die Course-Instanzen s_3 und s_1 an *dieselbe* Kreuzung k zurückgeführt, an der er die Aufgabe wiederholt. Der zugehörige Skript-Ausschnitt für das Streckennetz lautet:

```
Connections =
{
  s0.End <-> s1.Begin,
  s1.End <-> k.Port1,
  k.Port3 <-> s3.Begin,
  k.Port2 <-> s2.Begin,
  s3.End <-> s1.Begin
};
```

Der topologische Verkehrsknotenpunkt befindet sich im Port `Begin` von s_1 . Zwei Ports, `End` von s_0 und `End` von s_3 , sind daran angeschlossen, obwohl sich dort rein geometrisch keine Kreuzung befindet.

Zur Auflösung von topologischen Verkehrsknotenpunkten während der Simulation muss für den Port, der mehrfach verbunden ist, eine eindeutige Verknüpfung selektiert werden. Hierzu wird, sobald sich der Port im Sichtbarkeitsbereich des Fahrers befindet, die Verknüpfung gewählt, die den Port mit der Strecke verbindet, auf der sich der Fahrer gerade befindet. Im Beispiel wäre dies, von s_0 kommend, die Verknüpfung

```
s0.End <-> s1.Begin
```

Nähert sich der Fahrer dem topologischen Verkehrsknotenpunkt auf s_3 , wird die Verknüpfung

```
s3.End <-> s1.Begin
```

ausgewählt. Wenn die Strecke `s1` von `k` aus befahren wird und der Port `Begin` von `s1` sichtbar wird, wird diejenige Verknüpfung selektiert, die gültig war, als der Fahrer zuletzt von `s0` oder `s3` gekommen ist. War dies noch nie der Fall, dann wird die erste im Connections-Abschnitt aufgelistete Verknüpfung gewählt. Auch die Auflösung dieses Verknüpfungstyps stellt eine Bedingung an das Straßennetzwerk: Der topologische Verkehrsknotenpunkt darf zu einem bestimmten Zeitpunkt der Simulation immer nur von einer der zu ihm hinführenden Strecken (in obigem Beispiel die Strecken `s0` oder `s3`) aus sichtbar sein.

Bedingte Streckenverknüpfungen

Die Verknüpfung von Strecken kann, wie Grammatik 7 zeigt, von Bedingungen abhängig gemacht werden. Die Bedingungen werden an die Eingänge `CondVar1` bis `CondVar10` der `DPUSCNX` gestellt. Wenn ein Strecken-Port mit bedingten Verknüpfungen in den Sichtbarkeitsbereich des Fahrers gelangt, wird diejenige Strecke selektiert, deren Bedingung zu diesem Zeitpunkt zutrifft. Es sollte möglichst immer eine Default-Alternative (Operator **default**) angegeben werden, damit das Streckennetz nicht endet, falls keine der Bedingungen zutrifft.

Im folgenden Beispiel soll sich die Streckenführung in Abhängigkeit von der aktuellen Geschwindigkeit des Testfahrers verändern. Hierzu muss der Ausgang `v_kmh` der Fahrdynamik-Schnittstelle `VDyn` (vgl. Dokument „SILAB Konfiguration und Bedienung“) mit dem Eingang `CondVar1` von `DPUSCNX` verknüpft werden. Fährt der Fahrer auf der Straße `s1` schneller als 120 km/h, gelangt er auf eine kurvige Strecke (Instanz `kurvig`), ansonsten auf die Straße `s2`.

```
s1.End <-> CondVar1
(
    > 120 : kurvig.Begin,
    default : s2.Begin
)
```

Bemerkung zur Verknüpfung von Modulinstanzen

Da – wie in Abschnitt 2.4.2.1 beschrieben – jedem Port eines Moduls der Port einer Streckeninstanz zugeordnet wird, entspricht die Verknüpfung von Modulinstanzen im Grunde einer Verknüpfung von Streckeninstanzen. Die folgende Abbildung verdeutlicht diesen Zusammenhang:

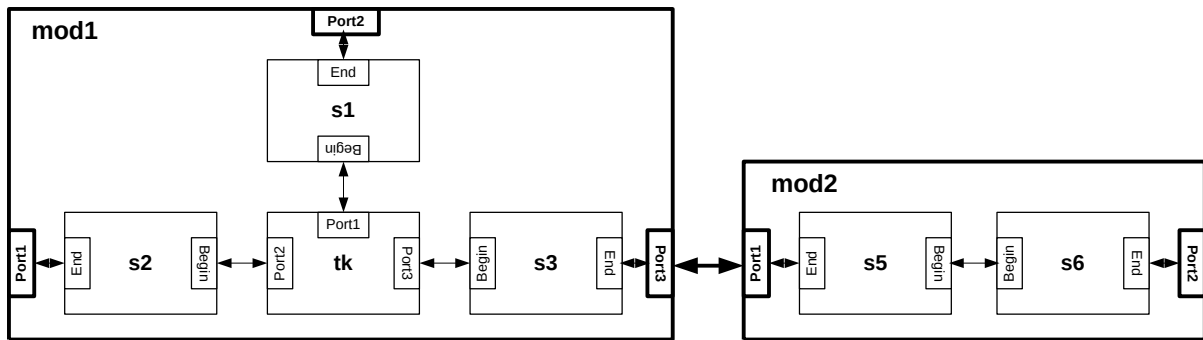


Abbildung 7: Die Modulverknüpfung $\text{mod1.Port3} \leftrightarrow \text{mod2.Port1}$ entspricht der Streckenverknüpfung $s3.\text{End} \leftrightarrow s5.\text{End}$.

Aus diesem Grund können topologische Schleifen, Verkehrsknotenpunkte und bedingte Verknüpfungen auch bei einem Ablauf von Modulinstanzen eingesetzt werden.

Anwendungsbeispiel: Das Streckennetz eines Parcours, bestehend aus den Streckennetzen mehrerer Modulinstanzen, soll eine Schleife bilden. Durch topologische Verkehrsknotenpunkte kann der Ablauf des Parcours, abhängig von der Route des Fahrers in einem bestimmten Modul, in unterschiedliche Teile aufgespaltet werden. Alle Teile des Ablaufs werden mittels eines topologischen Verkehrsknotenpunktes in einem Port *eines* Moduls wieder zusammengeführt.

2.4.2.3 Einbinden von Area2-Streckentypen

Insbesondere für einfache Kreuzungen auf Überlandstrecken bietet sich die direkte Einbindung des mit SILABAEEdit erzeugten Area2 als Streckentyp in ein von Hand als Skript geschriebenes Modul an⁵. Dadurch kann die Kreuzung mit den umliegenden Course-Streckentypen in einem einzigen Modul zusammengefasst werden und der simulierte Verkehr kann in die Kreuzung hinein- und wieder herausfahren⁶. Eine solche Area2 sollte in SILABAEEdit als Streckentyp exportiert werden:

⁵ Wenn die Area2 ein komplexes Innenstadt-Szenario darstellt, empfiehlt sich stattdessen die Einbindung als Modul, siehe dazu Kap. 2.4.1.

⁶ Der simulierte Verkehr findet üblicherweise nur innerhalb des aktiven Moduls statt. Näheres dazu in „Referenz Verkehrssimulation TRFX“.

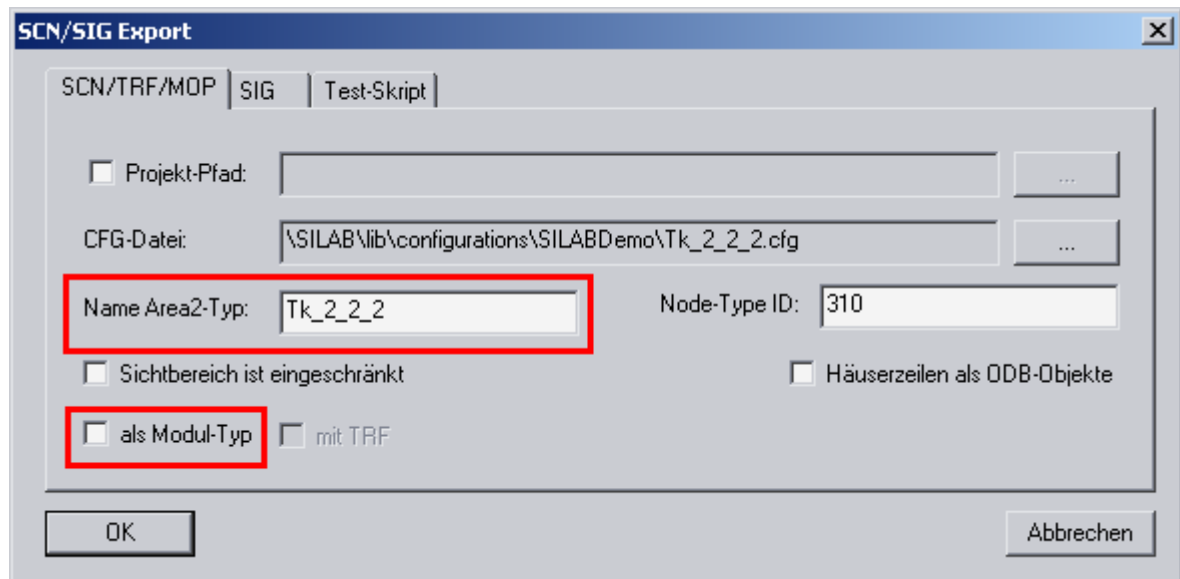


Abbildung 8 Export einer Area2 als Streckentyp

Die Area2 kann nun leicht in ein Modul eingebunden werden. Dies wird anhand eines Beispiels erläutert: Es soll ein Modul erstellt werden, in dem eine T-Kreuzung (siehe Abbildung 8) auf allen 3 Seiten mit dem Course-Typen S1 verbunden wird. Die Endpunkte dieser Straßen bilden die Ports des Moduls. Der Modulaufbau ist in Abbildung 9 dargestellt.

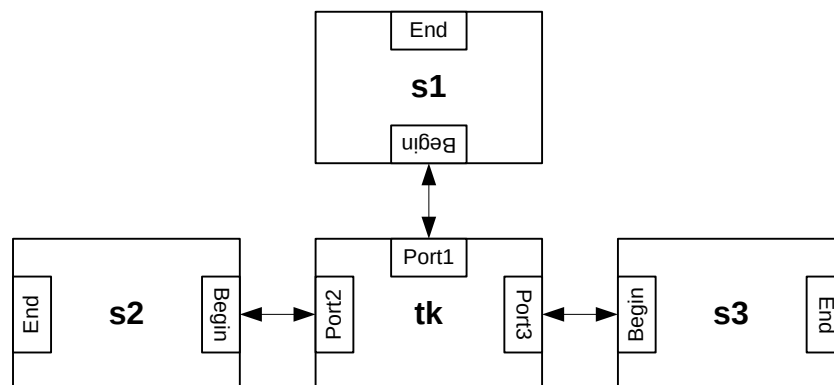


Abbildung 9 Streckenaufbau für Einbindung eines Area2 als Streckentyp

Das Modul kann wie folgt definiert werden:

```
define Module Area2_als_Strecke
{
  define Course S1
  {
    # ... Definition des Anschluss-Stücks, siehe Kap. 2.5
  };

  # Einbindung des von SILABAEEdit exportierten Area-Streckentyps
  include SILABDemo\Tk_2_2_2_Area2.cfg

  # Instanzierung des Area2
  Tk_2_2_2 tk;

  # Dreimalige Instanzierung des Anschluss-Stücks
```



```
S1 s1;
S1 s2;
S1 s3;

# Verbindungen innerhalb des Moduls:
Connections =
{
  # Anschluss an Area2
  tk.Port1 <-> s1.Begin,
  tk.Port2 <-> s2.Begin,
  tk.Port3 <-> s3.Begin,
  # Definition der Modulports
  s1.End <-> Port1,
  s2.End <-> Port2,
  s3.End <-> Port3
};
```

2.5 Streckentyp Course

2.5.1 Streckendefinition

Strecken sind Instanzen von innerhalb eines Modultyps definierten Streckentypen. Es gibt zwei Arten von Strecken (und damit auch Streckentypen): Straßen (Instanzen von Course-Typen, im Folgenden kurz „Courses“ genannt) und Verkehrsknotenpunkte (Instanzen von Area2-Typen, im folgenden kurz Area2's). Dieser Abschnitt stellt die Definition von Course-Typen vor.

Die Definition einer Straße besteht aus folgenden Teilen:

Grammatik 8 Definition eines Straßentyps

```

<Definition Course>→
  NodeID = <Nummer> ;
  CrossSection = <Name Querschnittsprofil> ;
  (0...1) Overlay = {<Overlay>, <Sichtbarkeit>};
  (1...*)<Lageplan-Element>
  <Höhenprofil(→Grammatik 9 Seite 30)>
  <Landschaftstypen(→Grammatik 18 Seite 47)>
  (0...*)<Events Course-Typ(→Grammatik 22 Seite 60)>
  (0...*)<Objekte von Course-Typen(→Grammatik 13 Seite 37)>
  (0...*)<Verkehrsregeln (→Grammatik 14 Seite 42)>
  (0...*)<Zusatz-Objektbänder (→Grammatik 15 Seite 43)>
  (0...*)<Zusatz-Objekte (→Grammatik 16 Seite 44)>
  (0...*)<OBS-IDs (→Grammatik 19 Seite 58)>
  (0...1)<Sichtbarkeitsradien>(→ Grammatik 20 Seite 59)>
};
<Lageplan-Element>→
  Straight(<Länge >); |
  Bend(<Länge>, <Radius>); |
  Bend(<Länge>, <Radius>, <Klothoidenanteil Eintritt>,
  <Klothoidenanteil Austritt>); |
  Bezier(<x Offset Ende>, <y Offset Ende>, <Tang.-Winkel Offset Ende>);

```

Wie die Modultypen werden Typen von Strecken mit einer eindeutigen Kennung (**NodeID**) versehen⁷. Diese Kennung steht, zusammen mit einem Zähler für die Instanzen von Streckentypen einer Modulinstanz, für die aktuell befahrene Strecke in den Ausgängen *NodeTypeID* und *NodeInstanceID* der DPUSCNX zur Verfügung. Werden diese Werte z.B. über die DPUDataFile aufgezeichnet, lassen sich in den Messwerten einzelne Streckenabschnitte identifizieren.

Querschnittsprofil

Bei Courses ändert sich das Querschnittsprofil innerhalb eines Typs nicht. Es wird daher für den ganzen Typ mit der Zuweisung **CrossSection** = ... festgelegt. Auf der rechten Seite der Zuweisung steht der Name des Querschnittsprofils, wie er gem. Kap. 2.5.5 definiert wurde.

Overlay

Normalerweise wird die im Querschnittsprofil (CrossSection) angegebene Overlay-Textur verwendet. Mit der Angabe **Overlay** = ... können die Parameter aber für einen

⁷ Für die NodeID sind positive Zahlen im Wertebereich bis maximal $2^{53}-1$ (9.007.199.254.740.992) zulässig.

speziellen Course individuell angepasst werden. Die Parameter entsprechen der CrossSection-Definition (siehe Kap. 2.5.5). Eine Liste der mitgelieferten Overlay-Texturen ist in Kap. 6.2 zu finden.

2.5.2 Lageplan

Der Lageplan legt den Verlauf der Spuren in der Ebene fest. Aufgrund des konstanten Querschnitts verlaufen bei einer Straße alle Spuren parallel zueinander. Es genügt daher den Verlauf der Straßenmitte zu definieren. Hieraus und aus den im Querschnitt definierten Spurbreiten kann der Verlauf der einzelnen Spuren abgeleitet werden.

Für die Modellierung des Lageplans stehen folgende grundlegende Elemente zur Verfügung:

- **Geraden:** Abschnitte mit einer gerade verlaufenden Fahrbahn. Eine Gerade ist durch ihre Länge definiert und hat die Krümmung 0.
- **Kreisbögen:** Abschnitte, die mit einem konstanten Lenkradeinschlag durchfahren werden können. Kreisbögen werden nach der RAA⁸ bzw. der RAL⁹ über ihre Länge und ihren Radius beschrieben (Tabelle 1 und Tabelle 2). Dabei entspricht ein positiver Radius einer Krümmung nach rechts, ein negativer Radius einer Linkskrümmung. Die Größe der Krümmung ist der inverse Radius. Die Krümmung eines Kreisbogens ist folglich konstant.
- **Übergangsbögen (Klothoiden):** Wäre im Lageplan ein Kreisbogen direkt an ein Geradenstück angeschlossen, würde sich die Krümmung sprunghaft von 0 (Geradenstück) auf eine Krümmung $\neq 0$ im Kreisbogen ändern. Für eine korrekte Spurhaltung müsste in diesem Fall der Fahrer den Lenkwinkel sprunghaft ändern, was in der Realität nicht möglich ist. Deshalb schreiben die RAA bzw. die RAL zwischen Streckenstücken mit unterschiedlicher konstanter Krümmung sog. Übergangsbögen vor. Diese überführen die unterschiedlichen Krümmungen langsam ineinander. Übergangsbögen werden über ihre Länge sowie die Krümmung am Anfang und am Ende parametrisiert.
- **Kubische Bezier-Kurven:** Kurven (und auch S-Kurven), die über ihren Endpunkt sowie den Tangentwinkel im Endpunkt definiert sind.

Der Lageplan wird als eine Folge der Elemente Gerade (**Straight**), Kurve (**Bend**) und Bezier-Kurve (**Bezier**) spezifiziert.

Eine **Kurve (Bend)** fasst dabei drei Elemente zusammen: Sie beginnt mit einem Übergangsbogen, der von der Krümmung 0 zu der eines Kreisbogens überführt. Ein abschließender Übergangsbogen blendet die Krümmung des Kreisbogens wieder auf die Krümmung 0 aus. Für die Definition gibt es zwei Alternativen: (1) Die gesamte Länge der Kurve (in Meter) und der Radius des Kreisbogens (in Meter, negativ entspricht Linkskurve, positiv entspricht Rechtskurve) wird angegeben. Die Übergangsbögen werden automatisch gemäß den RAA bzw. RAL eingefügt. (2) Die gesamte

⁸ RAA (Richtlinien für die Anlage von Autobahnen), Forschungsgesellschaft für Straßen- und Verkehrswesen, Köln, 2008

⁹ RAL (Richtlinien für die Anlage von Landstraßen), Forschungsgesellschaft für Straßen- und Verkehrswesen, Köln, 2012

Länge der Kurve, der Radius des Kreisbogens sowie die Längenanteile der beiden Übergangsbögen an der Gesamtlänge der Kurve (Zahlen im Bereich von 0 bis 1) werden angegeben. Damit sind gezielte Übertretungen der RAA bzw. RAL, z.B. die Konstruktion von Kurven ohne Übergangsbögen, möglich.

Tabelle 1: Mindestradien und Mindestlängen von Kreisbögen gemäß der RAA.

RAA – Entwurfsklasse	Fernautobahn	Überregionalautobahn	Autobahnähnliche Straße	Stadtautobahn
Mindestradien (in m)	900	720	470	280
Mindestlänge (in m)	75	75	55	55

Tabelle 2: Empfohlene Radien und Mindestlängen von Kreisbögen gemäß der RAL.

RAL – Entwurfsgeschwindigkeit	110 km/h	100 km/h	90 km/h	70 km/h
Radienbereich (in m)	≥ 500	400 - 900	300 - 600	200 - 400
Mindestlänge (in m)	70	60	50	40

Das folgende Beispiel zeigt die Definition eines Course-Typs, der aus zwei Geraden und einer Kurve besteht.

```

define Course CK2
{
  NodeID = 2;
  CrossSection = CT_RV_350_350;
  Straight (50);
  Bend (1000,-1200);
  Straight (50);
  ...
};

```

Der Lageplan besteht aus einer 50 m langen Geraden, gefolgt von einer 1 km langen Linkskurve mit dem Radius 1.2 km. Die Längen der Übergangsbögen am Anfang und am Ende der Kurve werden automatisch berechnet. Die Straße endet mit einer 50m langen Geraden.

Der Verlauf von **Bezier-Kurven** wird über folgende Parameter spezifiziert:

- *<x Offset Ende>, <y Offset Ende>*: Der Endpunkt des vorhergehenden Elements (und damit der Startpunkt der Bezier-Kurve) ist der Ursprung eines lokalen Koordinatensystems. In diesem Koordinatensystem definieren *<x Offset Ende>* und *<y Offset Ende>* den Endpunkt der Bezier-Kurve.
- *<Tang.-Winkel Offset Ende>*: Tangentenwinkel (in Grad) im Endpunkt der Bezier-Kurve, gemessen im oben beschriebenen lokalen Koordinatensystem.

Im Bild unten ist der Verlauf der folgenden zwei Bezier-Kurven dargestellt:

```

Bezier(500, 500, 90);
Bezier(200, -200, -45);

```

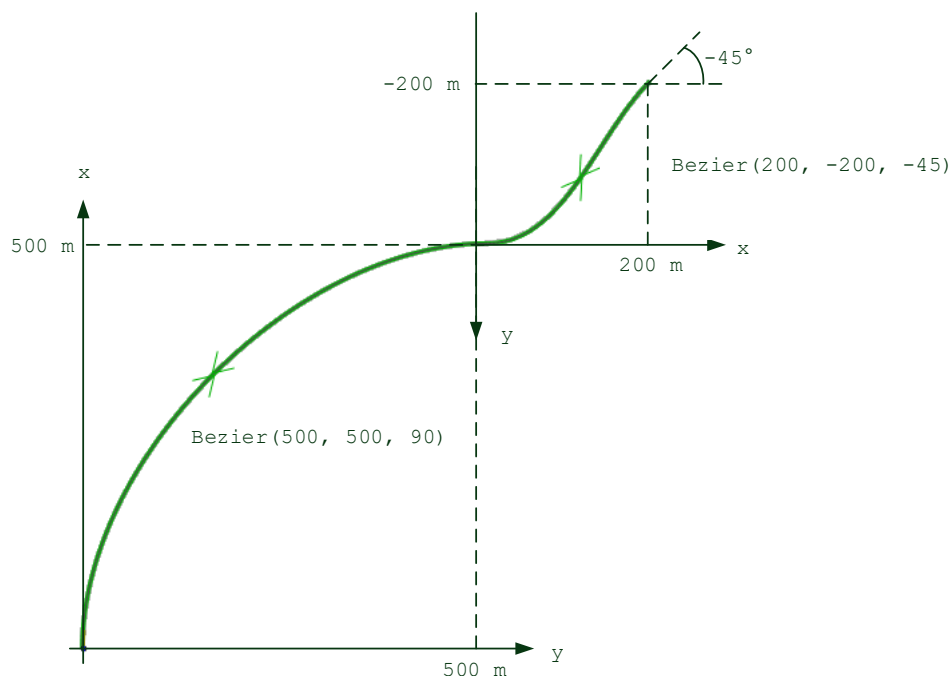


Abbildung 10: Definition von Bezier-Kurven

2.5.3 Höhenplan

Die Definition des Höhenplans eines Course-Typs hat folgende Struktur:

Grammatik 9 Definition des Höhenplans

```

<Höhenprofil>→
    HeightProfile =
    {
        (,)<Höhenplan-Element>
    };

<Höhenplan-Element>→
    (<Steigung/Gefälle>, <Länge>, <Ausrundungshalbmesser>)

```

Der Höhenplan besteht aus einer Abfolge von Elementen, bei denen jedes aus einer Ausrundung am Anfang und einer anschließenden konstanten Steigung besteht. Als Parameter werden pro Element die Länge, die Steigung und der Ausrundungshalbmesser angegeben. Die Gesamtlänge des Höhenplans muss der des Lageplans entsprechen. Damit sich die Steigungen aneinanderschließender Strecken nicht aufsummieren, ist im ersten und im letzten Element des Höhenplans nur die Steigung 0 zulässig. Dadurch wird beim ersten Element keine Ausrundung benötigt, der Ausrundungshalbmesser muss deswegen mit 0 angegeben werden. Für die Ausrundungshalbmesser der restlichen Segmente schreiben die RAA¹⁰ bzw. die RAL¹¹ je nach Entwurfsklasse folgende Mindestwerte vor:

Entwurfsklasse (RAA)	Fernautobahn	Überregionalautobahn	Autobahnähnliche Straße	Stadtautobahn
Kuppenmindesthalbm. (in m)	13 000	10 000	5 000	3 000
Wannenmindesthalbm. (in m)	8 800	5 700	4 000	2 600

Entwurfsgeschwindigkeit (RAL)	110 km/h	100 km/h	90 km/h	70 km/h
Kuppenmindesthalbm. (in m)	8 000	6 000	5 000	3 000
Wannenmindesthalbm. (in m)	4 000	3 500	2 000	2 000

2.5.4 Landschaftstypen

Durch Landschaftsgeneratoren (**LandscapeTypeLeft**, **LandscapeTypeRight** und **LandscapeTypeExtra**) können Hügel und Täler auf beiden Seiten des Course-Typs

¹⁰ RAA (Richtlinien für die Anlage von Autobahnen), Forschungsgesellschaft für Straßen- und Verkehrswesen, Köln, 2008

¹¹ RAL (Richtlinien für die Anlage von Landstraßen), Forschungsgesellschaft für Straßen- und Verkehrswesen, Köln, 2012

erzeugt werden und automatisch Objekte platziert werden. Dies wird in Kap. 2.5.13 näher erläutert.

2.5.5 Querschnittsprofil (CrossSections)

Jedem Course-Type wird ein Querschnittsprofil zugeordnet. In einer Datenbasis-Definition können beliebig viele solcher Querschnittsprofile definiert werden. Die Syntax hierfür lautet:

Grammatik 10 Definition eines Querschnittsprofils

```

<Querschnittsprofil>→
    CrossSection <Name Querschnittsprofil>
    {
        ve = <Entwurfsgeschwindigkeit in km/h>;
        Shoulder = {<ShoulderD1>, <ShoulderD2>, <ShoulderH>,
                    <Belag>, <Transparenz>};
        Overlay = {<Overlay>, <Sichtbarkeit>};
        (0..1)GraphicOptions = {(,)<Grafik-Option>};
        LaneInfos =
        {
            (,)<Spurinformation(→Grammatik 11 Seite 33)>
        };
        LaneTransitions =
        {
            (,)<Spurübergangsinformation(→Grammatik 12 Seite 34)>
        };
    };

```

Bemerkung: Statt **CrossSection** ist aus Kompatibilitätsgründen auch das Schlüsselwort **CourseType** möglich.

Neben der Angabe der Entwurfsgeschwindigkeit (Geschwindigkeit, für die dieser Straßentyp entworfen ist), setzt sich ein Querschnittsprofil aus vier Komponenten zusammen: Die Definition der Schulter, einer Menge von Spurinformationen (Breite, relative Höhe, Fahrtrichtung usw.), einer Menge von Spurübergangsinformationen (Pfosten, Fahrbahnmarkierungen, Bebauungen) und optional zusätzlichen Grafikoptionen.

Shoulder-Parameter

Die *Shoulder*-Parameter werden in Abbildung 11 verdeutlicht.

Dabei muss der Wert <ShoulderH> kleiner oder gleich -0.5 sein, d.h. die umgebende Landschaft liegt mindestens 0.5 m unter der Höhe der Straße.

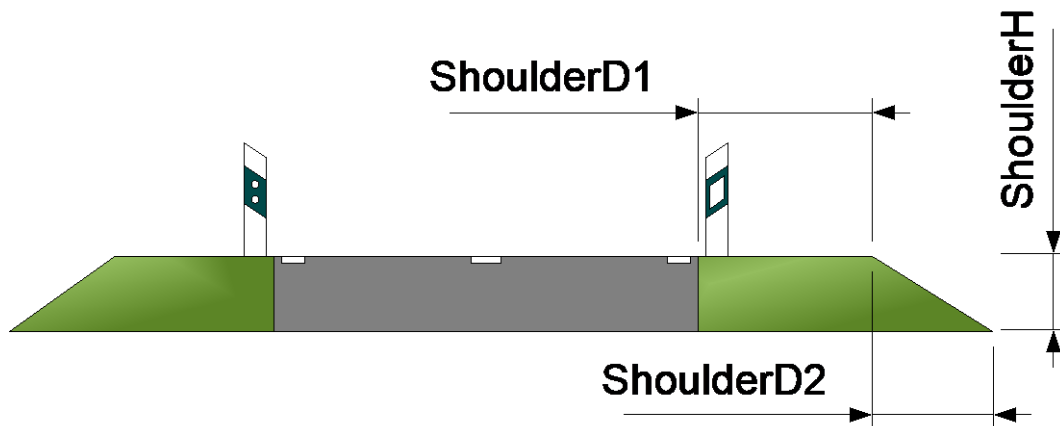


Abbildung 11: Shoulder-Parameter der Querschnittsdefinition

Um den Übergang zwischen Schulter und der umgebenden Landschaft zu verbessern, kann die Schulter mit dem Parameter *<Transparenz>* teilweise transparent dargestellt werden. Mögliche Werte sind im Bereich 0 (harter Übergang) bis 1 (sehr weicher Übergang mit starker Transparenz).

Anmerkung: Die Transparenz wird nur dargestellt, wenn Antialiasing aktiviert ist (*SGEWorld.AntiAlias > 1*). Im „Nachfahrt“-Modus muss zusätzlich Multisampling aktiviert sein (*SGEWorld.PreferMultisampling = true*). Andernfalls ergibt sich immer ein harter Übergang.

Overlay-Parameter

Die Overlay-Parameter verweisen auf den Namen einer Overlay-Textur (siehe Kap. 6.2). Die *Sichtbarkeit* des Overlays kann von 0 (unsichtbar) bis 1 (undurchsichtig) stufenlos gewählt werden, so dass auch eine Kombination mit der darunter liegenden Straßentextur möglich ist.

Grafik-Option

Angaben zur *<Grafik-Option>* werden ebenfalls nur von der Visualisierung SGE interpretiert. Es sind folgende Grafikoptionen möglich:

Grafik-Option	Beschreibung
NormallyDark	Die Straße und zugehörige Objektbänder in großer Entfernung werden nicht von der Sonne beleuchtet, sondern abgeschattet dargestellt.
ObjectsCourseRelative	Bebauungen, die aus einzelnen Objekten bestehen (s. Seite 35), werden immer relativ zum Anfang des Course positioniert. Falls diese Grafikoption nicht angegeben ist, werden Objekte entlang der kompletten Strecke so positioniert, dass stets der angegebene Zwischenabstand eingehalten wird.

Mehrere Grafikoptionen können beliebig miteinander kombiniert werden.

Grammatik 11 Spurinformatoren

<Spurinformation> →

```
(
    <Spurindex>, <Spurbreite>, <rel. Höhe>, <Richtung>,
    <Belag>, <Reibbeiwert>, <Rauigkeit>,
    {
        (,)<Bebauung>
    }
    , <Befahrbarkeit>
)
```

<Richtung> → **Onwards** | **Towards**

<Belag> → **Tar** | **Concrete1** | **Grass1** | *<sonstiger Belag (siehe Kap. 6.1)>*

<Bebauung> → (*<Bebauungstyp>*, *<YOffset>*)

<Bebauungstyp> →

None | **Guardrail** | **ReflectionPosts** | **Line** | *<sonstiger Bebauungstyp (siehe Kap. 6.2)>*

<Befahrbarkeit> → 0|1

Der Spurindex zählt die Spuren einer Straße, von ihrem Beginn nach Ende blickend und von links nach rechts, mit Null beginnend. Die Spurbreite legt die Breite der Spur in Metern fest. Der Parameter *<rel. Höhe>* (Einheit Meter) wird derzeit nicht unterstützt und sollte stets als 0 angegeben werden. Die Fahrtrichtung bezieht sich ebenfalls auf die durch den Verlauf von Beginn nach Ende vorgegebene Richtung der Straße. Der Eintrag der Befahrbarkeit zeigt dem Verkehr, ob diese Spur benutzt werden darf (=1) oder nicht. Spuren ohne diesen Parameter sind befahrbar. Die Parameter *<Reibbeiwert>* und *<Rauigkeit>* geben Informationen über die Straßenoberfläche an die Fahrdynamik weiter. Sie werden jedoch von den meisten Fahrdynamiken (z.B. *DPUMstVeda*) nicht unterstützt.

Eine Spur kann beliebig viele Bebauungen haben. Der Parameter *<YOffset>* (Einheit Meter) einer Bebauung verschiebt das entsprechende grafische Objekt bezgl. der Spurmitte nach links (*<YOffset>* < 0) bzw. nach rechts (*<YOffset>* > 0).

Fahrbahnmarkierungen, Pfosten etc. werden mit folgender Syntax auf Spurübergängen definiert:

Grammatik 12 Spurübergangsinformationen

```

<Spurübergangsinformation>→
(
    <Spurindex1>, <Spurindex2>,
    {
        (,) <Bebauung Spurübergang>
    }
)
<Bebauung Spurübergang>→
(
    <Bebauungstyp(→Kap. 6.2)>, <YOffset>, <Größe>,
    <Orientierung>, <Abstand>, <Zwischenraum>
)
<Orientierung>→ Towards | Onwards

```

Die Übergänge zwischen den Spuren werden durch ein Paar von aufeinanderfolgenden Spurindizes adressiert. Sonderfälle sind die Übergänge von der Landschaft auf die am weitesten links liegende Spur des Querschnitts (Spur 0) und auf die am weitesten rechts liegende Spur des Querschnitts (Spur N). Sie werden über <Spurindex1> = <Spurindex2> = 0 bzw. <Spurindex1> = <Spurindex2> = N spezifiziert.

Bebauungen

Straßen können durch verschiedene Bebauungen ausgeschmückt werden. Unter Bebauungen im Sinne von SILAB fallen dabei sowohl Fahrbahnmarkierungen (Linien) als auch Hecken, Waldbänder, Mauern, Leitplanken und Pfosten. Eine Vielzahl von Bebauungen ist bereits vordefiniert und im Lieferumfang von SILAB enthalten (siehe Kap. 6.2). Weitere Bebauungen können projektspezifisch im Konfigurationsabschnitt **SCNX Params** hinzugefügt werden (siehe Kap. 5.2.2). Der Name der gewünschten Bebauung muss als <Bebauungstyp> eingetragen werden.

Die genaue Positionierung und Größe der Bebauungen kann durch die Bebauungsparameter angepasst werden. Dabei muss allerdings zwischen den drei möglichen Arten von Bebauungen unterschieden werden:

1. Fahrbahnmarkierungen

Fahrbahnmarkierungen und andere direkt auf der Fahrbahn aufgebrachte, flache, regelmäßig angeordnete Objekte können direkt in die Straßenbelags-Textur eingebettet werden, die automatisch aufgrund der Angabe der Spurbreiten und Fahrbahnbeläge beim Start der Simulation erzeugt wird.

Für Fahrbahnmarkierungen wirken die Bebauungsparameter wie folgt:

- <YOffset>: Verschiebung der Fahrbahnmarkierung nach links (<0) bzw. nach rechts (>0) zum exakten Spurübergang (Einheit Meter).
- <Größe>: Breite der Fahrbahnmarkierung in Meter.
- <Orientierung>: Keine Auswirkung, **Onwards** verwenden.
- <Abstand>: Länge einer Fahrbahnmarkierung in Meter.
- <Zwischenraum>: Länge des Zwischenraums zwischen zwei Fahrbahnmarkierungen in Meter. Für eine durchgezogene Linie wird <Abstand> = 1 und <Zwischenraum> = 0 gesetzt.

Das folgende Beispiel zeigt die Spurübergangsinformationen für die Mittellinie einer Bundesstraße:

```
(0, 1,          # Übergang zwischen linker und rechter Spur
  {(Line, 0.0, 0.15, Towards, 2.0, 6.0)}    # Mittellinie
)
```

Es wird eine unterbrochene Linie mit Breite 15 cm angebracht, wobei die Striche 2 m lang und jeweils 6 m voneinander entfernt sind.

2. Objektbänder

Mit Objektbändern lassen sich besonders gut Bebauungen realisieren, die im festen Abstand parallel zur Straße verlaufen. Die häufigsten Anwendungsfälle hierfür sind Leitplanken. Objektbänder lassen sich auch gut für Hecken, Mauern, Brückengeländer und ähnliche Objekte verwenden.

Für Objektbänder wirken die Bebauungsparameter wie folgt:

- **<YOffset>**: Verschiebung des Objektbands nach links (<0) bzw. nach rechts (>0) zum exakten Spurübergang (Einheit Meter).
- **<Größe>**: Skalierung des Objektbands in der Höhe (mit dem Wert 1.0 wird die Originalgröße (wie sie im Abschnitt SCNX Params definiert wurde, vgl. Abschnitt 5.2.2) verwendet).
- **<Orientierung>**: Keine Auswirkung, **Onwards** verwenden.
- **<Abstand>** und **<Zwischenraum>**: Keine Auswirkung, 0 eintragen.

Beispiel: Spurübergangsinformation für eine Leitplanke am Fahrbahnrand

```
(0, 0,          # Linker Fahrbahnrand
  {(Guardrail, -0.5, 1.0, Onwards, 0, 0)}    # Leitplanke
)
```

Die Leitplanke wird 50 cm neben dem Straßenrand aufgestellt und in Originalgröße angebracht.

3. Einzelne Objekte

Zuletzt ist es möglich, Bebauungen für einzelne, unabhängige Objekte, die in regelmäßigen Abständen neben der Straße aufgestellt werden, zu definieren. Dies wird häufig zur Darstellung von Begrenzungspfosten verwendet.

Für einzelne Objekte wirken die Bebauungsparameter wie folgt:

- **<YOffset>**: Verschiebung der Bebauung nach links (<0) bzw. nach rechts (>0) zum exakten Spurübergang (Einheit Meter).
- **<Größe>**: Keine Auswirkung, 1 eintragen.
- **<Orientierung>**: Richtung, in die Pfosten zeigen sollen. Der Parameter bezieht sich auf die Richtung, die durch den Verlauf der Straße von Beginn nach Ende vorgegeben ist.
- **<Abstand>**: Abstand der Objekte in Metern.
- **<Zwischenraum>**: Falls die Grafikoption *ObjectsCourseRelative* gesetzt ist, ist dies der Abstand des ersten Objekts vom Anfang des Course. Ansonsten keine Auswirkung, 0 angeben.

Beispiel: Spurübergangsinformation für Begrenzungspfosten

```
(0, 0,          # Linker Fahrbahnrand
  {(ReflectionPosts, -0.3, 1.0, Towards, 50, 0)} # Pfosten
)
```

Die Pfosten werden 30 cm neben dem Straßenrand aufgestellt, der Abstand zwischen je zwei aufeinander folgenden Pfosten beträgt 50 m.

Beispiel für ein Querschnittsprofil

Das folgende Beispiel zeigt das Querschnittsprofil einer einfachen Bundesstraße. Am rechten Fahrbahnrand ist eine Leitplanke platziert. Auf beiden Seiten stehen Begrenzungspfosten.

```
CrossSection Bundesstrasse
{
  ve = 90;
  LaneInfos =
  {
    (0, 3.5, 0.0, Towards, Tar, 1.0, 0.0, {(None, 0.0)}),
    (1, 3.5, 0.0, Onwards, Tar, 1.0, 0.0, {(None, 0.0)})
  };
  LaneTransitions =
  {
    (0, 0, {
      (ReflectionPosts, -0.3, 1.0, Towards, 50.0, 0.0),
      (Line, 0.12, 0.15, Towards, 1.0, 0.0)
    },
    (0, 1, {
      (Line, 0.0, 0.15, Towards, 2.0, 6.0)
    },
    (1, 1, {
      (ReflectionPosts, 0.3, 1.0, Onwards, 50.0, 0.0),
      (Line, -0.12, 0.15, Onwards, 1.0, 0.0),
      (Guardrail, 0.4, 1.0, Onwards, 0.0, 0.0)
    }
  )
  };
};
```

2.5.6 Grafische Objekte (Objects)

Zu den grafischen Objekten zählen Verkehrsschilder, Straßenpfosten, Fahrbahnmarkierungen in Verkehrsknoten, Bäume, Häuser usw. Sie werden in Course-Typen über den Abschnitt **Objects = { ... }** eingefügt. Dieser Abschnitt hat folgende Syntax:

Grammatik 13 Objekte von Course-Typen

<Objekte von Course-Typen> →

Objects =

{

(,)<Objektdefinition Course-Typ>

};

<Objektdefinition Course-Typ> →

(<Objektnamen>, <Streckenmeter>, <Abstand>) |

(<Objektnamen>, <Streckenmeter>, <Abstand>, <Winkel>) |

(<Objektnamen>, <Streckenmeter>, <Abstand>, <Winkel>,

<Offset>) |

(<Objektnamen>, <Streckenmeter>, <Abstand>, <Winkel>,

<Offset>, (,)<objektspezifische Parameter>)

<Objektnamen> →

(1...*)[<Objektverzeichnis>.]<Name Objekt>

Es gibt also je nach Art des Objekts vier Arten der Definition (Reihenfolge der Beschreibung wie in obiger Grammatik):

1. In der ersten Definition wird neben dem Objektnamen ein Streckenmeter angegeben, der den Abstand entlang der Straße (vom Beginn der Straße aus) angibt. Der dritte Parameter spezifiziert den gerichteten Abstand (in Meter) von der Mittellinie der Straße, an dem das Objekt positioniert wird. Negative Abstände geben Positionen links von der Straßenmitte, positive Abstände rechts davon an. Bäume werden beispielsweise so platziert:

```
Objects =
{
    (sign.big276, 50, 4.2),
    (environment.tree.tree2, 214, -50),
    (environment.tree.tree1, 380, 12),
    ...
};
```

2. Definition 1 kann um einen Winkel (in Grad) ergänzt werden, der das Objekt um seine Hochachse (relativ zum Tangentenwinkel der Straße) nach rechts dreht. Mit diesem Parameter kann beispielsweise ein Haus gedreht werden:

```
Objects =
{
    (house.house1, 100, -20, 90),
    ...
};
```

3. Diese Definition ermöglicht es, Objekten in der Szenerie einen gemeinsamen Bezugspunkt auf der Strecke zu geben. Wird eine Gruppe zusammenstehender Objekte benötigt, so bietet die Definition 3 eine Platzierung, die unabhängig ist von der Krümmung der Straße.

```
Objects =
{
    (house.house1, 100, -20, 0, 30),
```

```

        (house.house1, 100, -20, 0, -10),
        ...
    };

```

In diesem Beispiel werden beide Häuser am Streckenmeter 100 verankert und dann noch um 30 bzw. -10 Meter entlang der Tangente verschoben.

4. Die letzte Definition bietet die Möglichkeit, dem grafischen Objekt weitere Parameter zu übergeben. Wenn ein Objekt solche Parameter unterstützt, müssen alle vier Parameter (*Streckenmeter*, *Abstand*, *Winkel* und *Offset*) vor den zusätzlichen Parametern angegeben werden.

Durch Height und Friction Maps (siehe Kap. 5.2.3) kann die Oberfläche von grafischen Objekten zusätzlich beeinflusst werden.

2.5.7 Schilder als grafische Objekte

Verkehrsschilder können automatisch beim Start der Simulation generiert werden. Größe und Aufbau der Schilder (z.B. Position und Anzahl von Schildpfosten) sind im Abschnitt `SCNX SignGeneration` der Datei `\SILAB\lib\configurations\SCNX\SCNX_600.cfg` definiert. Es wird lediglich noch eine Textur des Schilds benötigt, die sich im Unterverzeichnis `sign` des Projektverzeichnisses oder des Verzeichnisses `\SILAB\lib\graphics\default\texture` befinden muss.

ACHTUNG: Schilder, die auf diese Weise platziert werden, sind nur als grafisches Objekt in der Simulation vorhanden. Falls der simulierte Verkehr auf die Schilder reagieren soll, muss stattdessen eine `TrafficRule` definiert werden (s. Abschnitt 2.5.9).

Der `<Objektnamen>` des Schilds hat folgende Syntax:

`sign.<Size_><Pos_><Number><~Text1><~Text2><~Text3>`

Im Einzelnen bedeutet dies:

- Die *Size*-Angabe bezeichnet die Größe des Schilds.
 - Die Schildbreite kann mit der Angabe **Sign**, gefolgt von der Breite in cm angegeben werden (z.B. bedeutet **Sign100**, dass das Schild 100 cm breit ist).
 - Die vordefinierten Größen **Big**, **Middle** und **Small** können verwendet werden.
 - Die *Size*-Angabe kann auch weggelassen werden. In diesem Fall wird die Größe des Schilds automatisch anhand der Entwurfsgeschwindigkeit bestimmt, die im zugehörigen **CrossSection** der Strecke festgelegt wurde.
- Die *Pos*-Angabe gibt die Position des Schilds an. Dabei bestehen folgende Möglichkeiten:
 - (keine Angabe): Das Schild wird in normaler Höhe am Pfosten aufgehängt.
 - **N**: Das Schild wird in niedriger Höhe am Pfosten aufgehängt. Diese Position ist für „An dieser Verkehrsinsel vorbeifahren“-Schilder geeignet.

- **H:** Das Schild wird oben am Pfosten aufgehängt. Es wird sichergestellt, dass ein Zusatzschild unterhalb Platz findet.
- **U:** Das Schild wird in niedriger Höhe platziert. Es wird kein Pfosten aufgestellt. Diese Position sollte für Zusatzschilder verwendet werden.
- Die *Number*-Angabe gibt die Nummer des Schilds entsprechend der Straßenverkehrsordnung des jeweiligen Landes¹², evtl. zusätzlich mit einer Varianten- oder Beschriftungsangabe an. Sie entspricht dem Dateinamen (ohne Dateierweiterung) der Texturdatei.
- Bei Wegweisern wird die Beschriftung als *Text*-Angabe(n) (je nach Schild bis zu 3 Beschriftungen, je nach Schildtyp) an den Namen angehängt.
 - Jede *Text*-Angabe ist der Name des Ziels als einfacher Text. Dabei sind bis zu 14 Zeichen möglich, inklusive Sonderzeichen, wie z.B. Leerzeichen, Bindestriche und Umlaute.
Abweichend hiervon sind bei Schildern, auf denen Straßennummern eingesetzt werden, bis zu drei Ziffern möglich.
Falls die Text-Angabe Sonderzeichen enthält, muss der komplette Objektname in Anführungszeichen stehen.
 - Zusätzlich sind im Lieferumfang von SILAB, im Verzeichnis
`\SILAB\lib\graphics\default\texture\sign\text`, Texturen für Sonderziele mit grafischen Besonderheiten vorhanden. Zur Verwendung dieser Sonderziele muss der Name des Bilds innerhalb des Texturverzeichnisses (ohne Dateierweiterung), gefolgt von einem Unterstrich, angegeben werden.
 - Bei Wegweisern, die mehrere Ziele anzeigen, werden die Ziele in der Reihenfolge von links nach rechts bzw. von oben nach unten angegeben.
- Einige Beispiele für Schilddefinitionen:

```
Objects = {
    (sign.205, 20, 5),           # (1) Vorfahrt achten
    (sign.H_278_060, 40, 5),    # (2a) Geschwindigkeitsbeschränkung
    (sign.U_1002_24a, 40, 5),   # (2b) Zusatzschild
    (sign.Sign300_N_222_020, 60, 3) # (3) Vorbeifahren
};
```

¹² Länderspezifische Schilder sind für China, Deutschland, Österreich, die Schweiz und die Vereinigten Staaten verfügbar.



- Beispiele für Wegweiser mit Beschriftung:

```
Objects = {  
  ("sign.438_20~Stadion_~Alle Richtungen", 50, -6), # (1)  
  ("sign.434_31~Würzburg~Berlin~Hamburg", 50, 6), # (2)  
};
```

Hinweis: Bei „Stadion“ handelt es sich um ein Sonderziel (daher hat es einen weißen Hintergrund).



2.5.8 Weitere Hinweise zu grafischen Objekten

In Verkehrsknoten (Area2) werden die Objekte mit Hilfe des Editors SILABAEEdit platziert.

Es ist in SILAB auch möglich, eigene grafische 3D-Objekte zu gestalten bzw. mit 3D-Modelling-Programmen entworfene Modelle zu importieren. Die Vorgehensweise wird in Kap. 3 des Dokuments „Referenz Visualisierung SGE“ genauer beschrieben.

2.5.9 Verkehrsregeln (TrafficRules)

Verkehrsschilder können – neben dem Einfügen als rein grafische Objekte (vgl. 2.5.6) – auch über den TrafficRules-Abschnitt von Course-Streckentypen eingefügt werden.

Mit dieser Methode werden auch zum Schild gehörende Verkehrsregeln eingefügt, so dass beispielsweise der Fremdverkehr (mit geeigneten Verhaltensmodellen) korrekt auf das Schild reagieren kann. Auch DPUs, die mit dem Entwicklerpaket Datenbasis (iSCNX) erstellt werden, können diese Verkehrsregeln an beliebigen Stellen des Streckennetzwerks abfragen.

Grammatik 14 Verkehrsregeln

```

<Verkehrsregeln>→
    TrafficRules <Name>
    {
        (1..*)<Schilddefinition >
    };
<Schilddefinition>→
    Sign <Name>
    {
        Sign = ((), <Schildtyp>);
        s = <Streckenmeter>;
        d = <Abstand>;
        optional zusätzlich möglich:
        Dir = (Onwards|Towards);
        Angle = <Winkel>;
        Size = <Größe in cm>;
        Lane = ((), <Spurnummern>);
        Show = (false|true);
    };

```

Als <Schildtyp> wird jeweils der Name einer Textur (ohne Dateierweiterung) aus dem Sign-Verzeichnis verwendet. Dieser wird üblicherweise nach dem Schema „<Schild-Nummer>_<Variante>_<Variante2>~<Text>“ gebildet. Eine Übersicht über die vorhandenen Schildertypen und die dazugehörigen Verkehrsregeln gibt Abschnitt 6.5. Details zu Schildbeschriftungen (Angabe <Text>) können Sie im Abschnitt 2.5.7 nachlesen.

Die Schildtypen, die in der **Sign**-Liste aufgezählt werden, werden von oben nach unten am Pfosten befestigt.

Für die Angabe des Streckenmeters, Abstands und Winkels gilt dasselbe wie bei **Objects**. Wird allerdings die Richtung (**Dir**), für die das Schild gedacht ist als **Towards**, d.h. Gegenrichtung, angegeben, so wird das Schild auch in Gegenrichtung aufgestellt, d.h. so dass es von der Gegenrichtung kommend sichtbar ist. Ein positiver Abstand ist immer in der Richtung, in der das Schild lesbar ist, rechts der Straßenmitte. Optional kann eine Größe **Size** des Schilds (Breite in cm) angegeben werden. Diese wird entsprechend der Entwurfsgeschwindigkeit des Course-Typen automatisch gewählt, falls die Größe weggelassen wird.

Die **Lane**-Liste gibt an, für welche Spur(en) das Schild gilt und muss nur angegeben werden, falls es nicht alle Spuren in der angegebenen Richtung sind.

Durch die Angabe **Show=false** wird das Schild nicht angezeigt und kann somit zur Steuerung des Fremdverkehrs eingesetzt werden, ohne dass dies für den Testfahrer ersichtlich ist.

Der <Name> des **TrafficRules**-Abschnitts und der **Sign**-Definition wird ignoriert.

Im folgenden Beispiel wird auf einer Landstraße bei Streckenmeter 100 die Geschwindigkeit 80 km/h beschränkt. Bei Streckenmeter 500 gilt ein Überholverbot:

```

TrafficRules T
{
    Sign X
    {
        Sign = ("274_080");
        s = 100;
        d = 5;
    };

    Sign X
    {
        Sign = ("276");
        s = 500;
        d = 5;
    };
};

```

2.5.10 Zusätzliche Objektbänder

Zusätzlich zu den im Querschnittsprofil definierten Objektbändern, die entlang des kompletten Streckenverlaufs des Course platziert werden, können auch einzelne Objektbänder entlang von Teilstrecken platziert werden. Damit können z.B. in gefährlichen Kurven Leitplanken aufgestellt werden.

Grammatik 15 Definition zusätzlicher Objektbänder

```

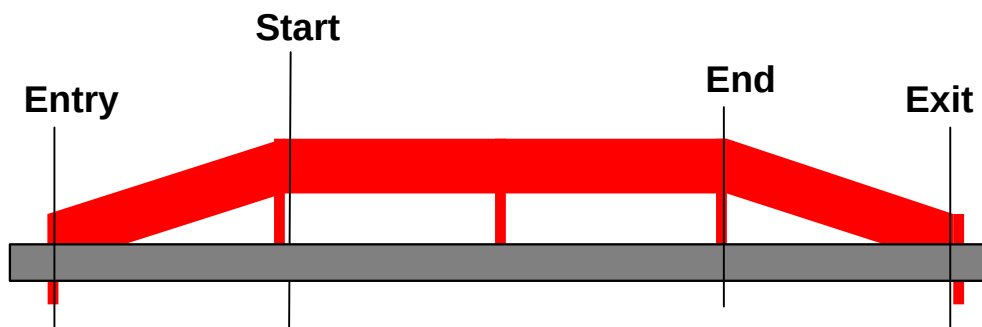
<Zusatz-Objektband> →
    Strip <Bebauungstyp>
    {
        (0..1) Orientation = Onwards | Towards;
        YOffset = <seitlicher Versatz>;
        (0..1) Width = <Skalierungsfaktor>;

        (0..1) Entry = <Einblende-Streckenmeter>;
        (0..1) Start = <Anfangs-Streckenmeter>;
        (0..1) End = <End-Streckenmeter>;
        (0..1) Exit = <Ausblende-Streckenmeter>;
    };

```

Die Parameter *Bebauungstyp*, *Orientation*, *YOffset* und *Width* funktionieren genau wie beim Einbinden eines Objektbands über das Querschnittsprofil (siehe S. 35). Dabei ist zu beachten, dass zusätzliche Objektbänder immer von der Mittellinie des Courses aus platziert werden, d.h. *YOffset* ist ein Versatz gegenüber der Straßenmitte. Falls Parameter weggelassen werden, werden die Standardwerte *Orientation=Onwards* und *Width=1.0* verwendet.

Mit den Parametern *Entry*, *Start*, *End* und *Exit* kann die Positionierung des Objektbands entlang der Strecke festgelegt werden. Es handelt sich jeweils um Streckenmeter. Folgende Abbildung veranschaulicht die Wirkung der Parameter am Beispiel einer Leitplanke (Guardrail-Bebauung):



Da die Strecke und ihre Bebauungen in festen Abständen abgetastet werden, sollten für diese Streckenmeter jeweils Vielfache des Abtastabstands *ds_max* (siehe Abschnitt 5.3.1), welcher üblicherweise 5 m beträgt, verwendet werden.

Falls eine Streckenmeter-Angabe weggelassen wird, werden für *Entry* und *Start* der Course-Anfang (=0) und für *End* bzw. *Exit* das Course-Ende verwendet.

2.5.11 Zusätzliche, wiederholte Objekte

Zusätzlich zu den im Querschnittsprofil definierten wiederholten Objekten, die entlang des kompletten Streckenverlaufs des Course platziert werden, können auch wiederholte Objekte entlang von Teilstrecken platziert werden. Damit können z.B. in einem Teilbereich des Course Straßenlampen aufgestellt werden, ohne dass jede einzelne Lampe separat im Objects-Abschnitt positioniert werden müsste.

Grammatik 16 Definition zusätzlicher wiederholter Objekte

```

<Zusatz-Objekte>→
  Object <Bebauungstyp>
  {
    (0..1)Orientation = Onwards | Towards;
    Distance = <Abstand zwischen Objekten>;
    YOffset = <seitlicher Versatz>;

    (0..1)Start = <Anfangs-Streckenmeter>;
    (0..1)End = <End-Streckenmeter>;
  };

```

Die Parameter *Bebauungstyp*, *Orientation*, und *YOffset* funktionieren genau wie beim Einbinden wiederholter Objekte über das Querschnittsprofil (siehe S. 35). Dabei ist zu beachten, dass zusätzliche wiederholte Objekte immer von der Mittellinie des Courses aus platziert werden, d.h. *YOffset* ist ein Versatz gegenüber der Straßenmitte. Falls Parameter weggelassen werden, wird der Standardwert *Orientation=Onwards* verwendet.

Der Parameter *Distance* gibt den Abstand zwischen zwei aufeinander folgenden Objekten in Metern an.

Mit den Parametern *Start* und *End* kann der Bereich, in dem die Objekte entlang der Strecke positioniert werden, festgelegt werden. Objekte werden vom Streckenmeter *Start* bis zum Streckenmeter *End* positioniert, wobei der Abstand zwischen zwei Objekten mit dem Parameter *Distance* festgelegt wird.

Falls eine Streckenmeter-Angabe weggelassen wird, wird für *Start* der Course-Anfang (=0) und für *End* das Course-Ende verwendet.

2.5.12 Points of Interest (POI)

Bei Points of Interest (im Skript „POI“ genannt) handelt es sich um für den Testfahrer unsichtbare Markierungen in der Umgebung einer Strecke, die zur Steuerung von DPU's und auch zur Platzierung von 3D-Sounds genutzt werden können.

In der Definition einer Datenbasis können die Points of Interest innerhalb von Streckentypen (also Course-Typen bzw. Area2-Typen) eingefügt werden.

In Course-Typen hat die Definition von POI folgende Form:

Grammatik 17 POI in Course-Typen

<POI-Liste Course-Typ>→

POI =

 {

 (,)<POI Course-Typ>

 };

<POI Course-Typ >→

 (<Streckenmeter>, <Abstand>, <Winkel>, <Familie>

 (,)<POI-Argument>)

<POI-Argument>→

 <Name> , <Wert>

Jeder POI eines Course-Typs besteht aus folgenden Angaben:

- <Streckenmeter>: Der Abstand des POI entlang der Spur zum Beginn der Straße.
- <Abstand>: Gerichteter Abstand (in Meter) von der Mittellinie der Straße, an dem der POI positioniert wird. Negative Abstände geben Positionen links von der Straßenmitte, positive Abstände rechts davon an.
- <Winkel>: Winkel (in grad), der die Ausrichtung des POI definiert. Beim Winkel 0 zeigt der POI in Fahrtrichtung. Die Drehung erfolgt um die Hochachse, positive Winkel zeigen nach rechts.
- <Familie>: Der Name der Familie, zu der dieser POI gehört.
- <POI-Argument>: Jedes Argument besteht aus einem Namen und einem Wert.

POI in Area2-Typen werden mit Hilfe des Editors SILABAEEdit eingefügt. Die Vorgehensweise wird im Dokument „SILABAEEdit“ erläutert.

2.5.13 Landschaftsgeneratoren

Die Landschaft um Streckentypen herum (Course-Typen oder Area2-Typen) kann mit Landschaftsgeneratoren interessanter gestaltet werden. Einerseits kann das Höhenmodell der Landschaft beeinflusst werden, d.h. Hügel, Täler und auch kleine Unebenheiten können erstellt werden. Andererseits können automatisch Objekte wie z.B. Häuser und Bäume in der Landschaft platziert werden. So können Bäume, Agrargebiete mit Feldern, kleine Ortschaften und sogar Wälder erzeugt werden, ohne dass eine Vielzahl einzelner Objekte platziert werden müsste.

Für die linke und rechte Seite der Straße¹³ können jeweils unterschiedliche Landschaften definiert werden, dazu wird die Straßenseite als **LandscapeTypeLeft** bzw. **LandscapeTypeRight** angegeben.

Außerdem können zusätzliche Objekte in einem gewissen Bereich mit einem oder mehreren **LandscapeTypeExtra**-Generatoren platziert werden. Die **LandscapeTypeExtra**-Generatoren haben aber keinen Einfluss auf das Höhenmodell. Für die Extra-Generatoren muss über die Angabe **Left=true;** bzw. **Left=false;** festgelegt werden, ob die Objekte auf der linken oder der rechten Seite der Straße platziert werden sollen.

¹³ Wenn um eine Area2 herum Landschaftsgeneratoren verwendet werden sollen, müssen die Area2-Objekte im SILABAEEdit so ausgerichtet werden, dass die „Hauptstraße“ auf dem Bildschirm etwa waagerecht, von links nach rechts, verläuft. Auch muss das Area2 breiter als hoch sein. Die „linke Seite“ der Area2 ist dann oben auf dem Bildschirm; die „rechte Seite“ ist unten. Objekte und Hügel werden nur außerhalb des umschließenden Rechtecks der Area2 platziert.

Grammatik 18 Definition von Landschaftstypen

<Landschaftstypen>→

(0..*)<Landschaftstyp>

<Landschaftstyp>→

```

<Straßenseite> <Landschaftsname> {
  (0..*) <Parameter des Höhenmodells>
  (0..1) Override = { (,) <Objektgruppe> };
  (0..1) Left = (false|true); # nur für LandscapeTypeExtra
  (0..*) <Spezielle Parameter des Landschaftstyps>
};

```

<Straßenseite>→

LandscapeTypeLeft | LandscapeTypeRight | LandscapeTypeExtra

<Landschaftsname>→

**Alley1 | Alley2 | AncientVillage | Bushes | CityWall | CountryVillage |
 DryMeadow | Farmed | Forest | FruitTree | Grassland | GroundFog |
 Hedge | Inhabited | LeafForest | Meadow | Meadows |
 RoadsideBush | SoundBarrierMetal | SoundBarrierWood | Suburb |
 Tunnel | Tunnel | TunnelWide | TunnelRectangle | TunnelRectangleWide
 | Vineyard | Wooded |**
 <eigener Landschaftstyp (→Kap. 5.2.5)>

Wird auf einer Straßenseite kein Landschaftstyp angegeben, so ist die Landschaft auf dieser Seite weitgehend flach und es werden keine Objekte platziert.

Es gibt 27 vordefinierte Landschaftstypen, deren Aussehen jeweils durch einige Parameter beeinflusst werden kann. Jeder Parameter wird in Form eines Intervalls angegeben. Jedes Mal, wenn eine Straße in den Sichtbarkeitsbereich des Fahrers gelangt, wird der eigentliche Parameterwert zufällig aus diesem Intervall gezogen. Wenn sich ein Parameter nicht bei jedem Eintritt einer Straße in den Sichtbarkeitsbereich ändern soll, müssen die Intervallgrenzen gleich gewählt oder nur ein einziger Wert angegeben werden. Zusätzlich können Sie durch eine Erweiterung der SCNX-Konfiguration auch eigene Landschaftstypen, in denen andere grafische Objekte platziert werden, definieren. Dies wird in Kap. 5.2.5 näher beschrieben.

Unabhängig vom Landschaftstyp sind folgende Parameter des Höhenmodells verfügbar:

Height = (<min>, <max>);

Amplitude der Landschaftshöhen in Meter. Sinnvolle Werte liegen zwischen 0 und 10.

Offset = (<min>, <max>);

Offset der Landschaftshöhe in Bezug auf die Höhe der Straße in Meter. Sinnvolle Werte liegen zwischen -10 und 10.

Ripple = (<min>, <max>);

Welligkeit der Landschaft. Sinnvolle Werte liegen zwischen 0 und 4.

Gradient = (<min>, <max>);

Steigung der Landschaft senkrecht zur Straße (in Meter, Höhenunterschied pro Meter Abstand zur Straße). Sinnvolle Werte liegen zwischen -0.5 und 0.5.

Roughness = (<min>, <max>);

Maximale Höhe von kleinen Unebenheiten in der Landschaft in Meter.

MaxHillDist = (<min>, <max>);

Entfernung von der Straße in Meter, bis zu der noch Unebenheiten (entsprechend **Height**, **Ripple**, und **Roughness**) dargestellt werden. Im größeren Abstand wird nur noch eine relativ glatte Landschaft erzeugt, die nur grob der Höhe der Straße folgt. Der Standardwert ist 200. Sinnvolle Werte liegen zwischen 100 und 500.

Die Auswirkung dieser Parameter wird in Abbildung 12 verdeutlicht.

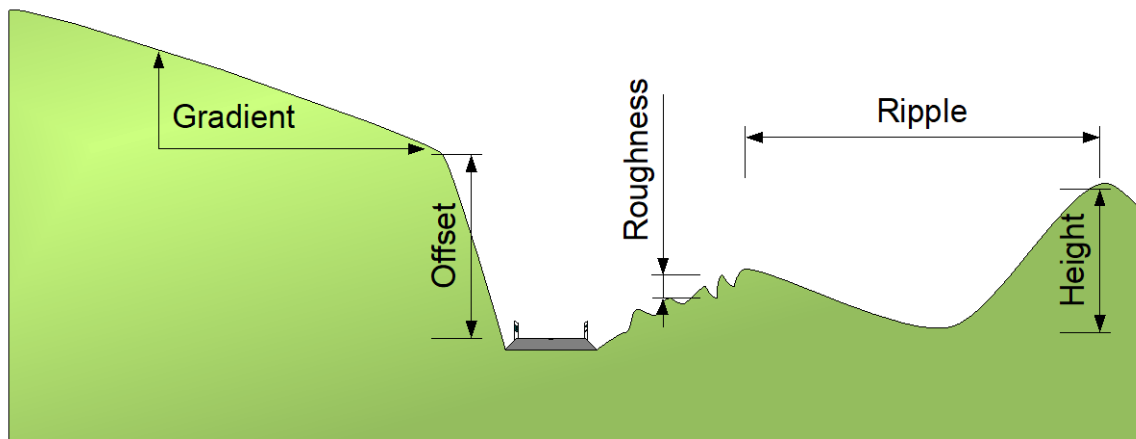


Abbildung 12 Landschafts-Parameter

Die Parameter **Offset** und **Gradient** können entlang der Strecke variiert werden. Dazu können optional die Parameter **OffsetEnd** und **GradientEnd** angegeben werden. Das Offset bzw. die Steigung der Landschaftshöhe werden dann vom Anfang bis zum Ende des Streckentyps linear interpoliert. Am Anfang der Strecke ist der Wert **Offset** bzw. **Gradient**, am Ende **OffsetEnd** bzw. **GradientEnd**. Dies kann genutzt werden, um eine sanfte Übergangsstrecke zwischen einem Tal und einer Bergkuppe zu definieren.

Weitere Parameter zur Landschaftsgestaltung, die für alle Landschaftstypen verfügbar sind, sind:

MinObjDist = <Abstand>;

Minimaler Abstand zur Fahrbahn in Meter, ab dem die Landschaftsobjekte gezeichnet werden sollen. Wertebereich: -2000 m ... 2000 m, Defaultwert: 0 m.

(Wird hier ein negativer Wert angegeben, so werden auf beiden Seiten der Straße Objekte platziert. Die Straße selbst wird aber freigehalten.)

MaxObjDist = <Abstand>;

Maximaler Abstand zur Fahrbahn in Meter, bis zu dem die Landschaftsobjekte gezeichnet werden sollen. Wertebereich: 0 m ... 2000 m, Defaultwert: 2000 m.

MinObjPos = <Streckenmeter>;

Minimaler Streckenmeter, ab dem für diesen Landschaftsgenerator Landschaftsobjekte gezeichnet werden sollen. Wertebereich: 0 m ... Länge des Streckentyps, Defaultwert: 0 m.

MaxObjPos = <Streckenmeter>;

Maximaler Streckenmeter, bis zu dem für diesen Landschaftsgenerator Landschaftsobjekte gezeichnet werden sollen. Wertebereich: 0 m ... Länge des Streckentyps, Defaultwert: Länge des Streckentyps.

Der minimale und maximale Objektabstand kann optional mit Hilfe der Parameter **MinObjDistEnd** und **MaxObjDistEnd** entlang der Strecke variiert werden. Die Objektabstände werden dann vom Anfang des Landschaftsgenerators bis zum Ende linear interpoliert.

Durch diese Parameter kann der Wirkungsbereich des Landschaftsgenerators auf ein Trapez eingeschränkt werden, beispielsweise um einzelne Baumreihen oder Hecken zu platzieren. Dies wird in Abbildung 13 verdeutlicht.

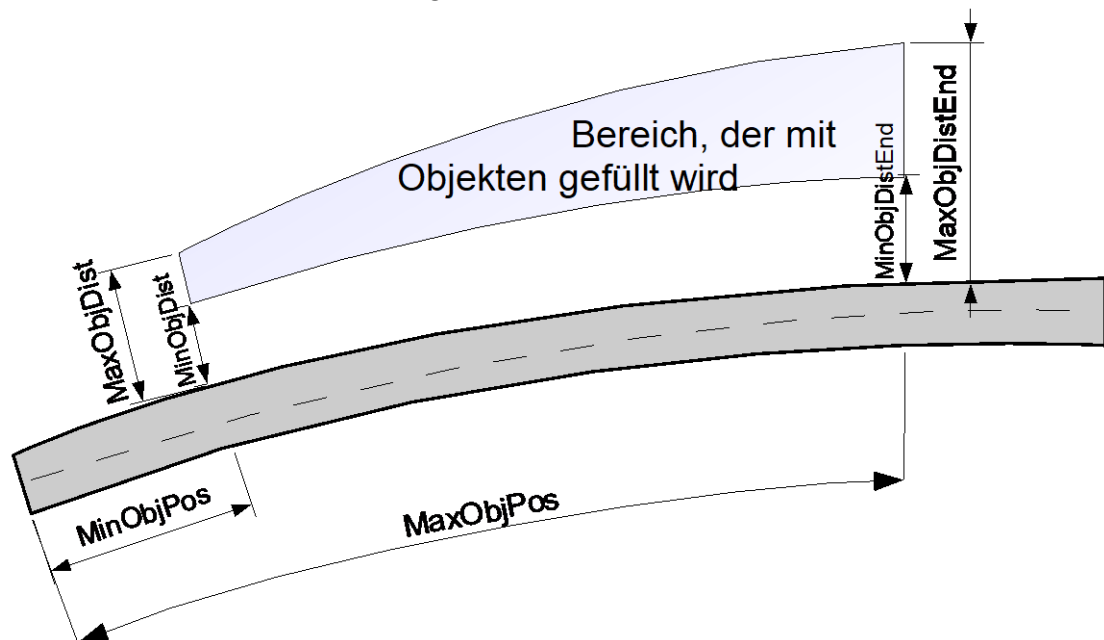


Abbildung 13 Einfluss der ObjDist- und ObjPos Parameter auf den Wirkungsbereich des Landschaftsgenerators

OverlayDensity = (<minDichte>, <maxDichte>);

Dichte von Overlay-Feldern. Felder, Wiesen oder Waldboden wird abhängig vom Landschaftstyp direkt auf die Landschaft gezeichnet. Wertebereich: 0 ... 1, Default-

wert 0.2. (Wenn der Wert 1 angegeben wird, wird die komplette Landschaftsoberfläche mit Overlay-Feldern gefüllt.)

MinScale = <Faktor>;

Minimaler Skalierungsfaktor für gleichmäßige Skalierung. Standardwert: 1 (keine Skalierung der Objekte).

MaxScale = <Faktor>;

Maximaler Skalierungsfaktor für gleichmäßige Skalierung. Standardwert: 1 (keine Skalierung der Objekte).

MinScaleZ = <Faktor>;

Minimaler Skalierungsfaktor für ungleichförmige Skalierung nur entlang der Z-Achse (Höhe). Standardwert: 1 (keine Skalierung der Objekte).

MaxScaleZ = <Faktor>;

Maximaler Skalierungsfaktor für ungleichförmige Skalierung nur entlang der Z-Achse (Höhe). Standardwert: 1 (keine Skalierung der Objekte).

Mit dem **Override**-Parameter kann festgelegt werden, ob die Objekte dieses Landschaftsgenerators auf anderen Objekten platziert werden dürfen. Die Objektgruppen, auf denen Objekte dieses Landschaftsgenerators platziert werden dürfen, werden als eine durch Komma getrennte Liste angegeben. Folgende Objektgruppen sind definiert:

Gruppenname	Beschreibung
Road	Von Straßen (Course- oder Area2-Streckentypen) belegte Gebiete
User	Vom Anwender im Objects-Abschnitt (siehe Kap. 2.5.6) einzeln eingefügte Objekte
Tree, Field, ...	Diverse Gruppen von Objekten, die von anderen Landschaftsgeneratoren platziert wurden. Es muss jeweils der Name der ObjectGroup lt. Grammatik 35 angegeben werden.

Falls der Override-Parameter nicht angegeben ist, werden die Objekte dieses Landschaftsgenerators nur dort platziert, wo sich keine anderen Objekte befinden.

Die folgenden Abschnitte beschreiben die einzelnen Landschaftstypen.

2.5.13.1 Landschaftstyp Alley1 (Allee)

Erzeugt eine Allee aus kleineren Straßenbäumen.

Für diesen Landschaftstyp sind keine zusätzlichen Parameter verfügbar.

2.5.13.2 Landschaftstyp Alley2 (Allee)

Erzeugt eine Allee aus schlanken, hohen Bäumen.

Für diesen Landschaftstyp sind keine zusätzlichen Parameter verfügbar.

2.5.13.3 Landschaftstyp AncientVillage (Dorf mit Fachwerkhäusern)

Erzeugt ein Dorf mit Fachwerkhäusern und kleineren Bäumen.

AncientHouseDensity = (<min>, <max>);

Durchschnittliche Anzahl von Fachwerkhäusern pro 10000 m².

SmallTreeDensity = (<min>, <max>);

Durchschnittliche Anzahl von Bäumen pro 10000 m². (Hier sind nur kleinere Bäume zugelassen.)

2.5.13.4 Landschaftstyp Bushes (Büsche)

Platziert Büsche auf der Landschaft.

BushDensity = (<min>, <max>);

Durchschnittliche Anzahl von Büschen pro 10000 m².

2.5.13.5 Landschaftstyp CityWall (Stadtmauer)

Erzeugt eine Stadtmauer entlang der Straße. Dieser Landschaftsgenerator kann als dekorative Alternative zu einer Lärmschutzwand verwendet werden. Es sind keine zusätzlichen Parameter verfügbar.

2.5.13.6 Landschaftstyp CountryVillage (Dorf mit Steinhäusern)

Erzeugt ein Dorf mit Steinhäusern und kleineren Bäumen.

CountryHouseDensity = (<min>, <max>);

Durchschnittliche Anzahl von Steinhäusern pro 10000 m².

SmallTreeDensity = (<min>, <max>);

Durchschnittliche Anzahl von Bäumen pro 10000 m². (Hier sind nur kleinere Bäume zugelassen.)

2.5.13.7 Landschaftstyp DryMeadow (Trockene Weide)

Erzeugt eine ausgetrocknete Weide aus Gräsern.

Für diesen Landschaftstyp sind keine zusätzlichen Parameter verfügbar.

2.5.13.8 Landschaftstyp Farmed (Felder)

Erzeugt Korn- und Maisfelder, die durch Feldwege erschlossen sind.

FieldWidthMin = (<min>, <max>);

Minimalbreite eines Felds in Metern (entlang der Straße gemessen). Standard: 200.

FieldWidthMax = (<min>, <max>);

Maximalbreite eines Felds in Metern (entlang der Straße gemessen). Standard: 300.

FieldSpacing = (<min>, <max>);

Breite des Zwischenraums zwischen zwei Feldern in Metern. Standard: 10.

FieldSetback = (<mittel>, <abw>);¹⁴

Abstand zwischen Straßenrand und Feldern in Metern. Dieser Bereich wird mit Graspflanzen gefüllt, falls er groß genug ist. Standard: (5, 2).

PlantDepth = (<min>, <max>);

Abstand von der Straße in Metern, bis zu dem einzelne Feldpflanzen dargestellt werden. Standard: 25.

WaySetback = (<mittel>, <abw>);

Abstand zwischen Straßenrand und Feldwegen in Metern. Standard: (0, 0) (geeignet für Landstraßen). Für Autobahnen sollten Werte > 0 betragen.

Hinweis: Damit die **Setback**-Parameter korrekt funktionieren, sollte für den Landschaftstyp **Farmed** die Einstellung **MinObjdist** = 0 verwendet werden (dies ist der Standardwert).

2.5.13.9 Landschaftstyp Forest (Nadelwald)

Erzeugt einen Nadelwald. Am Rand des Waldes zur Straße hin werden zusätzlich Unterholz und bodendeckende Pflanzen platziert.

Treedist = (<mittel>, <abw>);

Abstand zwischen Bäumen in Metern. Standard: (10, 1).

Edgedist = (<mittel>, <abw>);

Abstand zwischen Waldrandpflanzen (kleine Bäume und Sträucher) in Metern. Standard: (6, 1).

Objangle = <angle>;

Winkel der Baumreihen in rad (Radians), wobei 0 eine quer zur Straße verlaufende Baumreihe ist. Standard: 0.5.

TreeMinDepth = (<min>, <max>);

Abstand von der Straße, ab dem vollflächig Bäume platziert werden. Standard: 6.

TreeDepth = (<min>, <max>);

Abstand von der Straße, bis zu dem vollflächig Bäume platziert werden. Standard: 40.

PlantDepth = (<min>, <max>);

Abstand von der Straße, bis zu dem bodendeckende Pflanzen platziert werden. Standard: 25.

EdgeDepth = (<min>, <max>);

Abstand von der Straße, bis zu dem Waldrandpflanzen platziert werden. Standard: 20.

2.5.13.10 Landschaftstyp FruitTree (Obstbäume)

Erzeugt Obstbaugebiete. Die Obstbäume stehen auf einer Wiese.

Treedist = (<mittel>, <abw>);

Abstand zwischen Obstbäumen in Metern. Standard: (16, 1).

¹⁴ Hierbei bezeichnet <mittel> den Mittelwert und <abw> die Standardabweichung einer Gauß'schen Normalverteilung.

Objangle = *<angle>*;

Winkel der Baumreihen in rad (Radians), wobei 0 eine quer zur Straße verlaufende Baumreihe ist. Standard: 0.5.

TreeMinDepth = (*<min>*, *<max>*);

Abstand von der Straße, ab dem vollflächig Bäume platziert werden. Standard: 4.

TreeDepth = (*<min>*, *<max>*);

Abstand von der Straße, bis zu dem vollflächig Bäume platziert werden. Standard: 100.

PlantDepth = (*<min>*, *<max>*);

Abstand von der Straße, bis zu dem bodendeckende Wiesenpflanzen platziert werden. Standard: 25.

2.5.13.11 Landschaftstyp Grassland (Wiese)

Erzeugt eine Wiese aus verschiedenen Gräsern und Blumen.

Für diesen Landschaftstyp sind keine zusätzlichen Parameter verfügbar.

2.5.13.12 Landschaftstyp GroundFog (Bodennebel-Felder)

Platziert ein Feld von Nebelschwaden.

Objdist = (*<mittel>*, *<abw>*);

Abstand einzelner Nebelschwaden in Metern. Mit diesem Parameter kann die Dichte des Nebels beeinflusst werden. Die Standardeinstellung ist *Objdist* = (3.6, 0.5), wodurch sich ein dichtes Nebelfeld ergibt.

2.5.13.13 Landschaftstyp Hedge (Hecke)

Erzeugt eine Hecke aus kleinen Bäumen und Büschen.

Objangle = *<angle>*;

Winkel der Heckenreihen in rad (Radians), wobei 0 eine quer zur Straße verlaufende Baumreihe ist.

Objdist = (*<mittel>*, *<abw>*);

Abstand der Büsche. Dieser wird normalverteilt mit dem Mittelwert *<mittel>* und der Standardabweichung *<abw>* gewählt. Die Standardeinstellung ist *Objdist* = (6, 2).

Achtung: Falls mit *LandscapeTypeExtra* ein schmales, quer zur Straße verlaufendes Heckenband erzeugt wird, muss *Objangle* = 0 gesetzt werden, damit alle Bäume und Büsche platziert werden.

2.5.13.14 Landschaftstyp Inhabited (Bungalow-Siedlung)

Erzeugt eine Bungalow-Siedlung mit Personen und kleineren Bäumen.

HouseDensity = (*<min>*, *<max>*);

Durchschnittliche Anzahl von Bungalows pro 10000 m².

PersonDensity = (*<min>*, *<max>*);

Durchschnittliche Anzahl von Personen pro 10000 m².

SmallTreeDensity = (<min>, <max>);

Durchschnittliche Anzahl von Bäumen pro 10000 m². (Hier sind nur kleinere Bäume zugelassen.)

2.5.13.15 Landschaftstyp LeafForest (Laubwald)

Erzeugt einen Laubwald. Am Rand des Waldes zur Straße hin werden zusätzlich Unterholz und bodendeckende Pflanzen platziert.

Treedist = (<mittel>, <abw>);

Abstand zwischen Bäumen in Metern. Standard: (9, 1)

Edgedist = (<mittel>, <abw>);

Abstand zwischen Waldrandpflanzen (kleine Bäume und Sträucher) in Metern. Standard: (6, 1).

Objangle = <angle>;

Winkel der Baumreihen in rad (Radians), wobei 0 eine quer zur Straße verlaufende Baumreihe ist. Standard: 0.5.

TreeMinDepth = (<min>, <max>);

Abstand von der Straße, ab dem vollflächig Bäume platziert werden. Standard: 4.

TreeDepth = (<min>, <max>);

Abstand von der Straße, bis zu dem vollflächig Bäume platziert werden. Standard: 40.

PlantDepth = (<min>, <max>);

Abstand von der Straße, bis zu dem bodendeckende Pflanzen platziert werden. Standard: 25.

EdgeDepth = (<min>, <max>);

Abstand von der Straße, bis zu dem Waldrandpflanzen platziert werden. Standard: 20.

2.5.13.16 Landschaftstyp Meadow (Weide)

Erzeugt eine Weide oder frisch gemähte Wiese aus Gräsern.

Für diesen Landschaftstyp sind keine zusätzlichen Parameter verfügbar.

2.5.13.17 Landschaftstyp Meadows (Weiden)

Erzeugt Weiden, die durch Zäune begrenzt und durch Feldwege erschlossen sind. Auf den Weiden können optional Bäume und/oder Tiere platziert werden.

MeadowWidthMin = (<min>, <max>);

Minimalbreite einer Weide in Metern (entlang der Straße gemessen). Standard: 250.

MeadowWidthMax = (<min>, <max>);

Maximalbreite einer Weide in Metern (entlang der Straße gemessen). Standard: 400.

MeadowSpacing = (<min>, <max>);

Breite des Zwischenraums zwischen zwei Weiden in Metern. Standard: 10.

MeadowSetback = (<mittel>, <abw>);

Abstand zwischen Straßenrand und Weidezäunen in Metern. Dieser Bereich wird mit Graspflanzen gefüllt, falls er groß genug ist. Standard: (5, 2).

PlantDepth = (<min>, <max>);

Abstand von der Straße in Metern, bis zu dem einzelne Graspflanzen dargestellt werden. Standard: 25.

WaySetback = (<mittel>, <abw>);

Abstand zwischen Straßenrand und Feldwegen in Metern. Standard: (0, 0) (geeignet für Landstraßen). Für Autobahnen sollten Werte > 0 eingetragen werden.

GadgetDensity = (<min>, <max>);

Durchschnittliche Anzahl von Tieren oder Heuballen pro 10000 m².

TreeDensity = (<min>, <max>);

Durchschnittliche Anzahl von freistehenden Bäumen pro 10000 m².

Hinweis: Damit die **Setback**-Parameter korrekt funktionieren, sollte für den Landschaftstyp **Meadows** die Einstellung **MinObjdist** = 0 verwendet werden (dies ist der Standardwert).

2.5.13.18 Landschaftstyp RoadsideBush (Büsche)

Erzeugt eine Reihe von Büschen entlang der Straße. Dieser Landschaftsgenerator wird beispielsweise verwendet, um den Mittelstreifen einer Autobahn zu begrünen.

Objdist = (<mittel>, <abw>);

Zwischenraum zwischen zwei aufeinanderfolgenden Büschen. Dieser wird normalverteilt mit dem Mittelwert <mittel> und der Standardabweichung <abw> gewählt. Die Standardeinstellung ist *Objdist* = (0.2, 0.2).

ObjDensity = (<min>, <max>);

Wahrscheinlichkeit (zwischen 0.0 und 1.0), dass ein Objekt platziert wird. (Andernfalls bleibt ein Platz in der Buschreihe leer.) Standard: 1.0

2.5.13.19 Landschaftstyp SoundBarrierMetal (Lärmschutzwand)

Erzeugt eine Lärmschutzwand aus Metallbauteilen, die entlang der Straße platziert werden.

Für diesen Landschaftstyp sind keine zusätzlichen Parameter verfügbar.

2.5.13.20 Landschaftstyp SoundBarrierWood (Lärmschutzwand)

Erzeugt eine Lärmschutzwand aus Holzbauteilen, die entlang der Straße platziert werden.

Für diesen Landschaftstyp sind keine zusätzlichen Parameter verfügbar.

2.5.13.21 Landschaftstyp Suburb (Vorort mit modernen Wohnhäusern)

Erzeugt einen Vorort mit modernen Wohnhäusern und kleineren Bäumen.

SuburbHouseDensity = (<min>, <max>);

Durchschnittliche Anzahl von Wohnhäusern pro 10000 m².

SmallTreeDensity = (<min>, <max>);

Durchschnittliche Anzahl von Bäumen pro 10000 m². (Hier sind nur kleinere Bäume zugelassen.)

2.5.13.22 Landschaftstyp Tunnel

Platziert eine Reihe von Tunnelabschnitten entlang der Straße, die für eine zweispurige Landstraße oder für eine Fahrtrichtung einer Autobahn mit zwei Fahrspuren geeignet sind.

In regelmäßigen Abständen werden spezielle Tunnelabschnitte (z.B. Nothaltebucht, Notrufsäule, Fluchtweg) integriert.

Obj<n>Occurrence = (<min>, <max>);

Gibt die Häufigkeit an, mit welcher das *n*-te Tunnelobjekt auftritt. Das Objekt wird einmal alle *Occurrence* Abschnitte platziert. Die Standardwerte sind in folgender Tabelle beschrieben:

n	Objekt	Occurrence
1	Nothaltebucht	43
2	Fluchtweg	23
3	Notrufsäule	13
4	normales Tunnelsegment	1

2.5.13.23 Landschaftstyp Vineyard (Weingarten bzw. Weinberg)

Erzeugt einen Weingarten oder Weinberg.

Für diesen Landschaftstyp sind keine zusätzlichen Parameter verfügbar.

2.5.13.24 Landschaftstyp Wooded (baumbestandenes Gebiet)

Erzeugt ein baumbestandenes Gebiet. Optional können einzelne Häuser platziert werden.

TreeDensity = (<min>, <max>);

Durchschnittliche Anzahl von freistehenden Bäumen pro 10000 m².

HouseDensity = (<min>, <max>);

Durchschnittliche Anzahl von Häusern pro 10000 m².

2.5.14 Topographie: Form und Höhe der Landschaft

Die Form der Landschaft, die die definierten Strecken umgibt, wird zunächst durch die Strecken selbst bestimmt.

Der einfachste Fall tritt ein, wenn keine Landschaftsgeneratoren (Kap. 2.5.13) definiert werden bzw. wenn das Höhenmodell der Landschaftsgeneratoren keine Parameter enthält.

In diesem Fall befindet sich die Landschaft in der Nähe einer Strecke stets auf Höhe der Straße. Wenn mehrere Strecken sich kreuzen oder parallel zueinander verlaufen, bildet die Landschaft einen möglichst gleichmäßigen, sanften Übergang zwischen den Höhen dieser Strecken (siehe Abbildung 14).



Abbildung 14 Landschaftsübergang zwischen zwei Strecken

Durch Landschaftsgeneratoren kann die Landschaft in der Umgebung der Strecken beeinflusst werden und es können Hügel, Berge und kleine Unebenheiten erzeugt werden. Hierzu werden die Höhenparameter **Height**, **Offset**, **Ripple**, **Gradient**, **Roughness** und **MaxHillDist** der Landschaftsgeneratoren **LandscapeTypeLeft** und **LandscapeTypeRight** verwendet (siehe Kap. 2.5.13).

Zusätzlich kann die Höhe der Landschaft auch in einem lokal begrenzten Bereich durch entlang der Strecke platzierte Objekte (mit Hilfe des **Objects**-Abschnitts, siehe Kap. 2.5.6) beeinflusst werden.

Achtung: Nur die manuell im Objects-Abschnitt aufgeführten Objekte beeinflussen die Landschaftshöhe, nicht aber die durch einen Landschaftsgenerator automatisch platzierte Objekte.. Dies ist der Fall, wenn die SIG- bzw. SGE-Objekte das Attribut **HeightInfoX** enthalten. Durch **HeightInfoX** kann in an jedem Punkt eines Gitters mit frei wählbarem Gitterabstand unterhalb des Objekts die Landschaftshöhe festgelegt werden. Somit können beispielsweise einzelne Senken oder Hügel realisiert werden.

Im Lieferumfang von SILAB sind zwei Objekte enthalten, die Landschaftshöhen beeinflussen:

- **area2.hills.noisebarrier**: Ein Lärmschutzwall (Länge 40 m, Breite 10 m, Höhe 2.5 m), der entlang von Course-Instanzen aufgestellt werden kann.
- **environment.water.lake200h**: Ein See (Ausmaße ca. 140x140 m) mit kleiner, asymmetrischer Insel.

Näheres zur Definition von **HeightInfoX** finden Sie in „Referenz Visualisierung SGE“.

2.5.15 OBS-IDs

Abschnitte von Spuren in Courses können über sog. OBS-IDs markiert werden. Dabei handelt es sich um Zahlen, die einer Spur zwischen einem Start-Streckenmeter

und einem End-Streckenmeter zugewiesen werden. Die OBS-ID der Spur, auf der sich der Fahrer aktuell befindet, wird als Ausgang der DPUSCNX zur Verfügung gestellt. Mit dem Entwicklerpaket iSCNX ist es möglich, die OBS-ID an beliebigen Stellen des Streckennetzes im Sichtbarkeitsbereich des Fahrers abzufragen.

Grammatik 19 Definition von OBS-IDs

```
<OBS-IDs>→
  OBSIDs =
  {
    (<Spurindex>, <s0>, <s1>, <ID>)
  };
```

Beispiel:

Angenommen, das Querschnittsprofil eines Course hat zwei Spuren mit den Indizes 2 und 3, die in dieselbe Richtung führen. Ein kritisches Ereignis findet im Bereich von Streckenmeter 100 bis 500 auf diesen beiden Spuren statt. Die aufgezeichneten Daten sollen in diesem Bereich nach der Simulation ausgewertet werden. Mithilfe von OBS-IDs kann der Bereich folgendermaßen gekennzeichnet werden:

```
OBSIDs =
{
  (2, 100, 500, 24),
  (3, 100, 500, 24)
};
```

Wird der Ausgang *OBSLaneIdx* der DPUSCNX mit der DPUDataFile als Spalte aufgezeichnet, findet sich in dieser Spalte genau dann der Wert 24, wenn sich der Fahrer in dem markierten Bereich aufgehalten hat.

2.5.16 Sichtbarkeitsradien

Wenn an einem Port einer Course-Streckeninstanz ein anderes Streckenstück angeschlossen ist, ist dieses sichtbar, solange sich der Fahrer auf einer Position befindet, die weniger als der zugehörige Sichtbarkeitsradius entfernt ist. Dabei wird die Distanz entlang der Straße (d.h. als Streckenmeter) gemessen.

Der Standardwert für den Sichtbarkeitsradius beträgt bei Courses 2 km. Er wird im Parameter *VisibleRoadLength* der DPUSCNX vorgegeben. Für einzelne Szenarien ist es möglicherweise sinnvoll, andere Werte für den Sichtbarkeitsradius festzulegen. Insbesondere gilt dies dann, wenn topologische Schleifen aufgebaut werden sollen (siehe auch Kap. 2.4.2.2). Wenn – zum Beispiel mittels einem verdeckenden Waldstück – sichergestellt ist, dass der Port erst aus einer deutlich geringeren Distanz sichtbar wird, kann der Sichtbarkeitsradius deutlich reduziert und somit die topologische Schleife stark verkürzt werden.

Dazu werden die in Grammatik 20 beschriebenen Definitionen in die Course-Typdefinition eingefügt.

Grammatik 20 Definition von Sichtbarkeitsradien

<Sichtbarkeitsradien>→

Begin.rVis = <Streckenlänge in m>;

End.rVis = <Streckenlänge in m>;

2.5.17 GPS-Koordinaten

Optional kann die geographische Lage eines Course-Streckentyps angegeben werden. Diese Information ist möglicherweise nötig, um bestimmte Assistenzsysteme zu simulieren und wird während der Simulation über die GPS-Ausgänge der DPUSCNX bereitgestellt.

Dazu werden die in Grammatik 21 beschriebenen Definitionen in die Course-Typdefinition eingefügt.

Grammatik 21 Definition von GPS-Koordinaten

<GPS-Koordinaten>→

GPSData = (<Breitengrad>, <Längengrad>, <Himmelsrichtung>);

Die Angaben definieren die Position des Course-Anfangs (Port *Begin*) in der Einheit Grad:

- Der <Breitengrad> ist der Winkelabstand der Course-Position zum Äquator; positive Angaben sind nördlich des Äquators.
- Der <Längengrad> ist der Winkelabstand der Course-Position zum Nullmeridian; positive Angaben liegen östlich von Greenwich.
- Die <Himmelsrichtung> ist die Richtung der Straße am Course-Anfang; 0° ist „nach Norden“, 90° „nach Osten“ usw.

Innerhalb des Courses werden die Koordinaten auf Basis der tatsächlichen Streckenausdehnung weitergerechnet.

Der Anwender ist selbst dafür verantwortlich, die Koordinaten mehrerer Course- oder Area2-Streckentypen so festzulegen, dass sich beim Übergang zwischen den Strecken keine Sprünge ergeben.

Beispiel:

Die Balthasar-Neumann-Promenade in Würzburg beginnt an der Kreuzung zur Theaterstraße vor der Würzburger Residenz und verläuft von dort aus in südliche Richtung. (vgl. Google Maps: <https://maps.google.com/maps?q=49.793904,9.937234>)

Möchte man diese Straße als Course modellieren, sind die passenden GPSData-Angaben:

GPSData = (49.793904, 9.937234, 205);

2.6 Streckentyp Area2

Area2-Streckentypen können verwendet werden, um einfache Fahrbahnverengungen, Querschnittsübergänge, T-Kreuzungen bis hin zu kompletten Innenstadt-Häuserblocks zu realisieren. Aufgrund ihrer hohen Komplexität werden Area2-

Streckentypen nicht im Skript, sondern mit dem grafischen Editor SILABAEEdit erstellt. Eine ausführliche Anleitung dazu ist im Dokument „Handbuch SILABAEEdit“ zu finden.

Für die Integration der erzeugten Area2-Szenarien in eine Karte gibt es zwei Möglichkeiten:

- Sie können direkt als Streckentypen aus SILABAEEdit exportiert werden. Dadurch kann die Area2 in ein Modul integriert werden, in dem auch weitere Course-Streckentypen enthalten sind. Verkehr (aus TRFX) kann in die Kreuzung hinein- und wieder herausfahren. Der in SILABAEEdit definierte Verkehr wird allerdings in diesem Fall ignoriert. Näheres hierzu siehe Kap. 2.4.2.3.
- Area2-Szenarien können auch als Module aus SILABAEEdit exportiert werden. Dabei wird automatisch ein Modul erzeugt, das nur den Area2-Streckentyp einmal instanziiert und dessen Ports auf gleich benannte Modulports weiterverbindet. Der in SILABAEEdit definierte Verkehr und die Fußgänger werden in das Modul eingebunden. Nähere Informationen hierzu finden Sie in Kap. 2.4.1.

2.7 Strecken-Events (Hedgehogs)

Bei Strecken-Events (im Skript „Hedgehogs“ genannt) handelt es sich um für den Testfahrer unsichtbare Markierungen auf den Spuren einer Strecke, durch deren Überfahren die Parameter und nicht verschalteten Eingänge von DPUs verändert werden können. Jeder DPU, die über Events gesteuert werden soll, wird hierzu eine Instanz der DPUSCNXHedgehogKiller zugeordnet. Diese DPU analysiert in jedem Simulationsschritt die überfahrenen Events eines bestimmten Typs (im Skript und in DPUSCNXHedgehogKiller „Familie von Hedgehogs“ genannt) und stellt deren Argumente an ihren Ausgängen zur Verfügung.

In der Definition einer Datenbasis können die Strecken-Events sowohl in die Streckentypen (Course-Typen bzw. Area2-Typen) als auch in Streckeninstanzen eingefügt werden. Durch ersteres ist es möglich, für alle Instanzen eines Streckentyps gemeinsame Strecken-Events festzulegen. Zweiteres ermöglicht jeder Instanz eigene Events zuzuweisen.

In Course-Typen hat die Definition von Events folgende Form:

Grammatik 22 Events in Course-Typen

<Events Course-Typ> →

Hedgehogs =

{

(,)<Hedgehog Course-Typ>

};

<Hedgehog Course-Typ > →

(<Streckenmeter>, **Normal** | **Reverse**, <Spurnummer>, <Familie>

(,)<Hedgehogargument>)

<Hedgehogargument> →

<Name> , <Wert>

Jeder Hedgehog eines Course-Typs besteht aus folgenden Angaben:

- **<Streckenmeter>**: Der Abstand des Strecken-Events entlang der Spur zum Beginn der Straße.
- **Normal | Reverse**: Das Strecken-Event gilt als überfahren, wenn der Fahrer in Richtung der Spur (**Normal**) oder gegen die Richtung (**Reverse**), auf der es liegt, fährt.
- **<Spurnummer>**: Die Nummer der Spur, auf der das Strecken-Event liegt. Die am weitesten links liegende Spur der Straße (vom Beginn der Straße aus gesehen) hat die Nummer 0.
- **<Familie>**: Der Name der Familie, zu der dieser Hedgehog gehört.
- **<Hedgehogargument>**: Jedes Argument besteht aus einem Namen und einem Wert.

Hedgehogs in Area2-Typen werden mit Hilfe des Editors SILABAEEdit eingefügt. Die Vorgehensweise wird im Dokument „Handbuch SILABAEEdit“ erläutert.

Grammatik 23 Events in Streckeninstanzen

<Events Streckeninstanzen >→

Hedgehogs =

{

(,)[<Hedgehog Course-Instanz>]

};

<Hedgehog Course-Instanz>→

(<Name Streckeninstanz>, <Streckenmeter>, **Normal | Reverse**,

<Spurnummer>, <Familie>,

(,)<Hedgehogargument (→Grammatik 22 Seite 60)>

);

Events von Streckeninstanzen haben als erstes Tupel-Element den Namen der Streckeninstanz, in die sie eingefügt werden sollen. Die restlichen Elemente sind mit denen in den entsprechenden Events in Streckentypen identisch.

Events in Area2-Instanzen werden mit Hilfe des Werkzeugs „Event“ der Schicht „Verkehr“ in SILABAEEdit platziert.

Um Events beim Überfahren auszulösen, muss die DPUSCNXHedgehogKiller eingebunden werden. Diese DPU besitzt Ausgänge, die beim Überfahren von Events ihren Wert ändern. Damit können z.B. Soundausgaben, Aktivierung von Umgebungs- oder Wetterbedingungen, oder Ereignisse in der Fußgängersimulation ausgelöst werden.

2.8 Aufsetzpunkte (SetupPoints)

Beim Start der Simulation wird der Fahrer auf einem Port eines Moduls oder einer Course- bzw. Area2- Instanz aufgesetzt. Es können mehrere Aufsetzpunkte definiert werden. Über einen dafür vergebenen Namen wird der Startpunkt mit dem Parameter *SetupPoint* der DPUSCNX (vgl. Dokument „DPUSCNX“) selektiert. Alternativ kann er auch über die Launchdata *SCN_SetupPoint* gesetzt werden.

Grammatik 24 Aufsetzpunkte

<Aufsetzpunkte> →

SetupPoints =

{

(,)<Aufsetzpunkt>

};

<Aufsetzpunkt> →

("<Name>", <Modulinstantz>.<Streckeninstanz>.<Port>) |

("<Name>", <Modulinstantz>.<Streckeninstanz>.<Port>, <Spurwechsel>)

Der Fahrer steht dabei auf dem gewählten Port und blickt von dort aus in die gewählte Knoteninstanz hinein. Es gibt folgende Möglichkeiten, Aufsetzpunkte zu definieren:

- Aufsetzen auf **Courses**: <Port> ist **Begin**. Das Aufsetzen am Ende eines Courses (Port **End**) wird nicht unterstützt.
Beispiel: ("SetupCourse", Mod1.Course1.Begin)
- Aufsetzen auf **Area2s**: <Port> ist **Port1**, ..., **Portn**
Beispiel: ("SetupArea", Mod1.Area1.Port2)
- Aufsetzen auf **Modulport**: Hier entfällt der Name der <Streckeninstanz>. Es wird auf einen der definierten Modulports aufgesetzt: <Port> ist **Port1**, ..., **Portn**.
Beispiel: ("SetupModule", Mod1.Port3)

Normalerweise wird der Fahrer auf die am weitesten rechts befindliche Spur der Straße aufgesetzt. Durch die zusätzliche Angabe von <Spurwechsel> kann angegeben werden, dass das Fahrzeug beim Aufsetzen um die Anzahl <Spurwechsel> Spuren nach links versetzt wird.

3 HILFSPROGRAMME

Bei der Bearbeitung der SCNX Datenbasis-Definition in einem Plaintext-Editor können Hilfsprogramme zur Syntax-Überprüfung der Definition und zur Voransicht genutzt werden. Diese werden durch die beiden Anwendungen „SILABCFGXCheck“ und „SILABPreview“ bereitgestellt.

3.1 SILABCFGXCheck

Die Konsolenanwendung SILABCFGXCheck überprüft eine Konfigurationsdatei für SILAB auf syntaktische Korrektheit. Dies erfolgt in drei Schritten:

1. Zunächst wird die als Kommandozeilenparameter angegebene Datei als allgemeine SILAB-Konfigurationsdatei geladen. Die Ausgabe „Syntax-Check:“ informiert über den Erfolg dieser Operation.
2. Wenn die Konfigurationsdatei eine Datenbasis-Definition enthält (siehe Grammatik 1), wird die angegebene Datenbasis initialisiert. Die Ausgabe „SCNX-Check:“ informiert über den Erfolg dieser Operation.
3. Enthält die Konfigurationsdatei zusätzlich eine Verkehrs-Definition (siehe „Referenz Verkehrssimulation TRFX“), wird diese geladen und das Resultat unter „TRF-Check:“ gemeldet.

Um möglichst effektiv mit SILABCFGXCheck zu arbeiten, empfiehlt es sich, die Anwendung in den benutzten Plaintext-Editor einzubinden. Der verwendete Editor muss dazu die Möglichkeit bieten, externe Kommandos einzubinden. Die meisten Editoren – beispielsweise „Geany“, „TextPad“ und „Crimson Editor“ – bieten eine solche Möglichkeit an. Die Vorgehensweise wird in der *Installationsanleitung SILAB 7.1*, Abschnitt *Einrichten eines Texteditors*, beschrieben.

3.2 SILABPreview

Die Hilfsanwendung SILABPreview bietet eine Vorschau eines Parcours und ermöglicht den schnellen Test auf einem einzelnen Arbeitsplatzrechner. Dabei kann eine schematische zweidimensionale Ansicht dargestellt werden, die besonders nützlich ist, um Situationen zu überblicken.

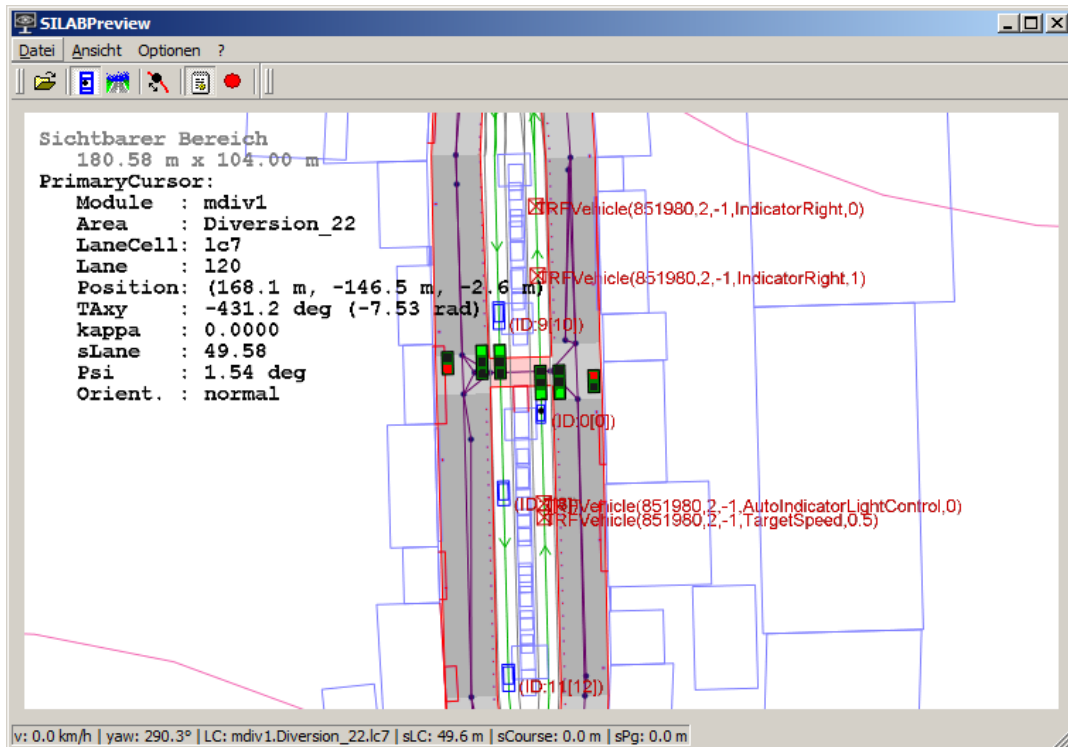


Abbildung 15 Schematische 2D-Ansicht in SILABPreview

Wenn der Rechner über eine leistungsfähige 3D-Grafikkarte verfügt, kann die aus SILAB bekannte dreidimensionale Darstellung verwendet werden.



Abbildung 16 3D-Ansicht in SILABPreview

In SILABPreview wird die Datenbasis sowie Verkehr und Fußgänger genau wie in SILAB simuliert und dargestellt. Die Bedienung erfolgt über die Tastatur. Da SILAB-

Preview die DPU-Konfiguration nicht auswertet und auch keine DPUs lädt, muss die geladene Konfigurationsdatei noch keine DPU-Konfiguration enthalten, um sie in SILABPreview betrachten zu können. Auch Konfigurationen, die von einem Simulator mit mehreren PCs stammen, können auf einem einzelnen Arbeitsplatz geöffnet werden. Allerdings werden daher alle Funktionalitäten ignoriert, die durch die Einbindung zusätzlicher DPUs¹⁵ bereitgestellt werden.

Laden einer Konfiguration

Nach dem Start von SILABPreview kann eine Konfiguration über den Menübefehl *Datei → Öffnen* geladen werden. Die Konfiguration muss eine Datenbasis-Definition enthalten. Falls eine Verkehrs- oder Fußgängerdefinition vorhanden ist, werden diese ebenfalls ausgewertet. Eine DPU-Konfiguration wird nicht benötigt und ggf. ignoriert. Wenn die Konfigurationsdatei syntaktisch korrekt ist, erscheint der Setup-Dialog (Abbildung 17). Hier kann der Aufsetzpunkt (Karte und SetupPoint) ausgewählt und Einstellungen zur Vorschau durchführt werden.

Folgende Einstellungen sind möglich:

- | | |
|-------------------------|---|
| Vorschaubereich: | Legt die anfängliche Größe des dargestellten Bereichs in der 2D-Ansicht fest. Die Größe kann während der Simulation mit den Bild-auf und Bild-ab Tasten geändert werden. |
| Sichtweite: | Legt fest, wie weit vor und hinter dem Fahrer die Straße gezeigt wird. Entspricht dem Parameter <code>VisibleRoadLength</code> der DPUSCNX. |
| 3D-Ansicht ermöglichen: | Wenn die Schaltfläche aktiviert ist, wird die 3D-Ansicht vorbereitet, und es kann dann während der Vorschau zwischen 2D- und 3D-Modus gewählt werden. Dies kann bei großen Strecken einige Zeit dauern und die Ladezeit stark verlängern. |
| Spurgebund. Bewegung: | Legt fest, ob anfangs die spurgebundene oder die freie Bewegung gewählt ist. |

¹⁵ Anstelle DPUs zu laden, startet SILABPreview die Komponenten SCNX und SCNXSGE mit ihren Standardeinstellungen. Falls eine Verkehrsdefinition vorhanden ist, werden auch TRFX und TRFSGE gestartet. Falls Fußgängerdefinitionen vorhanden sind, werden zusätzlich MOV und MOPSGE gestartet. Zur Eingabe wird eine Tastaturansteuerung verwendet, die ähnlich der `DPUKeyDX8Joystick` funktioniert.

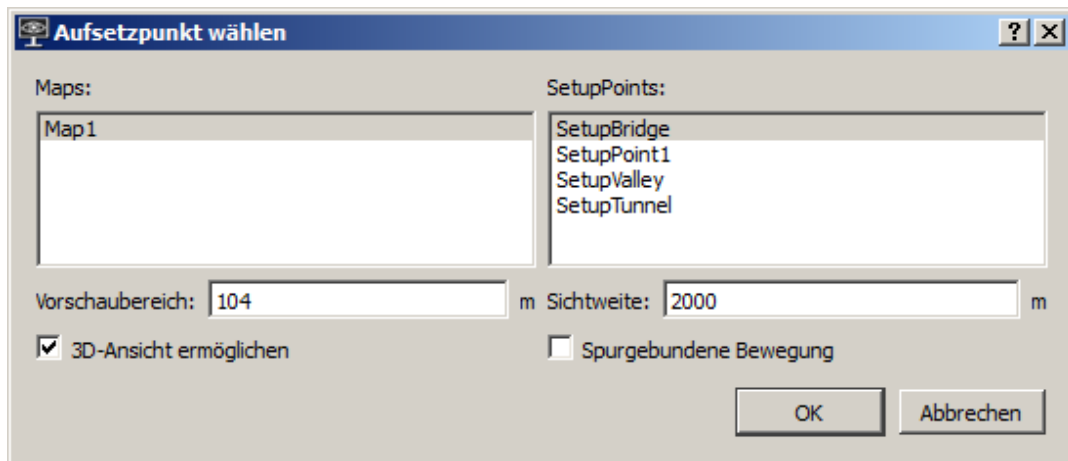


Abbildung 17 Setupdialog in SILABPreview

Integration in Text-Editoren

SILABPreview kann auch direkt als externe Anwendung in einen Plaintext-Editor integriert werden. Die Integration ist in der *Installationsanleitung SILAB <<Version>>*, Abschnitt *Einrichten eines Texteditors*, näher beschrieben.

Bedienung während der Vorschau

Die meisten Einstellungen können während der Vorschau über die Toolbar geändert werden:



2D-Ansicht auswählen

Wählt die schematische 2D-Ansicht aus der Vogelperspektive.



3D-Ansicht auswählen

Wählt die 3D-Ansicht wie in SILAB. Diese Option ist nur verfügbar, wenn sie beim Setup-Dialog nicht abgeschaltet wurde.



Spurgebundene/freie Bewegung

Schaltet zwischen spurgebundener Bewegung (wenn der Knopf gedrückt ist) und freier Bewegung (ähnlich wie in SILAB; wenn der Knopf nicht gedrückt ist) um.



Konfiguration der 2D-Ansicht

In diesem Fenster kann genau festgelegt werden, was in der 2D-Ansicht zu sehen ist. Eine Beschreibung der einzelnen Parameter ist in der Dokumentation zur DPUBirdsEye zu finden.



Simulation anhalten

Stoppt die Simulation und gibt den Prozessor frei.



Konfiguration öffnen

Lädt eine andere Konfigurationsdatei.

Steuerung

Abhängig von der gewählten Steuerungsmethode (spurgebunden oder freie Bewegung) kann die Bewegung mit unterschiedlichen Tasten gesteuert werden. Die Tasten-

kombinationen werden in der Hilfe von SILABPreview (Menü "?" → *Tastenbelegung*) erklärt.

4 VERWALTUNG VON SZENARIENPAKETEN MIT SILAB-SESSIONEDIT2

4.1 Überblick

Mit dem Programm SILABSessionEdit2 können Sie

- mitgelieferte Szenarienpakete verwalten
- eigene Szenarienpakete erstellen
- aus den Szenarien eines Szenarienpakets einen Parcours für die Simulation erstellen
- zusätzliche Optionen, die von Szenarienpaketen unterstützt werden (wie z.B. Replay-Aufzeichnung, Wetterbedingungen und Tageszeit) festlegen.

Wie Sie aus einem bestehenden Szenarienpaket (im Lieferumfang von SILAB enthalten oder von Ihnen selbst gemäß den folgenden Abschnitten erstellt) einen Parcours zusammenstellen, ist im Dokument „Handbuch SILAB Essential“ beschrieben. Die nachfolgenden Abschnitte erläutern, wie Sie eigene Szenarienpakete erstellen und verwalten können.

Im Folgenden wird ein Szenarienpaket auch mit „Szenarien-Datenbank“ bezeichnet.

4.2 Neue Szenarien-Datenbank anlegen

Um eine neue Szenarien-Datenbank anzulegen, in die Sie eigene Szenarien einfügen können, gehen Sie wie folgt vor:

1. Wählen Sie *Datei* → *Datenbank* → *Datenbank anlegen* ...
2. Selektieren Sie die Vorlage „DatabaseTemplate“ im Dialog.
3. Wählen Sie einen Namen für Ihre Szenarien-Datenbank. Bitte beachten Sie, dass der Name mit einem Buchstaben beginnen muss, keine Leerzeichen und keine Sonderzeichen außer dem Unterstrich enthalten darf.

Nach Klick auf OK legt SILABSessionEdit die neue Szenarien-Datenbank an. Angenommen, Sie haben als Name „XYZ“ gewählt. Dann finden Sie im Ordner „SILAB\Projects“ folgende Verzeichnisstruktur:

SILAB\Projects\XYZ

lib

ACON	Speichern Sie hier die von SILABAEEdit erzeugten *.acon Dateien.
AEdit	Speichern Sie hier SILABAEEdit *.aed Dateien
Includes	Speichern Sie hier Ihre Include-Dateien
Models	Speichern Sie hier spezielle grafische Modelle
Parts	Speichern Sie hier die Konfigurationen für Ihre Szenarien wie in Abschnitt 4.4 beschrieben
Scripts	Speichern Sie hier Skripten (Ruby, JAVA)
SEO	Systemordner von SILABSessionEdit2. Bitte nicht verändern.
Sounds	Speichern Sie hier Ihre Sound-Dateien
Texture	Speichern Sie hier Texturen für Ihre grafischen

VDS	Modelle
Projects	Systemordner, bitte nicht verändern
	Speichern Sie hier Ihre *.ses und exportierten *.cfg Dateien.

4.3 Bestehende Szenarien-Datenbank sichern/rücksichern

Bevor Sie eine der in den Abschnitten 4.4 - 4.6 beschriebenen Änderungen an der Szenarien-Datenbank vornehmen, sollten Sie die Szenarien-Datenbank sichern. Die Vorgehensweise ist dabei wie folgt:

1. Wählen Sie *Datei* → *Datenbank* → *Datenbank sichern ...*
2. Es erscheint der Windows Standard-Dialog zum Speichern einer Datei.
3. Wählen Sie in diesem Dialog das Verzeichnis und den Dateinamen, unter dem Sie die von SessionEdit2 erzeugte Sicherungsdatei speichern wollen. Es wird empfohlen hierfür ein Verzeichnis außerhalb der SILAB-Ordnerstruktur zu verwenden, beispielsweise *SILAB_ARCHIV\Sicherungen*
4. Durch Anklicken von *Speichern* startet die Sicherung. Je nach Umfang der Szenarien-Datenbank kann dieser Vorgang mehrere Minuten dauern.

Bei der Sicherung erzeugt SILABSessionEdit2 ein ZIP-Archiv. Sie können dieses ZIP-Archiv folgendermaßen rücksichern:

1. Löschen Sie das Verzeichnis mit der rückzusichernden Szenarien-Datenbank aus *SILAB\Projects*.
2. Klicken Sie im Explorer von Windows doppelt auf das ZIP-Archiv mit der gesicherten Szenarien-Datenbank. Sie sehen jetzt ein einzelnes Verzeichnis, das den gleichen Namen wie die gesicherte Szenarien-Datenbank hat.
3. Wählen Sie dieses Verzeichnis im Explorer aus (einzelner Klick) und drücken Sie *Strg+C* (Kopieren).
4. Wechseln Sie in das Verzeichnis *SILAB\Projects*.
5. Drücken Sie *Strg+V* (Einfügen).
6. Windows kopiert jetzt die gesicherten Dateien. Je nach Umfang der Szenariendatenbank kann dies einige Minuten dauern.

Selbstverständlich können Sie das gesicherte ZIP-Archiv auch mit jedem ZIP-fähigen Packprogramm in das Verzeichnis *SILAB\Projects* entpacken.

4.4 Neues Szenario der bestehenden Szenarien-Datenbank hinzufügen

Um ein Szenario der Szenarien-Datenbank mit dem Namen *<Datenbankname>* hinzuzufügen zu können, müssen folgende Vorbereitungen getroffen werden:

1. Alle SILAB Konfigurationsdateien, die zur Szenario-Definition gehören, müssen im Unterverzeichnis *SILAB\Projects\<Datenbankname>\lib\parts* liegen.
2. Für jedes Szenario, das Sie einfügen möchten, gibt es genau eine Konfigurationsdatei (.cfg Datei), welche auf oberster Ebene die Moduldefinition für das Szenario enthält. Für ein Szenario namens „SP_Base_Start_S_a“ sieht diese Datei auf oberster Ebene folgendermaßen aus:

```
define Module SP_Base_Start_S_a
{
    # hier folgt die Moduldefinition
};
```

3. Wenn Sie ein Szenario ausschließlich mit SILABAEEdit entwerfen (d.h. wenn das Straßennetzwerk des Szenarios nicht aus Kombinationen von Area2- und Course-Streckenabschnitten bzw. von mehreren Area2-Abschnitten besteht), dann müssen Sie im Export-Dialog von SILABAEEdit die Einstellungen wählen, die im folgenden Bild dargestellt sind (mit dem Beispiel aus dem vorgehenden Punkt). Alle benötigten Dateien werden automatisch so erzeugt, wie sie von SILABSessionEdit2 benötigt werden. Bitte achten Sie darauf, dass als Projekt-Pfad die Datenbank gewählt ist, in die das Szenario eingefügt werden soll.

SILAB Export

SCNX SGE Test

☒ Projekt-Pfad D:\SILAB_Dev\Projects\SPDE_Base Durchsuchen

CFG-Datei: LIB\Parts\SP_Base_Start_S_a.cfg Durchsuchen

Name Area2-Typ: SP_Base_Start_S_a Node-Typ ID: 0

☒ Sichtbereich ist eingeschränkt ☒ Häuserzeilen als ODB-Objekte

☒ als Modul-Typ ☒ mit TRF

Name Modul-Typ: SP_Base_Start_S_a Modul-Typ ID: 101300001

Area2-Instanz SP_Base_Start_S_a

OBS Include: Durchsuchen

LG Include: Durchsuchen

☐ mit MOP

Anzahl Fußgänger: 10 ☐ Automatisch festlegen Überwege sperren: 1000

OK Abbrechen

4. Wenn Sie das Straßennetzwerk aus einer Mischung von Area2- und Course-Streckenabschnitten bzw. mehreren Area2-Streckenabschnitten aufbauen möchten, dann gehen Sie wie folgt vor:
- Definieren Sie die Course-Abschnitte und das Verhalten des Verkehrs innerhalb des in Punkt 2 beschriebenen Moduls (z.B. SP_Base_Start_S_a).
 - Wählen Sie für die Area2-Abschnitte folgende Einstellungen (mit angepasstem Projekt-Pfad) in SILABAEEdit (hier am Beispiel eines Abschnitts „SP_Base_TKF_L_a“):

- Den für obiges Beispiel von SILABAEEdit erzeugten Area2-Abschnitt können Sie mit dem Befehl

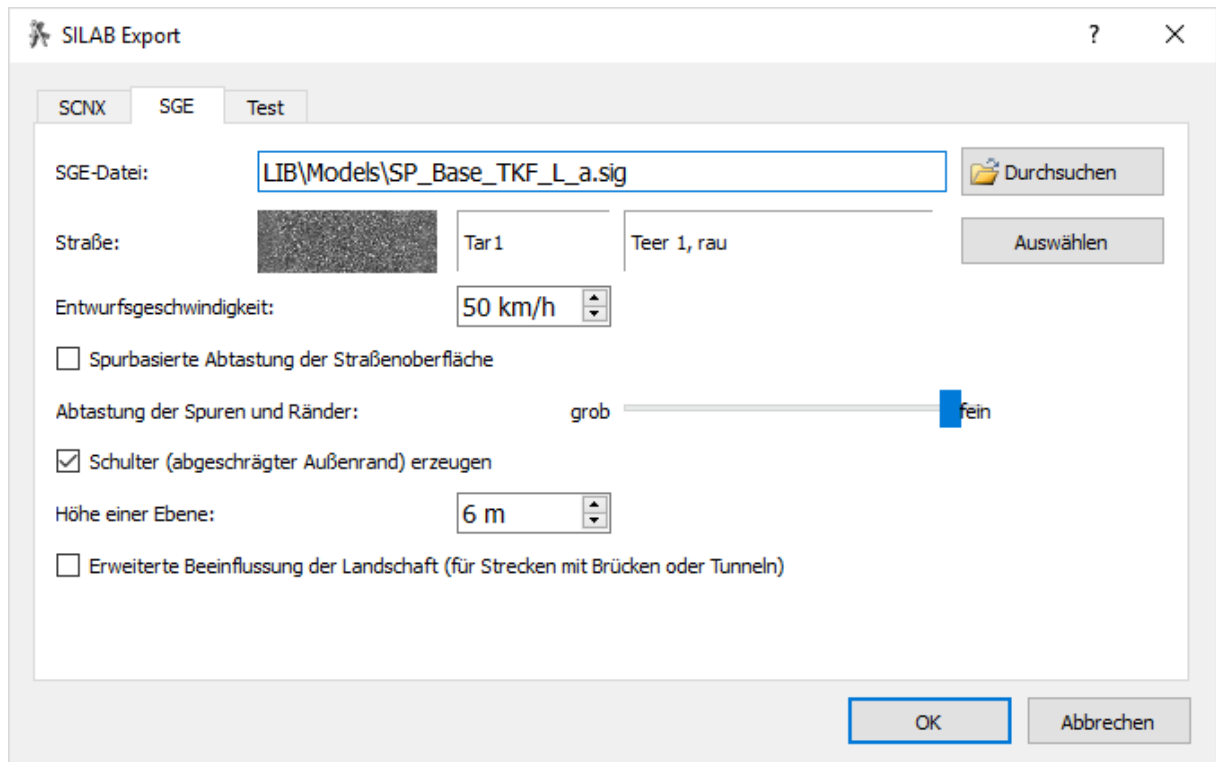
```
include lib\parts\SP_Base_TKF_L_a_Area2.inc
```

in das Szenario-Modul (z.B. SP_Base_Intersection.cfg) einbinden.

- Erstellen Sie für das obige Beispiel eine Datei mit dem Namen SP_Base_Intersection_TRFTable.cfg, in der Sie alle Fahrzeugtypen eintragen, die im Modul von Ihnen verwendet wurden (in diesem Beispiel Car und MAN_TGL):

```
UsedRoadUserTable SP_Base_Intersection
{
    Object = Car;
    Object = MAN_TGL;
    ...
};
```

5. Wenn Sie ein ganzes Szenario-Modul oder einzelne Streckenabschnitte eines Szenario-Moduls mit SILABAEEdit erstellt haben, dann wählen Sie beim Export im Reiter „SIG“ im Feld „SIG-Datei“ folgende Einstellung (wieder exemplarisch mit „SP_Base_TKF_L_a“):



6. Entwerfen Sie das Szenario-Modul so, dass es einen einzigen Port gibt, an dem der Fahrer das Szenario betritt. Er kann das Szenario durch mehrere Ports verlassen.
7. Verwenden Sie Querschnitte wie in der Dokumentation des Szenarienpakets beschrieben oder legen Sie für eigene Szenarienpakete einheitliche Querschnitte für verschiedene Straßentypen fest.

Nach diesen Vorbereitungen wird ein Szenario in SILABSessionEdit2 wie folgt eingefügt:

1. Öffnen Sie die Szenarien-Datenbank, in die das Szenario eingefügt werden soll.
2. Wählen Sie *Datei* → *Datenbank* → *Szenario hinzufügen* ...
3. Das folgende Bild zeigt den Dialog, der daraufhin erscheint. Die einzelnen Felder werden im Folgenden erläutert.

The screenshot shows the 'Szenario bearbeiten' (Edit Scenario) dialog box. It contains the following fields and options, each marked with a red number in a circle:

- 1** SCNX-Datei: SP_Base_Start_S_a.cfg
- 2** Kennung: SP_Base_Start_S_a
- 3** Name: Stadt
- 4** Kategorie:
 - ☒ Start/Ziel
 - ☐ Querschnittsübergang
 - ☐ Andere: Autobahn
- 5** Variante: a: Startpunkt in einem Innenhof [50m]
- 6** Beschreibung: Von der Startposition in einem Innenhof sind es nur ca. 50 m bis zum Modulrand. \$Der Testfahrer kann somit in der Startposition verweilen ohne die nachfolgenden Szenarien zu beeinflussen.
- 7** ☐ Aufsetzpunkt
- 8** Einfahrt:
 - Port1
 - Port2
- 9** Ausfahrt:
 - Port1
- 10** Querschnitt: Start
- 11** Spurversatz: 0
- 12** TRF-Include: SP_Base_Start_S_a_TRFTable.cfg
- 13** Screenshot: D:\SILAB_Dev\Projects\SPDE_Base\IB\Seo\SP_Base_Start_S_a.tga
- ☐ Icon:

Buttons at the bottom: OK, Abbrechen.

1 Wählen Sie hier die Konfigurationsdatei, die das Szenario-Modul des neuen Szenarios enthält.

2 Die Kennung wird automatisch anhand des Modul-Namens vergeben.

Das Szenario wird in der unter **4** angegebenen Kategorie einsortiert. Es erscheint unter dem in **3** gewählten Namen. Falls Sie in **5** eine Variante angeben, wird diese unter dem Szenario-Namen eingeordnet.

In **6** können Sie eine kurze Beschreibung für das Szenario angeben.

Wenn Sie möchten, dass Sie in jedem Parcours, die dieses Szenario enthält, direkt an diesem Szenario starten können, dann wählen Sie **7** und geben einen Namen für den Startpunkt an.

Wählen Sie unter **8** aus, an welchem Modul-Port das Szenario betreten werden soll. Über das Feld „Spurversatz“ **11** kann festgelegt werden, um wie viele Spuren das Ego-Fahrzeug bei einem Start im aktuellen Szenario nach links versetzt wird.

Unter **9** geben Sie an, über welchen Modul-Ports das Szenario verlassen werden kann. Alle in **9** gewählten Ports werden im Ablauf automatisch mit dem Einfahrts-Port des Folgeszenarios verbunden. Sollten Sie in **9** einen Port nicht anwählen und der Fahrer dennoch aus diesem Port heraus fahren, dann befindet er sich in der Landschaft auf der Wiese.

Wählen Sie in **11** die Straßenquerschnitte für Einfahrts- bzw. Ausfahrts-Ports. Verwenden Sie dabei die Querschnitte „Q2“ für Landstraßen und innerstädtische Ports und „A4“ für Autobahnen.

Unter **12** erfolgt die Angabe der TRF-Datei, welche die freigeschalteten Fahrezugtypen enthält. Diese wird entweder automatisch beim Export eines Szenarios aus SILABAEEdit heraus oder es wurde von Ihnen von Hand erzeugt (für Szenarien, die in einem Script definiert wurden).

Unter **13** können Sie ein Bild im BMP-, PNG- oder TGA-Format wählen, das von SILABSessionEdit2 für das Szenario dargestellt werden soll. Achten Sie darauf, dass dieses Bild im Format 4:3 ist.

4. Durch klicken auf „Ok“ wird das Szenario der Szenarien-Datenbank hinzugefügt.

4.5 Szenario einer bestehenden Szenarien-Datenbank bearbeiten

Um die Informationen zu einem Szenario, das bereits in die Szenarien-Datenbank eingefügt ist, bearbeiten zu können, gehen Sie wie folgt vor:

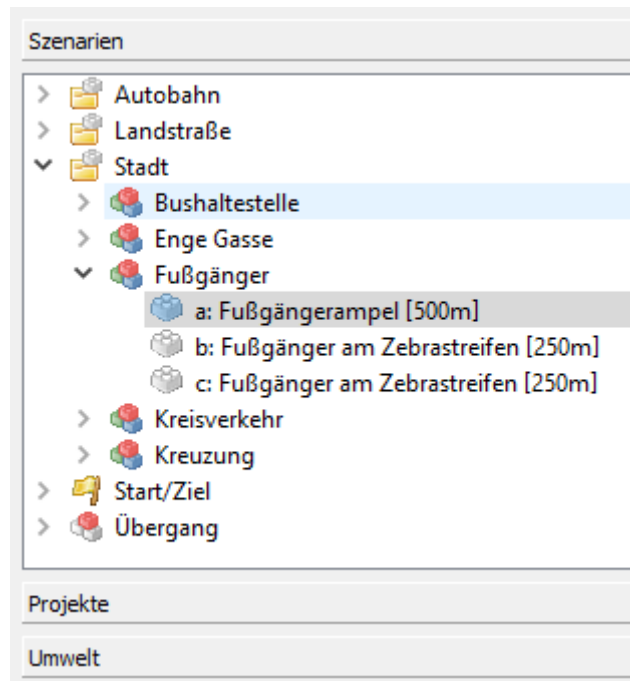
1. Öffnen Sie die Szenarien-Datenbank, aus der Sie ein Szenario bearbeiten möchten.
2. Es ist empfehlenswert, die Szenarien-Datenbank vor dem Bearbeiten eines Szenarios zu sichern (s. Abschnitt 4.3).
3. Wählen Sie in der Baumansicht das Szenario (bzw. die Variante), das Sie bearbeiten möchten, aus.
4. Wählen Sie Datei *Datei* → *Datenbank* → *Szenario bearbeiten* ...
5. Es erscheint der in Abschnitt 4.4 abgebildete Dialog. Die Bearbeitung der einzelnen Informationen erfolgt wie in diesem Abschnitt beschrieben.
6. Klicken Sie auf „Ok“, um die Informationen zu übernehmen.

Hinweise:

- Wurde die Zuordnung der Einfahrts- oder Ausfahrtsports verändert, dann müssen **alle** Parours, welche das betroffene Szenario beinhalten, neu geladen und exportiert werden.
- Wurden lediglich Beschreibungstexte, die Kategorie, der Name oder der Screenshot geändert, dann ist dieser Schritt nicht notwendig.

4.6 Szenario einer bestehenden Szenarien-Datenbank löschen

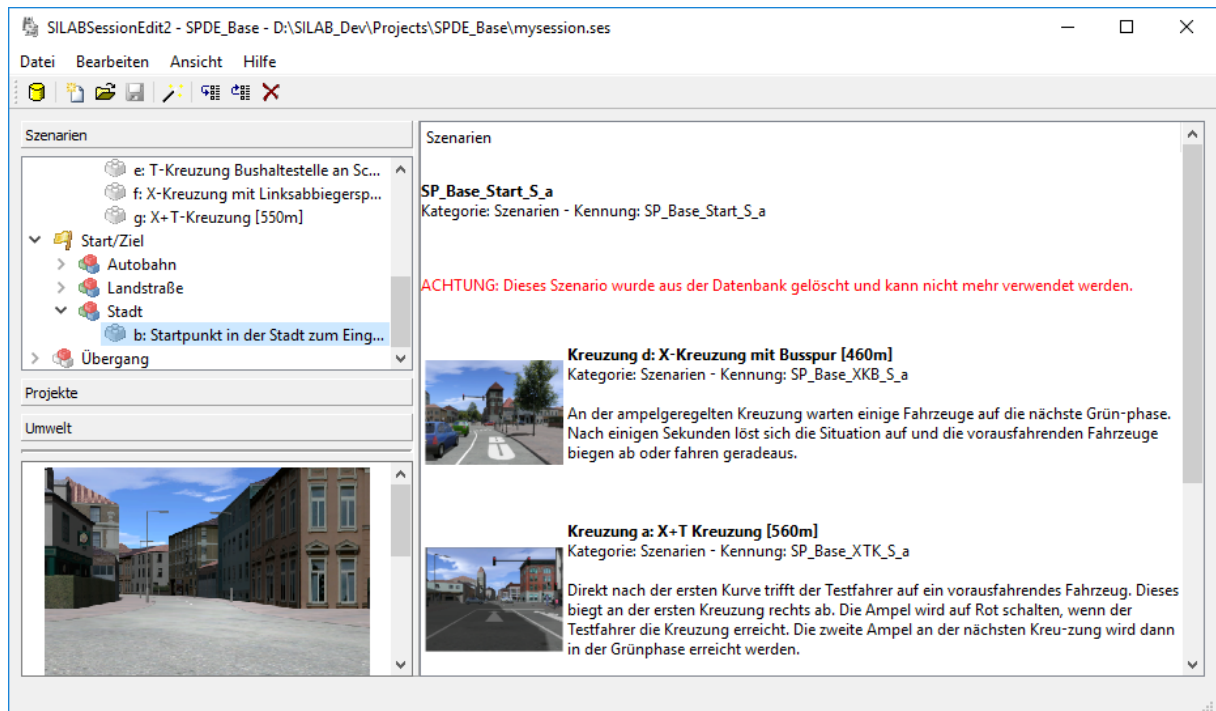
1. Öffnen Sie die Szenarien-Datenbank, aus der Sie ein Szenario löschen möchten.
2. Es ist empfehlenswert, die Szenarien-Datenbank vor dem Löschen eines Szenarios zu sichern (s. Abschnitt 4.3).
3. Selektieren Sie im Szenario-Baum das Szenario (bzw. die Variante), die Sie löschen möchten. Im folgenden Bild ist dies „a: Fußgängerampel [500m]“.



4. Wählen Sie *Datei* → *Datenbank* → *Szenario löschen* und bestätigen Sie die Rückfrage.

Hinweise:

- Das Löschen von Szenarien ist nur möglich, wenn kein Szenario-Ablauf definiert ist (z.B. unmittelbar nach dem Öffnen einer Datenbank).
- Szenarien, die Sie aus der Szenarien-Datenbank gelöscht haben, werden **nicht** automatisch aus den Parcours gelöscht, in denen sie verwendet werden. Wenn Sie einen solchen Parcours öffnen, wird das gelöschte Szenario gekennzeichnet (s. roter Text im Ablauf der folgenden Abbildung). Entfernen Sie ein gelöscht Szenario auch aus allen Parcours, in denen es vorkommt.



4.7 Eine geänderte Szenarien-Datenbank auf einem Simulator installieren

Wenn Sie die Szenarien-Datenbank geändert haben (z.B. durch Hinzufügen/Löschen von Szenarien) oder wenn Sie neue Parours für die Szenarien-Datenbank erstellt bzw. bestehende modifiziert haben, dann muss die veränderte Szenarien-Datenbank auf dem Simulator installiert werden. Gehen Sie dabei wie folgt vor:

1. Erstellen Sie eine Sicherung der Szenarien-Datenbank (s. Kapitel 4.3).
2. Erstellen Sie auf dem Bedienrechner des Simulators eine Sicherung der Szenarien-Datenbank, falls sie bereits vorhanden ist. Dies ist eine reine Sicherheitsmaßnahme. Sollte beim Installieren der neuen Version der Szenarien-Datenbank etwas schief gehen, können Sie mit dieser Sicherung den Ursprungszustand des Simulators wieder herstellen.
3. Führen Sie eine Rücksicherung der unter Punkt 1 erstellten Szenarien-Datenbank auf dem Bedienrechner des Simulators durch (s. Kapitel 4.3).
4. Starten Sie auf dem Bedienrechner des Simulators das Programm SILABAdmin. Eine Verknüpfung hierfür finden Sie i.d.R. auf dem Desktop.
5. Wählen Sie alle Computer aus, die im Hauptfenster von SILABAdmin angezeigt werden, in dem Sie den ersten der Liste anklicken und dann Strg+A drücken.
6. Wählen Sie in SILABAdmin den Menübefehl *Aktionen → Verzeichnis übertragen ...*
7. Es erscheint der Windows Standarddialog zur Verzeichnisauswahl.
8. Wählen Sie das Datenbankverzeichnis.
9. Nach Bestätigung dieser Auswahl beginnt SILABAdmin, die Szenarien-Datenbank auf den Rechnern des Simulators zu installieren. Je nach Umfang der Datenbank kann dies mehrere Minuten dauern.

5 DPU-KONFIGURATION, ANPASSUNGEN UND ERWEITERUNGEN

5.1 Konfiguration der DPUs

5.1.1 DPUSCNX

Die meisten SILAB-Module (z.B. TRFX, SCNVISX, und MOV) greifen auf die Datenbasis SCNX zu. Die Kommunikation zwischen den anderen SILAB-Modulen und SCNX erfolgt dabei nicht über DPU-Variablen (dies wäre zu ineffizient), sondern es wird direkt auf die Datenstrukturen von SCNX zugegriffen. Daher muss SCNX auf jedem Rechner des Fahrsimulators, der an Grafikausgaben, Verkehrs- oder Fußgängersimulation beteiligt ist, ausgeführt werden. Dazu wird die **DPUSCNX** in die Konfiguration eingebunden. Die DPUSCNX hat die Aufgabe, für die ordnungsgemäße Initialisierung und Aktualisierung der Datenbasis zu sorgen.

Während der Simulation erhält **DPUSCNX** die Position des Ego-Fahrzeugs in jedem Zeitschritt (üblicherweise von der Fahrdynamik-DPU). Sie sucht den Punkt im Streckennetz, der dem Ego-Fahrzeug am nächsten ist und aktualisiert das Streckennetz im Sichtbarkeitsbereich des Fahrers. Informationen über den Zustand der Straße an der aktuellen Position werden wieder an die Fahrdynamik zurückgeführt. Die Kommunikation erfolgt über DPU-Variablen:



Daher sollte darauf geachtet werden, dass in der Konfiguration die **DPUSCNX** einen höheren *Index* hat als die Fahrdynamik-DPU (z.B. **DPUmstVeda**).

Die DPUSCNX ist in der Standard-Konfigurationsarchitektur, die im Lieferumfang von SILAB enthalten ist, bereits korrekt instanziiert und verschaltet. Basis für benutzerdefinierte Fahrsimulationskonfigurationen sollte daher immer diese Konfigurationsarchitektur sein.

5.1.2 Initialisierung der DPUSCNX

Bei der Initialisierung der Simulation muss aus den definierten Karten und Aufsetzpunkten der Versuchskonfiguration jeweils einer ausgewählt werden. Dies geschieht über die Parameter *Map* und *SetupPoint* der **DPUSCNX**.

Beispiel: Fügen Sie die folgenden Zeilen (rot markiert) Ihrem vom Basis-Pool der Standard-Konfiguration abgeleiteten DPU-Pool hinzu:

```
# System-Grundeinstellungen (Standard-Launchdatas, Watchdog)
include system/SYSBase.inc

SILAB Configuration
{
    Computerconfiguration Rechner
    {
        include System/CompBase.inc
    };
    DPUConfiguration DPUs
    {
        include System/DPUBase_SGE.inc

        Pool MyPool : Full
        {
            Executable = true;

            # eigene DPU Konfiguration
            ...

            # Auswahl von Karte und Aufsetzpunkt
            SCN.Map = "MyMap";
            SCN.SetupPoint = "MySetupPoint";
        };
    };
};
```

Dabei ist `MyMap` und `MySetupPoint` der Name, den Sie für die Karte bzw. den Aufsetzpunkt gewählt haben.

Wenn mehrere Aufsetzpunkte definiert wurden und zwischen diesen häufig gewechselt werden muss, ist es empfehlenswert, die Auswahl über Launchdaten beim Start der Simulation vorzunehmen¹⁶. **DPUSCNX** verwendet dazu die Launchdata `SCN_SetupPoint`. Damit der Anwender den Aufsetzpunkt aus der Liste aller definierten Punkte auswählen kann, muss eine Auswahlbox für diese Launchdata verwendet werden und auf dem Operator-Rechner ebenfalls die Datenbasis *initialisiert* werden. Dies sollte mit der speziell zu diesem Zweck entwickelten **DPUSCNXLaunch** und nicht mit der **DPUSCNX** geschehen¹⁷.

Beispiel: Zur Konfiguration des Aufsetzpunkts über Launchdaten muss die Launchdata zunächst im SILAB-Systemabschnitt definiert werden:

```
SILAB System
{
    ...
    LaunchData1 = ("Aufsetzpunkt auswählen", SCN_SetupPoint, LD_COMBO_DEFAULT, "SetupPoint1");
};
```

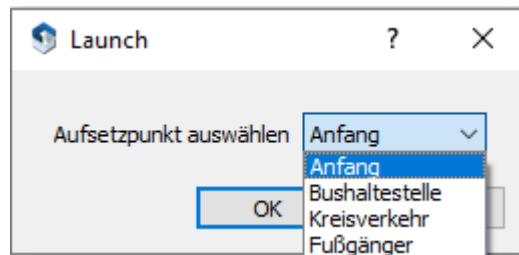
Die **DPUSCNXLaunch** muss zusätzlich auf dem Operator gestartet werden:

¹⁶ Näheres zu Launchdaten finden Sie in „SILAB Konfiguration und Bedienung“.

¹⁷ Da der Operator-Rechner in vielen Fällen nicht an Grafik- oder Verkehrssimulation beteiligt ist, wird die benötigte Rechenzeit reduziert, indem die Datenbasis während der Simulation nicht *aktualisiert* wird.

```
DPUSCNXLaunch Launch
{
    Computer = {OPERATOR};
    Index = 100;
};
```

Beim Start der Simulation erscheint folgendes Dialogfenster:



5.1.3 Bedingte Verknüpfungen

In der dynamischen Datenbasis von SILAB können Streckenteile abhängig vom Verhalten des Testfahrers oder von Eingaben des Operators neu verknüpft werden. Dazu müssen bedingte Streckenverknüpfungen in die Konfiguration aufgenommen werden (siehe Kap. 2.4.2.2).

Für eine bedingte Verknüpfung werden jeweils logische Bedingungen an die Bedingungsvariablen *CondVar1* bis *CondVar10* gestellt. Dabei handelt es sich um Eingänge der **DPUSCNX**, die beliebig mit anderen Variablen der Simulation verknüpft werden können (z.B. der aktuellen Geschwindigkeit des Ego-Fahrzeugs).

5.1.4 Ausgänge der DPUSCNX

Die **DPUSCNX** bietet zahlreiche Ausgänge, auf denen verschiedene Informationen über Zustand und Position des Ego-Fahrzeugs ausgegeben werden. Diese umfassen unter anderem den aktuellen Streckenmeter in der Simulation, die derzeitige Krümmung der Straße und die aktuelle TLC (Time to Lane Crossing, Zeit bis zum Überqueren des Fahrbahnrandes). Je nach Fragestellung werden meist einige dieser Ausgänge zur Auswertung benötigt und dazu mit der **DPUDataFile** in eine Textdatei aufgezeichnet. Eine vollständige Liste der verfügbaren Ausgänge finden Sie in der Dokumentation „DPUSCNX“.

5.1.5 Visualisierung

Auf den Grafik-Rechnern des Simulators wird zusätzlich eine Visualisierung der Simulation durchgeführt. Dazu werden folgende DPU's verwendet: **DPUSCNXSGE** (Landschaft und Straße), **DPUTRFSGE** (Verkehr) und **DPUMOPSGE** (Fußgänger). Zusätzlich werden grundlegende DPU's für die Grafikenengine SGE benötigt (**DPUSGEWorld** und **DPUSGEView**).

Die Konfiguration der Visualisierung SGE wird detailliert in der Beschreibung zu „Referenz Visualisierung SGE“ erläutert.

5.2 Konfiguration von SCNX („SCNX Params“)

Die Datenbasis SCNX kann an Ihre Anforderungen angepasst und um benutzerdefinierte Elemente (z.B. Straßenbeläge) erweitert werden. Bei der Installation von

SILAB wurde bereits eine Standard-Konfiguration eingerichtet, die an Ihr Simulator-System angepasst ist. Daher muss bei den meisten Simulationsfahrten lediglich die Standard-Konfiguration eingebunden werden, die im Lieferumfang von SILAB enthalten ist. Dazu muss in der Versuchskonfiguration (Haupt-CFG-Datei) innerhalb der Konfiguration der Datenbasis folgende (fett markierte) Zeile hinzugefügt werden:

```
SILAB SCN {
    include scnx\SCNX_600.cfg
    ...
};
```

Hiermit wird die Konfigurationsdatei `\SILAB\lib\configurations\SCNX\SCNX_600.cfg` eingebunden. Für besondere Anforderungen können Sie die Einstellungen dieser Datei ändern bzw. erweitern. Sie sollten dabei nicht die Standard-Konfigurationsdatei ändern, sondern unterhalb der Include-Zeile von `SCNX_600.cfg` den entsprechenden Konfigurationsabschnitt erneut aufführen. Die neu angegebenen Werte überschreiben dabei die Werte der Parameter aus der `SCNX_600.cfg`.

Die Konfiguration ist wie folgt aufgebaut:

```
SCN Params {
    SCN ToDo {
        # Konfiguration der SCNX Update-Verwaltung
    };
    SCN ObjectManager {
        # Konfiguration der Landschafts-Objektverwaltung
    };
    SCN CourseLaneCell {
        # Konfiguration der Streckenabschnitts-Verwaltung
    };
    SCN CrossSection {
        # Definition der Straßenbeläge (Kap. 5.2.1)
        # Definition von Overlay-Texturen (Kap. 5.2.2)
        # Definition der Bebauungstypen (Kap. 5.2.4)
    };
    SCN H {
        # Konfiguration des Höhenmodells
    };
    SCN LandscapeGeneration {
        # Konfiguration der Landschaftsgenerierung (Kap. 5.2.5)
    };
    SCN SignGeneration {
        # Konfiguration der Schildergenerierung (Kap. 5.2.6)
    };
    SCN TRFRules {
        # Konfiguration der Verkehrsregeln (darf nicht verändert wer
        # den)
    };
};
```

5.2.1 Definition von Straßenbelägen für Courses

Ein Straßenbelag wird folgendermaßen angegeben:


```
SCN Pavings {
    # Definitionen der Straßenbeläge
};
```

Grammatik 25 Definition Straßenbeläge

```
<Straßenbeläge> →
    (1...*)<Straßenbelag>
```

Die einzelnen Straßenbeläge werden mittels einer Textur spezifiziert:

Grammatik 26 Definition Straßenbelag

```
<Straßenbelag> →
    <Belagsname> = {<Texturname>[, <OverlayAktiv>]?};
```

Eine Spurdefinition verweist somit direkt auf die Textur, die für die graphische Gestaltung der Straße herangezogen wird. Die Textur sollte in x- wie in y-Richtung kachelbar sein.

Wenn für den Parameter *<OverlayAktiv>* der Wert **UseOverlay** angegeben wird, werden auf diese Straßentextur zusätzlich Overlay-Texturen (siehe Kap. 5.2.2) aufgebracht, falls dies im Querschnittsprofil aktiviert wurde (siehe Kap. 2.5.5).

Durch Height und Friction Maps (siehe Kap. 5.2.3) kann die Oberfläche von Straßenbelägen zusätzlich beeinflusst werden.

5.2.2 Definition von Overlay-Texturen

Overlay-Texturen werden folgendermaßen definiert:

```
SCN Overlays {
    # Definitionen der Overlay-Texturen
};
```

Grammatik 27 Definition Overlay-Texturen

```
<Overlay-Texturen> →
    (1...*)<Overlay-Textur>
```

Für jede einzelne Overlay-Textur sind folgende Angaben erforderlich:

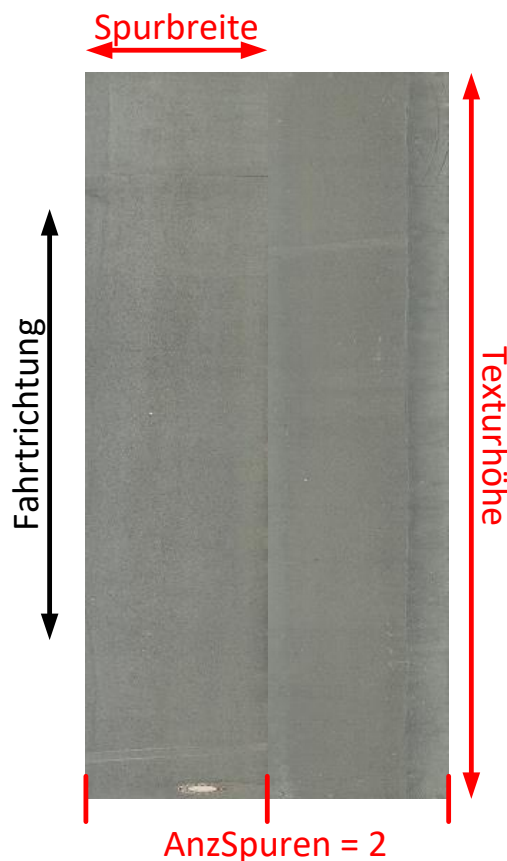
Grammatik 28 Definition Overlay-Textur

$\langle \text{Overlay-Textur} \rangle \rightarrow$
 $\langle \text{Overlay-Name} \rangle = \{ \langle \text{Texturname} \rangle, \langle \text{AnzSpuren} \rangle, \langle \text{Spurbreite} \rangle, \langle \text{Texturhöhe} \rangle \};$

Die Angaben haben folgende Wirkung:

$\langle \text{Texturname} \rangle$	Pfad zur Texturdatei, in SGE-Ressourcen-Notation. Die Texturdatei wird unterhalb des Ordners SILAB\lib\graphics\default\texture\SCNX bzw. SILAB\Projects\MeinProjekt\SCNX gesucht.
$\langle \text{AnzSpuren} \rangle$	In der Textur muss die Fahrtrichtung „senkrecht“ sein. Anzahl von Spuren, die in der Textur nebeneinander liegen.
$\langle \text{Spurbreite} \rangle$	Breite einer Spur in der Textur in Metern.
$\langle \text{Texturhöhe} \rangle$	Höhe der Textur in Metern, d.h. die Länge der abgebildeten Straßenstrecke.

Die Parameter sind in der folgenden Abbildung veranschaulicht.



Durch Height und Friction Maps (siehe Kap. 5.2.3) kann die Oberfläche von Overlay-Texturen zusätzlich beeinflusst werden.

5.2.3 Height und Friction Maps

Durch zusätzliche **Height Maps** kann eine unregelmäßige, raue oder rissige Straßenoberfläche simuliert werden. Durch zusätzliche **Friction Maps** kann der Reibbeiwert der Straße beeinflusst werden, um beispielsweise den Einfluss kleiner Blätter oder Wasserpfützen zu simulieren.

Height und Friction Maps können auf Straßenbeläge (Kap. 5.2.1), auf Overlay-Texturen (Kap. 5.2.2), Strip-Bebauungen (Kap. 5.2.4.2) und auf Einzelobjekte (Kap. 2.5.6) angewendet werden.

Um die Verwendung von Friction Maps und Height Maps zu aktivieren, muss der Eingang *EnableRefinement* der *DPUSCNX* auf 1 gesetzt werden.

Height und Friction Maps sind **Bilder** im Format PNG, wobei nur der Graustufenkanal berücksichtigt wird. Diese Graustufeninformationen werden wie folgt verwendet:

Für **Height Maps** wird der Graustufen-Wert wie folgt skaliert, um die Höhe der Fahrbahnoberfläche bzw. des Objekts zu beeinflussen:

Graustufenwert	Höhenänderung
0	-0.1 m
128	0 m
255	0.1 m

Anschließend wird die Höhenänderung auf den Ausgang *ZOut* der *DPUSCNX* addiert.

Für **Friction Maps** wird der Graustufen-Wert auf den Wertebereich 0...1 skaliert:

Graustufenwert	Reibbeiwert-Multiplikator
0	0
128	0.5
255	1

Anschließend wird dieser Faktor auf den vorhandenen Reibbeiwert der Fahrbahnoberfläche (Ausgang *mu* der *DPUSCNX*) multipliziert.

Weitergehende Konfigurationseinstellungen zu Height und Friction Maps sind nicht notwendig, da SILAB die Dateien automatisch in folgenden **Verzeichnissen** sucht:

<Basispfad>\<Typspezifisch>\<Anwendungsspezifisch>\<Elementname>.png

Dabei sind die einzelnen Pfadkomponenten wie folgt definiert:

System-<i><Basispfad></i>	<i>\SILAB\lib\configurations\scnx</i>
Projektspezifischer <i><Basispfad></i>	<i>\SILAB\Projects\MeinProjekt</i>
<i><Typspezifisch></i>	<i>\heightmap\ bzw. \frictionmap\</i>
<i><Anwendungsspezifisch></i>	<i>\pavings\, \objects\ oder \strips\</i>
<i><Elementname></i>	<i><Belagsname>, <Overlay-Name>, <Bebauungsname> oder (für Einzelobjekte) der Dateiname der Objektdatei, ohne Verzeichnis und Dateierweiterung</i>

Beispiele:

Eine Height Map zum mitgelieferten Straßenbelag *Tar1* befindet sich in der Datei

\SILAB\lib\configurations\scnx\heightmap\pavings\tar1.png

Eine Friction Map zu dem selbst erstellten Objekt

\SILAB\Projects\MeinProjekt\models\ductcover.sig

können Sie unter

\SILAB\Projects\MeinProjekt\frictionmap\objects\ductcover.png
ablegen.

5.2.4 Definition von Bebauungstypen

Es gibt drei grundsätzlich unterschiedliche Typen von Bebauungen. Durch folgenden Abschnitt werden die Bebauungstypen festgelegt:

```
SCN Superstructures {
    # Definition der Bebauungstypen
};
```

Grammatik 29 Bebauungstypen

```
<Bebauungstypen> →
    (1..*)<Bebauungstyp >
```

Grammatik 30 Bebauungstyp

```
<Bebauungstyp> →
    <Texture-Definition> |
    <Strip-Definition> |
    <Object-Definition> |
    <Bebauungsname> = {<Objekttyp-Texture> | <Objekttyp-Strip> |
                        <Objekttyp-Object>};
```

Seit SILAB 3.0 wird zur Definition der Bebauungstypen ein neues Format verwendet. Dies sind die *<Texture-Definition>*, *<Strip-Definition>* und *<Object-Definition>*.

Das bisherige Format *<Bebauungsname> = {...};* wird aus Kompatibilitätsgründen weiterhin unterstützt, sollte aber nicht mehr verwendet werden.

5.2.4.1 Bebauungstyp Texture

Der erste Bebauungstyp *Texture* sind Texturen, die auf die Straße gezeichnet werden sollen bzw. ein Teil des Straßenquerschnitts sind, z.B. Fahrbahnmarkierungen.

Grammatik 31 Bebauungstyp Texture

<Texture-Definition> →

```
Texture <Bebauungsname> {  
    TextureName = <Texturname>;  
};
```

<Objektyp-Texture> →

```
Texture, <Texturname>
```

Als **TextureName** muss ein Texturdateiname in SGE-kompatibler Notation verwendet werden. Die Textur wird unterhalb des *AdditionalPath* gesucht, der im Abschnitt *SCNX CourseLaneCell* definiert wurde.

Beispielsweise bezeichnet `road.superstructures.strips.hedge2` in der Standardkonfiguration die Datei `\SILAB\lib\graphics\default\texture\SCNX\road\superstructures\strips\hedge2.tga`.

5.2.4.2 Bebauungstyp Strip

Der zweite Typ *Strip* sind Bänder, die parallel zur Straße verlaufen, wie Hecken, Schutzplanken oder Zäune. Bei diesem Typ ist zu beachten, dass die Bänder immer nur auf der Straße platziert werden können und nicht in der umgebenden Landschaft.

Grammatik 32 Bebauungstyp Strip

<Strip-Definition> →

```
Strip <Bebauungsname> {  
    (0..1) Smooth = true | false;  
    (1..*) <Strip-Element>  
};
```

<Strip-Element> →

```

Element <Elementname> {
  (0..1)TextureName = <Texturname>;
  (0..1)NormalMap = <Texturname>;
  (0..1)TemplateName = <Objektname>;
TextureWidth = <Textur-Länge>;
Texturey1 = <Textur-T1>;
Texturey2 = <Textur-T2>;
  (0..1)FixTextureSize = true | false;

  Vy1 = <y1>;
  Vz1 = <z1>;
  Vy2 = <y2>;
  Vz2 = <z2>;

  (0..1)Ey1 = <ey1>;
  (0..1)Ez1 = <ez1>;
  (0..1)Ey2 = <ey2>;
  (0..1)Ez2 = <ez2>;
  (0..1)AlphaEntry = <ea>;
};

```

<Objekttyp-Strip>→

Strip, <Texturname>, <Textur-T1>, <Textur-T2>, <y1>, <z1>, <y2>, <z2>,
<Textur-Länge>

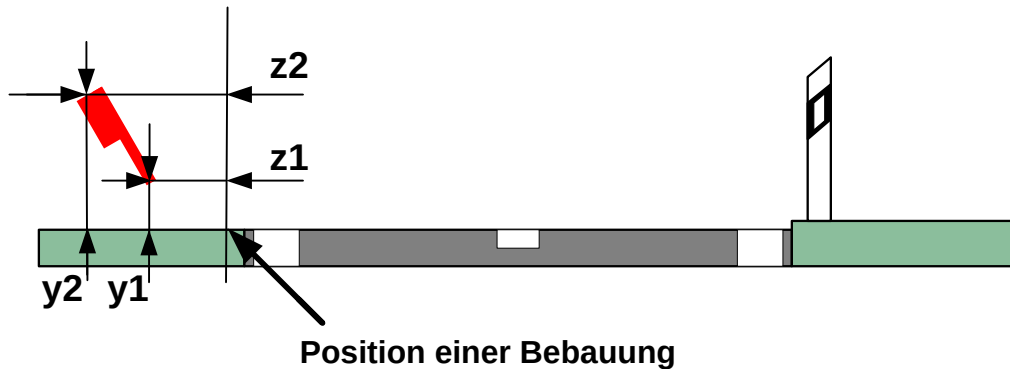
Ein Strip-Bebauungstyp kann mehrere Bänder beinhalten. Jedes Band verläuft innerhalb einer Textur horizontal von links nach rechts. Als **TextureName** muss ein Texturdateiname in SGE-kompatibler Notation verwendet werden. Die Textur wird unterhalb des *AdditionalPath* gesucht, der im Abschnitt *SCNX CourseLaneCell* definiert wurde. Die untere und obere Grenze wird durch die Parameter <Textur-T1> und <Textur-T2> angegeben, die - wie bei Texturen üblich - in einem Wertebereich von [0, 1] liegen (0 = ganz unten, 1= ganz oben).

Zusätzlich zur Textur kann optional eine **NormalMap** zur Darstellung von Straßenunebenheiten angegeben werden. Wie unter *TextureName* beschrieben, wird diese unterhalb des *AdditionalPath* gesucht.

Alternativ zur Angabe eines *TextureName* kann auch ein **TemplateName** angegeben werden. Dies ist eine SGE-Objektdatdatei, die unterhalb des *AdditionalPath* für SGE-Modelle gesucht wird. Sie darf keine Subobjekte, Vertex- oder Primitivendefinitionen enthalten. Die weiteren darin angegebenen Objektattribute, z.B. Farben, Animation und Shader, werden für das Bebauungsobjekt übernommen.

Jedes einzelne Band wird parallel zur Straße gezeichnet. Seine Position wird durch die zwei Vektoren <y1, z1> und <y2, z2> festgelegt. Zur jeweiligen Position der Bebauung definiert die y-Richtung die Abweichung nach links und rechts und die z-Richtung die Höhe des Bandes. Zu jedem Punkt der Straße können somit die zwei Randpunkte des Bandes ermittelt werden.

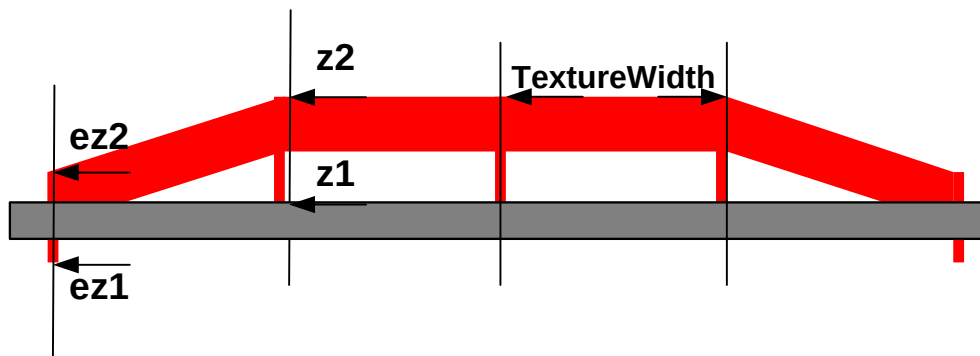
Die Positionsparameter sind am Beispiel einer schräg angebrachten Schutzplanke in der folgenden Abbildung dargestellt:



Eine Bebauung, die als zusätzliches Objektband eines Course-Streckentyps platziert wurde, kann „sanft beginnen“ und/oder „sanft enden“ (Parameter *Entry* und *Exit*, siehe Kap. 2.5.10). Über die optionalen Parameter $\langle ey1, ez1 \rangle$ und $\langle ey2, ez2 \rangle$ wird die Position des Bandes für einen Entry- oder Exit-Punkt festgelegt (siehe Bild unten).

Beim Guardrail ist z.B. $z1=0$, $z2=0.7$ definiert. Durch die Definition von $ez1=-0.5$ und $ez2=0.2$ wird ein diagonaler Abschluss des Guardrails bis fast hinunter auf Bodenhöhe definiert, wie in folgender Abbildung veranschaulicht ist.

Wird einer der ey- oder ez-Parameter nicht angegeben, übernimmt SILAB den Wert des zugehörigen y- bzw. z-Parameters, so dass ein harter Abschluss entsteht.



Alternativ zu einem diagonalen Abschluss kann ein Objektband auch über einen Transparenzwert am Anfang und Ende ausgeblendet werden, dazu muss der Parameter *AlphaEntry*=0 gesetzt werden. Standardwert ist *AlphaEntry*=1, d.h. es wird keine Transparenz verwendet.

Die $\langle \text{Textur-Länge} \rangle$ (siehe Bild oben) gibt an, wie oft sich die Textur entlang des Straßenverlaufs wiederholen soll, d.h. wie groß das Objekt in Realität ist, welches durch das Band dargestellt wird.

Der Parameter *FixTextureSize* beeinflusst das Verhalten, wenn das Objektband skaliert wird (durch Setzen von *Width* $\neq$ 1 in der *CrossSection*-Definition). Die Stan-

dardeinstellung *FixTextureSize=false* skaliert die Textur mit, so dass das Aussehen des Objekts erhalten bleibt. Bei der Einstellung *FixTextureSize=true* wird die Textur entsprechend der Objektgröße häufiger angebracht, d.h. die Größe einer im Texturbild abgebildeten Struktur bleibt gleich. Die Textur muss dazu in beide Richtungen nahtlos wiederholbar sein.

Falls in einem Strip mehrere aneinander anschließende Bänder definiert werden, kann optional der Parameter *Smooth=true* gesetzt werden. Dadurch werden die Normalenvektoren der angrenzenden Punkte so abgeändert, dass der Eindruck einer runden Oberfläche entsteht. Bei der Standardeinstellung *Smooth=false* wird keine Anpassung durchgeführt.

Durch Height und Friction Maps (siehe Kap. 5.2.3) kann die Oberfläche von Strip-Bebauungen zusätzlich beeinflusst werden.

5.2.4.3 Bebauungstyp Object

Der dritte Typ „Object“ beschreibt Objekte, die in regelmäßigen Abständen entlang der Straße platziert werden. Im Gegensatz zum „Strip“-Typ, sind diese keine „durchgehenden“ Bänder, sondern einzelne wiederkehrende Objekte wie z.B. Leitpfosten.

Grammatik 33 Bebauungstyp Object

<Object-Definition>→

```
Object <Bebauungsname> {
    ObjectName = <Objektname>;
    MaxNumber = <Maximale Anzahl>;
    MaxDist = <Maximaler Abstand>;
};
```

<Objekttyp-Object>→

```
Object, <Objektname>, <Maximale Anzahl>, <Maximaler Abstand>
```

Als **ObjectName** muss ein Objektdatenname in SGE-kompatibler Notation verwendet werden. Das Objekt wird unterhalb des *AdditionalPath* gesucht, der im Abschnitt *SCNX CourseLaneCell* definiert wurde.

Beispielsweise bezeichnet `road.superstructures.objects.post` in der Standardkonfiguration die Datei `\SILAB\lib\graphics\default\models\SCNX\road\superstructures\objects\post.sge`.

Ein Object-Bebauungstyp wird aus Effizienzgründen nur in einem gewissen Radius um den Fahrer herum gezeichnet, der durch den maximalen Abstand definiert wird. Zusätzlich ist die maximale Anzahl der gleichzeitig dargestellten Objekte durch den entsprechenden Parameter begrenzt.

5.2.5 Konfiguration der Landschaftsgenerierung

Die Landschaftsgenerierung erzeugt Objekte entlang von Course-Streckentypen, die meist in größerem Abstand zur Straße unregelmäßig angeordnet sind, insbesondere

Bäume und einzeln stehende Häuser. Außerdem kann das Aussehen des Bodens beeinflusst werden.

Grammatik 34 Parameter der Landschaftsgenerierung

```

SCN LandscapeGeneration {
  (0..1)GrassTexture = <Texturname>;
  (0..1)RockTexture = <Texturname>;
  (0..1)OverlayTexture = <Texturname>;
  (0..1)OverlayWidth = <Zahl>;
  (0..1)OverlayHeight = <Zahl>;
  (0..1)MaxSupply = <Zahl>;
  (0..*)<ObjectGroup>
  (0..*)<LandscapeGenerator>
};

```

5.2.5.1 Aussehen der Landschaftsoberfläche

Durch die Parameter **GrassTexture**, **RockTexture** und **OverlayTexture** kann das Aussehen der Landschaftsoberfläche bestimmt werden. Es muss jeweils ein Texturname in SGE-Ressourcen-Notation¹⁸ angegeben werden. Die Gras-Textur wird auf allen flachen Landschaftsflächen und die Felstextur auf allen steilen Abhängen aufgebracht. Über diese beiden Texturen werden entsprechend des Landschaftstyps, wenn der Parameter *OverlayDensity* des Landschaftsgenerators des Course-Typs nicht 0 ist, Overlay-Elemente (Felder, Waldboden) aufgebracht. Diese werden aus der Overlay-Textur entnommen, die in **OverlayWidth** Spalten und **OverlayHeight** Zeilen von einzelnen Overlay-Elementen unterteilt ist. Die einzelnen Elemente der Overlay-Textur sind von links oben nach rechts unten nummeriert¹⁹. Das Feld in der linken oberen Ecke hat den Index 0 und sollte immer vollständig transparent sein, da hiermit alle Bereiche gefüllt werden, in denen keine Overlay-Elemente platziert werden.

5.2.5.2 Objekterzeugung

Die Objekterzeugung platziert Objekte entsprechend dem im Course-Typ angegebenen Landschaftstyp. In der Standard-SCNX-Konfiguration sind bereits viele Landschaftstypen vordefiniert (siehe Kap. 2.5.13). Sie können aber jederzeit andere Typen als <LandscapeGenerator> hinzufügen.

¹⁸ siehe dazu Kapitel „Verzeichnisstruktur“ im Dokument „Referenz Visualisierung SGE“

¹⁹ Dies ist für die Parameter *MinOverlay* und *MaxOverlay* wichtig, siehe Grammatik 35.

Grammatik 35 Landschaftsgeneratoren

```

<LandscapeGenerator> →
    LandscapeGenerator <Name> {
        (0..1)Type = <Algorithmus>;
        ObjectGroups = { (1..*) <ObjectGroup>_(1) };
        MinOverlay = <Zahl>;
        MaxOverlay = <Zahl>;
    };

<ObjectGroup> →
    ObjectGroup <Name> {
        Objects = { (1..*)<SIG-Objektnamen>_(1) };
        (0..1)MaxSupply = <Zahl>;
        Override = { (0..*)<Objektgruppenname>_(1) };
        (0..1)Rotate = true;
        (0..1)Incline = true;
        (0..1)ColorizeSet = <Name>;
    };

```

Jeder <LandscapeGenerator> benutzt alle SGE-Objekte, die in seinen <Object-Groups> angegeben sind, und verteilt diese innerhalb des ausgewählten Bereichs. Die Platzierung der Objekte erfolgt abhängig vom ausgewählten Algorithmus (Parameter **Type**). Folgende Algorithmen sind verfügbar:

Type	Beschreibung
Default	Verteilt die Objekte zufällig. Wird auch verwendet, wenn der Type-Parameter weggelassen wurde.
Regular	Verteilt die Objekte in einem regelmäßigen Raster
Fill	Füllt die Fläche möglichst lückenlos mit den Objekten
LongLine	Erstellt eine einzelne Linie von zufällig ausgewählten Objekten entlang der Straße
LongLineSequence	Erstellt eine einzelne Linie von Objekten entlang der Straße, mit regelmäßig eingestreuten Objekten
Field	Erzeugt Felder und erschließt sie mit Feldwegen
Forest	Erzeugt Wälder mit Unterholz und bodendeckenden Pflanzen
Meadow	Erzeugt Weiden mit Zäunen und Feldwegen und verteilt zufällig weitere Objekte darauf.
MeadowLOD	Erzeugt Weiden mit Zäunen und Feldwegen und verteilt zufällig weitere Objekte darauf. Die Objekte sind in zwei Versionen (komplexe Objekte in der Nähe der Straße und einfachere weiter entfernte Objekte) vorhanden.

Bei den meisten Algorithmen wird die Anzahl der von der Objektgruppe *Group* zu verteilenden Objekte entsprechend des Parameters <Group>*Density* des Course-Typs festgelegt.

Entsprechend der *OverlayDensity* des Course-Typs werden Overlay-Elemente platziert. Dabei werden nur die Overlay-Elemente mit Index **MinOverlay** bis **MaxOverlay** verwendet.

Zur Definition einer *<ObjectGroup>* muss angegeben werden, welche SIG-Objekte in dieser enthalten sind. Diese werden in der Liste **Objects** in der SIG-Ressourcen-Notation angegeben. Mit dem **Override**-Parameter kann festgelegt werden, innerhalb welcher anderer *ObjectGroups* die Objekte dieser *ObjectGroup* platziert werden dürfen. Die Angabe `Override = { Field };` in der *ObjectGroup Tree* bewirkt zum Beispiel, dass auch innerhalb von Feldern Bäume stehen können. Wenn Objekte nicht in *Override* aufgeführt sind, werden sie hingegen stets so platziert, dass sie sich nicht überschneiden. Als Sonderfall kann `Override = { Road };` angegeben werden, wenn Objekte auf der Straße platziert werden sollen.

Falls **Rotate = true;** angegeben wird, werden die Objekte zufällig verdreht. Andernfalls wenden sie der Straße immer ihre Vorderseite zu.

Falls **Incline = true;** angegeben wird, wird die Neigung jedes Objekts an die Landschaftsneigung an dessen Position angepasst. Andernfalls werden die Objekte immer senkrecht stehend platziert.

Falls ein **ColorizeSet** angegeben wird, werden die Objekte mit einem zufällig daraus ausgewählten Farbsatz eingefärbt. Das **ColorizeSet House** ist bereits vordefiniert und enthält Farben für Dächer, Fenster und Wände von Häusern.

In der gesamten sichtbaren Szene von jedem SIG-Objektyp jeweils maximal **MaxSupply** Objektinstanzen platziert (dies ist unabhängig davon, in wie vielen *ObjectGroups* der SIG-Objektyp auftaucht). Dieser Wert kann für jede *ObjectGroup* einzeln festgelegt werden. Wenn er dort nicht angegeben wird, wird der globale Defaultwert aus dem *SCN LandscapeGeneration*-Abschnitt verwendet. Wenn aufgrund von hohen *<Group>Density*-Parametern mehr Objekte eines Objektyps angefordert werden, als vorhanden sind, so werden diese zusätzlichen Objekte nicht platziert und auch nicht dargestellt.

Mit der *MaxSupply*-Einstellung kann sichergestellt werden, dass die Simulation stets flüssig läuft, denn aktuelle Grafikkarten können nur eine begrenzte Anzahl unabhängiger Objekte darstellen (ca. 2000-4000). Deswegen sollte darauf geachtet werden, dass die Summe der *MaxSupply*-Werte für alle Objektypen nicht deutlich über 2000 liegt. Eine Ausnahme hierbei sind Bäume und ähnliche Billboard-Objekte, die mit der *multibillboard*-Animation dargestellt werden, da diese besonders effizient verwaltet und dargestellt werden können: Von *multibillboard*-Objekten können etwa fünfmal so viele Instanzen bei derselben Darstellungsgeschwindigkeit gezeichnet werden.

5.2.6 Konfiguration der Schildergenerierung

Im Abschnitt *SignGeneration* werden Parameter für die Schildergenerierung festgelegt.

Grammatik 36 Schildergenerierung

```

<Konfiguration Schildergenerierung> →
  SCN SignGeneration
  {
    <allgemeine Parameter, s.u.>
    (1..*)<Schildformat>
    (1..*)<Schildposition>
  };

```

Folgende allgemeine Parameter legen grundsätzliche Eigenschaften der generierten Schilder fest:

Size = { *size0*, *ve1*, *size1*, *ve2*, *size2*, ... };

Definition der Basisgröße der Schilder abhängig von der Entwurfsgeschwindigkeit *ve* des Courses bzw. der Area:

Falls *ve* < *ve1*, wird *size0* verwendet;

Falls *ve1* ≤ *ve* < *ve2*, wird *size1* verwendet; usw.

Als Einheit für *size* wird cm, für *ve* km/h verwendet.

PostSize = *size*;

Definition des Durchmessers eines Schildpfostens in m.

ODBCategory = { *cat1*, *from1*, *to1*, *cat2*, *from2*, *to2*, ... };

Zuweisung einer ODB-Objektkategorie *cat* für die Schilder. *from* und *to* sind jeweils Schildtypen und definieren (je- weils inklusive) einen Bereich, für den diese Kategorie gilt.

CancellInfo = { "begin1", "end1", "begin2", "end2", ... };

Definiert Paare von Verbots- (*begin*) und Verbots-Ende- (*end*) Schildern. Für *begin* und *end* können jeweils einfache Schildtypen (z.B. "242") verwendet werden. Alternativ kann auch ein Schildtyp inklusive Variante angegeben werden (z.B. "274_01").

Für jedes Schild müssen dessen Ausmaße festgelegt werden. Dazu werden die unterschiedlichen Schildformate spezifiziert:

Grammatik 37 Schildformat

```

<SignFormat> →
  Format <Name> {
    (0..1)SignMinZ = <Höhe in m>;
    SignCenterZ = <Höhe in m>;
    (0..1)SignMaxZ = <Höhe in m>;
    (0..1)SignMinZN = <Höhe in m>;
    (0..1)SignMinZOff = <Höhe in m>;
    (0..1)SignMaxZN = <Höhe in m>;
    (0..1)SignMaxZOff = <Höhe in m>;
    Width = <Multiplikator>;
    Height = <Multiplikator>;
    PostYRel = { (,) <rel. Pfostenposition> };
    PostMinZ = <Höhe in m>;
    PostMaxZRel = <rel. Höhe>;
    ( Signs = { (,) <Typ> }; | IsDefault = true; )
    (0..1)Backsides = { (,) <Typ> };
  };

```

Die einzelnen Parameter eines Schildformats haben folgende Bedeutung:

<i>SignCenterZ</i>	Höhe über dem Boden, auf der sich die Mitte des Schilds bevorzugterweise befinden soll.
<i>SignMinZ, SignMaxZ</i>	Gibt den erlaubten Höhenbereich an, innerhalb welchem sich das Schild befinden darf. Falls nicht angegeben, wird keine Beschränkung vorgenommen. (Beispiel: Ist <i>SignCenterZ</i> = 2.25 und <i>SignMinZ</i> = 2 angegeben und die Höhe des Schilds beträgt 1 m, dann wird das Schild auf Höhe 2.5 m angebracht, um die Mindesthöhe nicht zu unterschreiten.)
<i>SignMinZN, SignMinZOff</i>	Gibt an, dass zusätzlich zur Beschränkung durch <i>SignMinZ</i> sichergestellt werden soll, dass unter dem Schild mindestens noch <i>SignMinZN</i> Schilder derselben Größe Platz finden müssen und das unterste dieser Schilder <i>SignMinZOff</i> Meter Abstand zum Boden hat.
<i>SignMaxZN, SignMaxZOff</i>	Gibt an, dass zusätzlich zur Beschränkung durch <i>SignMaxZ</i> sichergestellt werden soll, dass das Schild direkt unterhalb eines Schilds befestigt wird, dessen Positionierung über <i>SignMinZN</i> und <i>SignMinZOff</i> mit denselben Werten festgelegt wurde.
<i>Width, Height</i>	Breite und Höhe des Schilds. Es handelt sich um Multiplikatoren, d.h. die Schildgröße wird auf ein <i>Width</i> x <i>Height</i> -faches der Schild-Basisgröße festgelegt (Parameter <i>Size</i> der <i>SignGeneration</i>)
<i>PostYRel</i>	Horizontale Position der Schildpfosten. Die Angabe ist relativ zu verstehen, 0.0 bedeutet „am linken Schildrand“, 1.0 „am rechten Rand“.

<i>PostMinZ</i>	Höhe über dem Boden, an der der Pfosten anfängt. Kann auch negativ sein.
<i>PostMaxZRel</i>	Höhe, in der der Pfosten aufhört. Die Angabe ist relativ zur Schildposition, d.h. 0.0 bedeutet „am unteren Schildrand“, 1.0 „am oberen Rand“.
<i>Signs</i>	Schildtypen, für die dieses Format gilt. Es können jeweils einfache Schildtypen (z.B. "242") angegeben werden. Alternativ kann auch ein Schildtyp inklusive Variante angegeben werden (z.B. "274_01"). Das Format gilt dann nur, wenn sowohl Typ als auch Variante übereinstimmen.
<i>IsDefault</i>	Falls <i>true</i> , wird dieses Format für alle Schilder verwendet, auf die keines der anderen Formate passt. Diese Angabe sollte nur für ein Schildformat gesetzt werden. <i>Signs</i> kann in diesem Fall weggelassen werden.
<i>Backsides</i>	Optional: Texturen, die auf der Rückseite angezeigt werden sollen. (Standardmäßig wird eine einfarbige Rückseite verwendet.) Wenn diese Angabe vorhanden ist, müssen genauso viele Schildtypen aufgezählt werden wie unter <i>Signs</i> , da die Rückseiten paarweise zugeordnet werden.

Zuletzt folgt die Definition der *Positionen* von Schildern, durch die die Position eines Schilds am Mast verändert werden kann. Falls in der Schildspezifikation eine Position angegeben ist (z.B. "U_205"), wird das Format des Schilds durch diese Position zusätzlich modifiziert.

Grammatik 38 Schildposition

```

<SignPosition> →
  Position <Name> {
    (0..1)SignMinZ = <Höhe in m>;
    (0..1)SignCenterZ = <Höhe in m>;
    (0..1)SignMaxZ = <Höhe in m>;
    (0..1)Post = false;
  };

```

<i>SignMinZ</i> , <i>SignCenterZ</i> , <i>SignMaxZ</i>	Falls in der Position <i>SignMinZ</i> , <i>SignCenterZ</i> und/oder <i>SignMaxZ</i> angegeben werden, werden dadurch die entsprechenden Angaben des Schildformats überschrieben.
<i>Post</i>	Falls <i>false</i> , wird für dieses Schild kein Pfosten erzeugt, auch wenn im Schildformat Pfosten angegeben waren.

5.2.7 Andere Parameter

Die Parameter der Abschnitte **SCN ToDo**, **SCN ObjectManager**, **SCN CourseLaneCell** und **SCN H** sollten von SILAB-Anwendern normalerweise nicht geändert wer-

den, da diese komplexe Abhängigkeiten untereinander aufweisen und Änderungen die Geschwindigkeit der Simulation stark beeinträchtigen können.

5.3 Konfiguration von SCNXSGE („SCNXSGE Params“)

Die Darstellung von Strecke und Landschaft wird in der aktuellen Grafik-Engine SGE durch die DPUSCNXSGE durchgeführt.

Die Konfiguration der DPUSCNXSGE erfolgt nicht über DPU-Parameter, sondern über den Abschnitt „SILAB SCNXSGE“ der Versuchskonfiguration. Bei der Installation von SILAB wurde bereits eine Standard-Konfiguration eingerichtet, die an Ihr Simulator-System angepasst ist. Daher muss bei den meisten Simulationsfahrten lediglich die Standard-Konfiguration eingebunden werden, die im Lieferumfang von SILAB enthalten ist. Dazu muss in der Haupt-CFG-Datei auf oberster Ebene folgende Zeile hinzugefügt werden:

```
include scnx\SCNXSGE_600.cfg
```

Hiermit wird die Konfigurationsdatei

\SILAB\lib\configurations\SCNX\SCNXSGE_600.cfg eingebunden. Für besondere Anforderungen können Sie die Konfiguration der Visualisierung auch ändern. Sie sollten dabei nicht die Standard-Konfigurationsdatei ändern, sondern unterhalb der Include-Zeile von SCNXSGE_600.cfg den entsprechenden Konfigurationsabschnitt erneut aufführen. Die neu angegebenen Werte überschreiben dabei die Werte der Parameter aus der SCNXSGE_600.cfg.

Die Konfigurationsdatei ist wie folgt aufgebaut:

```
SILAB SCNXSGE {
    SCNXSGE CourseLaneCell {
        # Konfiguration der Straßen-Visualisierung, siehe Kap. 5.3.1
    };
    SCNXSGE CourseParams {
        # Weitere Parameter der Straßen-Visualisierung, siehe Kap. 5.3.2
    };
    SCNXSGE Ground {
        # Darstellung der Erdoberfläche, siehe Kap. 5.3.3
    };
    SCNXSGE Panorama {
        # Darstellung des Hintergrunds, siehe Kap. 5.3.4
    };
    SCNXSGE House {
        # Darstellung von Lichtern in Häusern, siehe Kap. 5.3.5
    };
    SCNXSGE SignGeneration {
        # Darstellung von Schildern, siehe Kap. 5.3.6
    };
    SCNXSGE Debug {
        # Interne Schalter zur Fehlersuche
    };
};
```

5.3.1 SCNXSGE CourseLaneCell

Zur Darstellung einer Straße in einem Streckenabschnitt (Course-Instanz) in der Simulation muss ihr Rand abgetastet und in einen Polygonzug umgewandelt werden. Mit den Parametern im Abschnitt CourseLaneCell kann die Abtastung der Straße konfiguriert werden:

<i>Show</i>	Sollen Straßen überhaupt in der Simulation dargestellt werden? (<i>true/false</i>)
<i>ds_min</i>	Minimaler Abstand zwischen zwei Punkten in der Abtastung des Straßenrands in Metern
<i>ds_max</i>	Maximaler Abstand zwischen zwei Punkten in der Abtastung des Straßenrands in Metern
<i>MaxErrorXY</i>	Maximaler Fehler bei der Abtastung der Straße in der Ebene (Maximal zulässige Abweichung der Straße von der Idealkurve aus der Vogelperspektive) in Metern
<i>MaxErrorSZ</i>	Maximal zulässige Abweichung der Straße von der Idealkurve im Höhenprofil in Metern (wirkt nur bei Steigungen)
<i>LaneBased</i>	Soll die spurbasierte Abtastung der Straßen verwendet werden? (<i>true/false</i>) Nur wenn hier <i>true</i> eingestellt ist, können Overlay-Texturen (Kap. 5.2.2) verwendet werden; die Einstellung <i>false</i> ist allerdings geringfügig schneller.

Bei niedrigeren Werten all dieser Parameter stimmt die dargestellte Straße genauer mit der Idealkurve, die durch den Lageplan definiert wird, überein. Gleichzeitig steigt aber auch der Aufwand für die Darstellung der Straße und möglicherweise sinkt die Bildwiederholrate der Simulation.

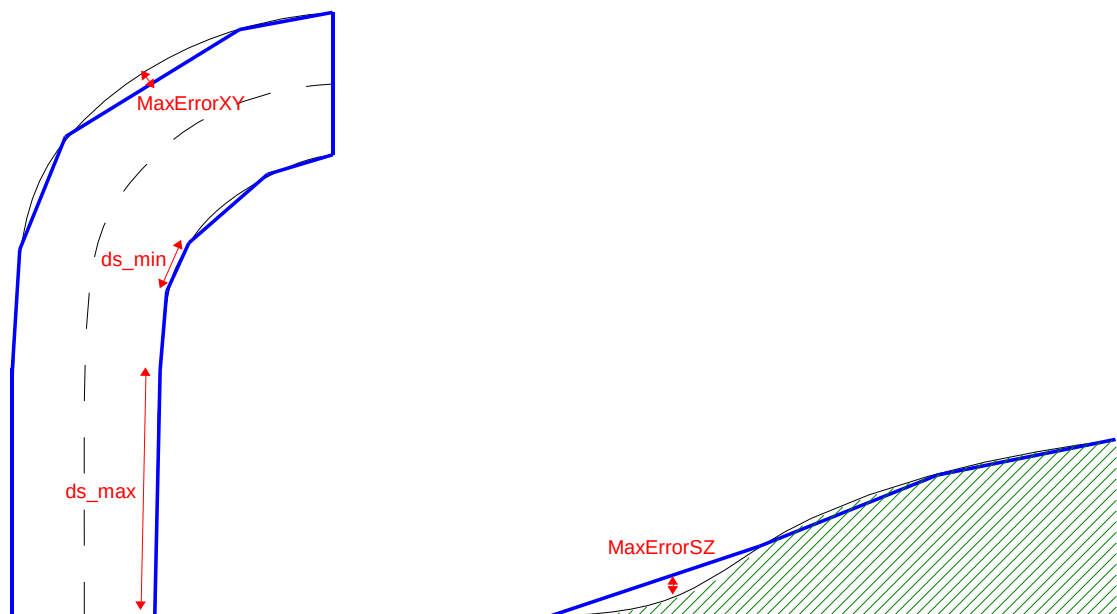


Abbildung 18 Parameter zur Abtastung der Straße

5.3.2 SCNXXSGE CourseParams

Im Abschnitt SCNXXSGE CourseParams kann die Texturgenerierung für die Straßenoberfläche von Course-Instanzen beeinflusst werden. Dieser Abschnitt sollte nicht verändert werden.

5.3.3 SCNXXSGE Ground

Im Abschnitt SCNXXSGE Ground kann die Darstellung der Erdoberfläche, also der umgebenden Landschaft, beeinflusst werden. Für fast alle Werte gilt, dass ein höherer Wert die Darstellungsqualität erhöht, aber auch die Bildwiederholrate reduziert. Die Standardeinstellungen sind so gewählt, dass auf üblicher Hardware die meisten Strecken gut aussehen und flüssig dargestellt werden.

Parameter zur Wahl der Darstellung:

<i>Show</i>	Soll die Landschaftsoberfläche überhaupt in der Simulation dargestellt werden? (true/false)
<i>MultiTexture</i>	Einstellung von Multitexturing, um die Landschaft zu verschönern. Mögliche Werte: 0 Kein Multitexturing, nur Basis-Grastextur 1 An Steigungen Felstextur anbringen 2 Landschaft auflockern durch Overlay-Textur (Felder usw.) 3 Sowohl Felstextur als auch Overlay-Textur verwenden
<i>TextureAnisotropy</i>	Einstellung der anisotropischen Texturfilterung für die Landschaft. Mögliche Werte: 0 (aus) bis 3 (maximal).

Parameter zur Anpassung an die Geschwindigkeit des Systems bzw. Anforderungen der Strecke:

<i>MaxRadiusX</i>	Maximale Entfernung in Metern zum Fahrer, bis zu der die Landschaft noch dargestellt wird. Standard: 2000 m. Durchschnittliche Anzahl von Dreiecken, die zur Darstellung der Landschaft verwendet werden. Standard 15000, muss bei sehr bergigen Landschaften evtl. auf bis zu 25000 erhöht werden.
<i>MaxRadiusY</i>	
<i>DesiredTri</i>	

5.3.4 SCNXXSGE Panorama

Im Abschnitt SCNXXSGE Panorama wird die Darstellung des Hintergrunds konfiguriert. Der Hintergrund wird in großer Entfernung zum Fahrer dargestellt und enthält Himmel, Wolken sowie ggf. Berge in der Ferne.

Folgende Parameter sind vorhanden:

<i>Show</i>	Soll das Panorama überhaupt dargestellt werden? (true/false)
<i>Texture</i>	Name der Textur, die für den Hintergrund verwendet werden soll. Es befinden sich mehrere Panorama-Texturen im Lieferumfang von SILAB (siehe Kap. 6.4). Standardwert ist „environment.surrounding.hills1“.
<i>GlowTexture</i>	Name der Textur, die für das Leuchten der Sonne verwendet werden soll. Standardwert ist „environment.surrounding.sunglow“.

5.3.5 SCNXSGE House

Im Abschnitt SCNXSGE House wird festgelegt, wann die Fenster der Häuser bei Nacht beleuchtet sind. (Um die Uhrzeit festzustellen, wird die Variable *TimeOfDay* der Umweltkontrolle verwendet (siehe Dokument „DPUEnvControl“).)

Die Lichter in Häusern werden nur angeschaltet, wenn es draußen dunkel ist (Umweltvariable *ArtificialLight*) und zu später Nachtstunde ausgeschaltet, wenn die Bewohner schlafen.

Für alle diese Parameter werden Wertebereiche angegeben, aus denen für einzelne Häuser dann zufällig und gleichverteilt ein Wert gezogen wird.

Folgende Parameter werden berücksichtigt:

<i>MinDarkness</i>	Dunkelheitswert (Umweltvariable <i>ArtificialLight</i>), bei dem das Licht frühestens angeschaltet wird
<i>MaxDarkness</i>	Dunkelheitswert (Umweltvariable <i>ArtificialLight</i>), bei dem das Licht spätestens angeschaltet wird
<i>FirstTimeOff</i>	Uhrzeit am Abend, zu der das Licht frühestens ausgeschaltet wird.
<i>LastTimeOff</i>	Uhrzeit am Abend, zu der das Licht spätestens ausgeschaltet wird.
<i>FirstTimeOn</i>	Uhrzeit am Morgen, zu der das Licht frühestens wieder angeschaltet wird.
<i>LastTimeOn</i>	Uhrzeit am Morgen, zu der das Licht spätestens wieder angeschaltet wird.

5.3.6 SCNXSGE SignGeneration

Die Darstellung von Schildern kann im Abschnitt SCNXSGE SignGeneration konfiguriert werden. Dies betrifft nur Schilder, die automatisch generiert werden (siehe Abschnitt 2.5.7). Folgende Parameter werden berücksichtigt:

<i>PostTexture</i>	Namen der Textur für die Schildpfosten. Standardwert ist "sign.sign".
<i>BackColor</i>	Farbe der Schildrückseite, im Format {Rot,Grün,Blau,Alpha}. Standardwert ist ein mittleres Grau {0.5, 0.5, 0.5, 1.0}.
<i>SaveSigns</i>	Falls <i>true</i> angegeben wird, werden die generierten Schilder als SGE-Objekte abgespeichert (unter \SILAB\data\sign). Standardwert ist <i>false</i> .

5.3.7 SCNXSGE Debug

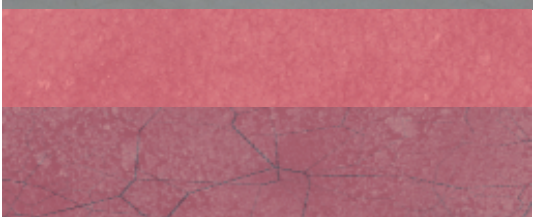
Diese Parameter werden nur intern zur Fehlersuche verwendet und sollten auf *false* eingestellt sein. Werden sie aktiviert, führen sie zu stark vergrößerten Log-Ausgaben und damit zu einer erhöhten Belastung der Simulatorrechner.

6 REFERENZ

6.1 Referenz der Standard-Straßenbeläge

Folgende Straßenbeläge, die für Spuren und Schulter verwendet werden können, sind vordefiniert:

Name	Bild
<i>Shoulder1</i>	
Tar, Tar1, <i>Tar1NoOverlay</i>	
Tar2, <i>Tar2NoOverlay</i>	
Tar7, <i>Tar7NoOverlay</i>	
Tar8, <i>Tar8NoOverlay</i>	
Tar9, <i>Tar9NoOverlay</i>	
Tar10, <i>Tar10NoOverlay</i>	
<i>Grass1</i>	
<i>Grass2</i>	

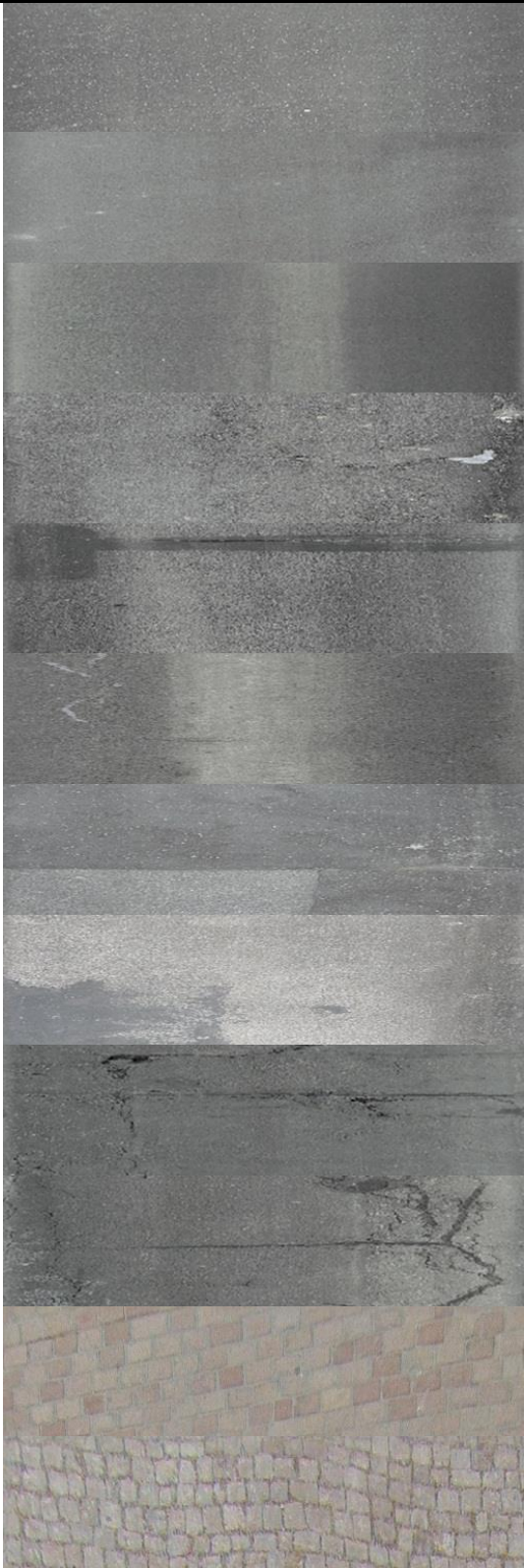
Name	Bild
Gravel1, <i>Gravel1NoOverlay</i>	
Gravel2, <i>Gravel2NoOverlay</i>	
Concrete1, <i>Concrete1NoOverlay</i>	
Concrete2, <i>Concrete2NoOverlay</i>	
Concrete3, <i>Concrete3NoOverlay</i>	
Concrete4, <i>Concrete4NoOverlay</i>	
BikewayNew1	
BikewayOld1	
GreenTarNew1	
GreenTarOld1	

Wenn für den Straßenquerschnitt zusätzlich eine Overlay-Textur verwendet wird (siehe Kap. 6.2), wird diese für alle Spuren der Straße verwendet. Ausgenommen davon sind lediglich Straßenbeläge, die in der obigen Tabelle *kursiv* gesetzt sind.

6.2 Referenz der Standard-Overlaytexturen

Der normale Straßenbelag (siehe Kap. 6.1) kann noch mit einer zusätzlichen Overlay-Textur überlagert werden. Diese Overlay-Texturen sind unabhängig von den Linienelementen und decken einen deutlich längeren Straßenabschnitt ab (z.B. 100 m) als die normalen Straßenbeläge (z.B. Tar1 mit Länge von nur 3 m). Dadurch ist in der Simulation praktisch keine Wiederholung der Textur mehr sichtbar und es ergibt sich ein wesentlich realistischerer Eindruck.

Folgende Overlays sind im SILAB-Lieferumfang enthalten:

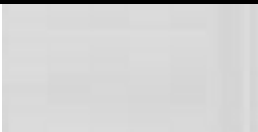
Name	Bild
TarNew1	
TarNew2	
TarNew3	
TarOld1	
TarOld2	
TarOld3	
TarOld4	
TarOld5	
TarBroken1	
TarBroken2	
Plaster1	
Plaster2	

6.3 Referenz der Standard-Straßenbebauungen

Zwischen die Spuren eines Course-Streckenelements können Straßenbebauungen platziert werden. Diese werden in drei Bauungsarten unterteilt:

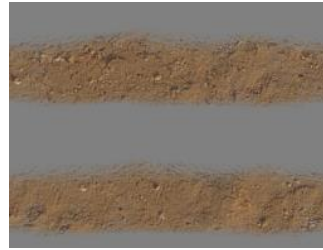
- Fahrbahnmarkierungen (Texture) werden direkt auf der Höhe der Straße angebracht.
- Objektbänder (Strip) werden parallel zur Straße aufgestellt.
- Einzelne Objekte (Object) werden entlang der Straße aufgestellt.

Folgende Straßenbebauungen sind vordefiniert:

Bebauung	Typ	Bild
Line	Texture	
StoneBorderLeft (Bankett für linken Straßenrand)	Texture	
StoneBorderRight (Bankett für rechten Straßenrand)	Texture	
YellowLine	Texture	
YellowLine2	Texture	
BridgeRailing	Strip	
BridgeBottom	Strip	
ConcreteSeparator (Trenner zwischen Spuren)	Strip	

Dirt
(Fahrbahnverschmutzung)

Strip



Forrest

Strip



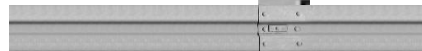
Guardrail

Strip



DoubleGuardrail

Strip



Hedge2

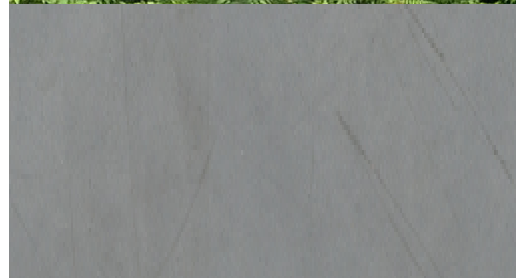
Strip



TunnelInside_2

Strip

(Innenwand eines 2-
spurigen Tunnels)



TunnelTop_8

Strip

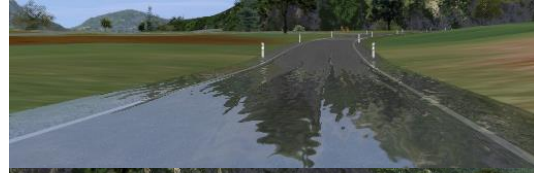
(Tunneldecke aus Fels)



WaterPath Strip



WaterPathStrong Strip



WaterPuddles Strip



WetStreet Strip



MoistStreet Strip



ReflectionPosts Object



ReflectionPosts_Night Object
(leuchtet bei Dunkelheit)



TunnelLight Object
(Tunnelbeleuchtung: in die Decke integrierte Straßenlampen)



TunnelVentilation Object
(Belüftung des Tunnels)



Weitere Straßenbebauungen können im Konfigurationsabschnitt **SCN Params** definiert werden (siehe Kap. 5.2.2).

6.4 Referenz der Panorama-Texturen

Eine Panorama-Textur wird von der Visualisierung SCNXSGE in großer Ferne gezeichnet und stellt in der Simulation Himmel, Wolken und die Landschaft in der Ferne dar. Bei SILAB werden folgende Panorama-Texturen mitgeliefert:



Abbildung 19 Textur environment.surrounding.hills1



Abbildung 20 Textur environment.surrounding.hills1overcast



Abbildung 21 Textur environment.surrounding.hills1rainy



Abbildung 22 Textur environment.surrounding.mountains1



Abbildung 23 Textur environment.surrounding.mountains1overcast

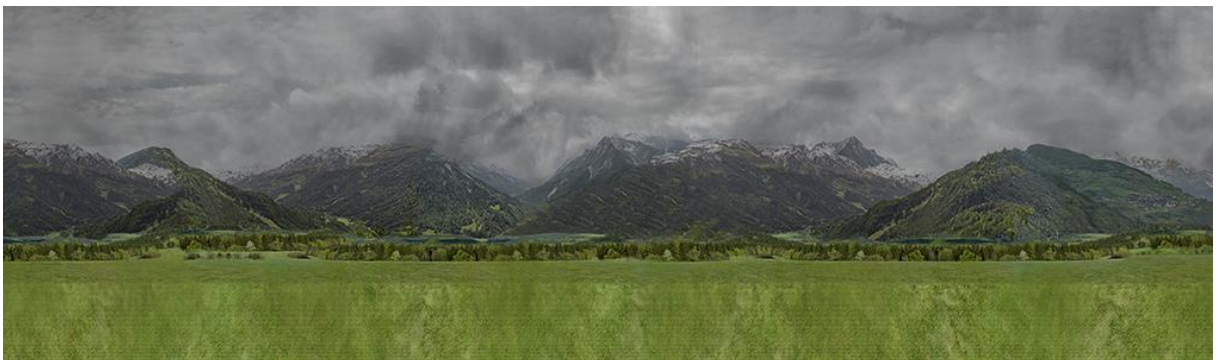


Abbildung 24 Textur environment.surrounding.mountains1rainy



Abbildung 25 Textur environment.surrounding.mountains3

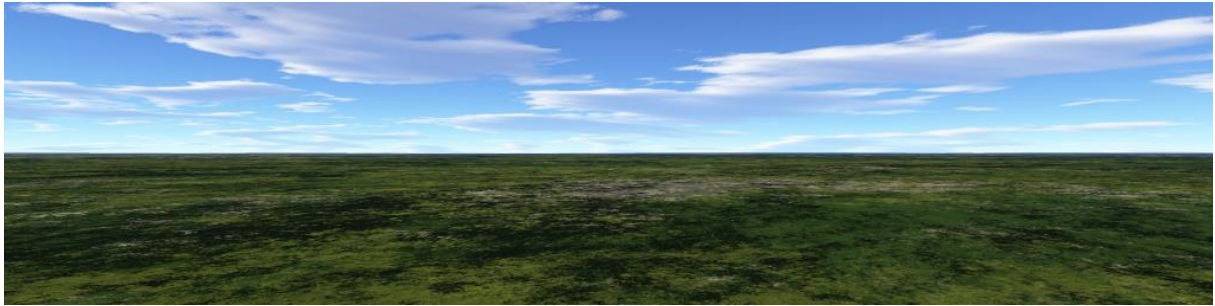


Abbildung 26 Textur environment.surrounding.flat1

Beispiel: Um das Panorama *mountains1* zu verwenden, fügen Sie auf der obersten Ebene der Konfigurationsdatei folgende Anweisungen hinzu:

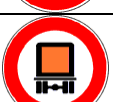
```
SILAB SCNXSGE
{
    SCNXSGE Panorama
    {
        Texture = environment.surrounding.mountains1;
    };
};
```

Hinweis: Detaillierte Informationen zur Konfiguration der Panoramen finden Sie in Kap. 5.3.4.

6.5 Referenz der Verkehrsschilder/-regeln

Die folgende Tabelle zählt alle Verkehrsschilder, die in SILAB eine Bedeutung als Verkehrsregel (siehe Kap. 2.5.9) besitzen, auf.

Typ		Kategorie	Bedeutung	Betroffene Fahrzeugtypen	Parameter
102		right_of_way	right_left	all	
205		right_of_way	yield	all	
206		right_of_way	stop_yield	all	
301		right_of_way	priority	all	
306		right_of_way	priority	all	
307		right_of_way	priority_end	all	
350_10, 350_20		right_of_way	pedestrian_crossing	all	

134_10, 134_20		right of way	pedestrian_crossing	all	
242		access	no_entry_start	all	
243		access	no_entry_end	all	
250		access	no_entry_start	all	
251		access	no_entry_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus	
252		access	no_entry_start	dangerous_goods_vehicle	
253		access	no_entry_start	goods_vehicle, dangerous_goods_vehicle	VehicleWeight = 3.5
254		access	no_entry_start	bicycle	
255		access	no_entry_start	motor_cycle	
261_35	 3,5 t	access	no_entry_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 3.5
261_55	s. 261_35	access	no_entry_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 5.5
261_65	s. 261_35	access	no_entry_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 6.5
262_35	s. 262_55	access	no_entry_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 3.5
262_55		access	no_entry_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 5.5
262_65	s. 262_55	access	no_entry_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 6.5
263		access	no_entry_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 8.0

267		access	no_entry_start	all	
274_010	s. 274_060	speed_limit	v_max_start	all	v = 10.0
274_01_030	s. 274_060	speed_limit	v_max_start	all	v = 30.0
274_020	s. 274_060	speed_limit	v_max_start	all	v = 20.0
274_02_030	s. 274_060	speed_limit	v_max_end	all	v = 30.0
274_030	s. 274_060	speed_limit	v_max_start	all	v = 30.0
274_040	s. 274_060	speed_limit	v_max_start	all	v = 40.0
274_050	s. 274_060	speed_limit	v_max_start	all	v = 50.0
274_060		speed_limit	v_max_start	all	v = 60.0
274_070	s. 274_060	speed_limit	v_max_start	all	v = 70.0
274_080	s. 274_060	speed_limit	v_max_start	all	v = 80.0
274_090	s. 274_060	speed_limit	v_max_start	all	v = 90.0
274_100	s. 274_060	speed_limit	v_max_start	all	v = 100.0
274_110	s. 274_060	speed_limit	v_max_start	all	v = 110.0
274_120	s. 274_060	speed_limit	v_max_start	all	v = 120.0
274_130	s. 274_060	speed_limit	v_max_start	all	v = 130.0
274_140	s. 274_060	speed_limit	v_max_start	all	v = 140.0
274_150	s. 274_060	speed_limit	v_max_start	all	v = 150.0
274_160	s. 274_060	speed_limit	v_max_start	all	v = 160.0
278_010	s. 278_060	speed_limit	v_max_end	all	v = 10.0
278_020	s. 278_060	speed_limit	v_max_end	all	v = 20.0
278_030	s. 278_060	speed_limit	v_max_end	all	v = 30.0
278_040	s. 278_060	speed_limit	v_max_end	all	v = 40.0
278_050	s. 278_060	speed_limit	v_max_end	all	v = 50.0
278_060		speed_limit	v_max_end	all	v = 60.0
278_070	s. 278_060	speed_limit	v_max_end	all	v = 70.0
278_080	s. 278_060	speed_limit	v_max_end	all	v = 80.0
278_090	s. 278_060	speed_limit	v_max_end	all	v = 90.0
278_100	s. 278_060	speed_limit	v_max_end	all	v = 100.0
278_110	s. 278_060	speed_limit	v_max_end	all	v = 110.0
278_120	s. 278_060	speed_limit	v_max_end	all	v = 120.0
278_130	s. 278_060	speed_limit	v_max_end	all	v = 130.0
278_140	s. 278_060	speed_limit	v_max_end	all	v = 140.0
278_150	s. 278_060	speed_limit	v_max_end	all	v = 150.0
278_160	s. 278_060	speed_limit	v_max_end	all	v = 160.0
275_010	s. 275_030	speed_limit	v_min_start	all	v = 10.0
275_020	s. 275_030	speed_limit	v_min_start	all	v = 20.0
275_030		speed_limit	v_min_start	all	v = 30.0
275_040	s. 275_030	speed_limit	v_min_start	all	v = 40.0
275_050	s. 275_030	speed_limit	v_min_start	all	v = 50.0
275_060	s. 275_030	speed_limit	v_min_start	all	v = 60.0
275_070	s. 275_030	speed_limit	v_min_start	all	v = 70.0

275_080	s. 275_030	speed_limit	v_min_start	all	v = 80.0
275_090	s. 275_030	speed_limit	v_min_start	all	v = 90.0
275_100	s. 275_030	speed_limit	v_min_start	all	v = 100.0
275_110	s. 275_030	speed_limit	v_min_start	all	v = 110.0
275_120	s. 275_030	speed_limit	v_min_start	all	v = 120.0
275_130	s. 275_030	speed_limit	v_min_start	all	v = 130.0
275_140	s. 275_030	speed_limit	v_min_start	all	v = 140.0
275_150	s. 275_030	speed_limit	v_min_start	all	v = 150.0
275_160	s. 275_030	speed_limit	v_min_start	all	v = 160.0
279_010	s. 279_030	speed_limit	v_min_end	all	v = 10.0
279_020	s. 279_030	speed_limit	v_min_end	all	v = 20.0
279_030		speed_limit	v_min_end	all	v = 30.0
279_040	s. 279_030	speed_limit	v_min_end	all	v = 40.0
279_050	s. 279_030	speed_limit	v_min_end	all	v = 50.0
279_060	s. 279_030	speed_limit	v_min_end	all	v = 60.0
279_070	s. 279_030	speed_limit	v_min_end	all	v = 70.0
279_080	s. 279_030	speed_limit	v_min_end	all	v = 80.0
279_090	s. 279_030	speed_limit	v_min_end	all	v = 90.0
279_100	s. 279_030	speed_limit	v_min_end	all	v = 100.0
279_110	s. 279_030	speed_limit	v_min_end	all	v = 110.0
279_120	s. 279_030	speed_limit	v_min_end	all	v = 120.0
279_130	s. 279_030	speed_limit	v_min_end	all	v = 130.0
279_140	s. 279_030	speed_limit	v_min_end	all	v = 140.0
279_150	s. 279_030	speed_limit	v_min_end	all	v = 150.0
279_160	s. 279_030	speed_limit	v_min_end	all	v = 160.0
276		overtaking	no_overtaking_start	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus, motor_cycle	
277		overtaking	no_overtaking_start	goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 3.5
280		overtaking	no_overtaking_end	passenger_car, goods_vehicle, dangerous_goods_vehicle, bus, motor_cycle	
281		overtaking	no_overtaking_end	goods_vehicle, dangerous_goods_vehicle, bus	VehicleWeight = 3.5
282		speed_limit, overtaking	v_max_end, no_overtaking_end	all	
310		speed_limit, overtaking	v_max_start, no_overtaking_start	all	v = 50.0
311		speed_limit, overtaking	v_max_end, no_overtaking_end	all	v = 50.0

Zusätzlich zu diesen Schildern sind im Lieferumfang von SILAB zahlreiche weitere Schilder enthalten. Für diese ist keine Bedeutung als Verkehrsregel hinterlegt. Da über 600 solcher Schilder mitgeliefert werden, werden diese in dieser Referenz nicht vollständig aufgezählt.

Sie finden das richtige Schild für Ihre Strecke, indem Sie das Verzeichnis „SILAB\lib\graphics\default\texture\sign“ durchsuchen. Alle hier vorhandenen Texturdateien (*.tga oder *.png) können als Schild verwendet werden (siehe Kap. 2.5.7). Einen schnellen Überblick über die Dateien erhalten Sie mit der Miniaturansicht des Windows-Explorers oder mit einem Bildbetrachtungsprogramm mit Thumbnail-Funktion (z.B. IrfanView).